

UNIVERSITÀ CA' FOSCARI VENEZIA

Laurea Triennale in Informatica

RETI DI CALCOLATORI

Autore

Giovanni Quacchia

Anno Accademico

2025–2026

Indice

Prefazione	1
I Principi delle Reti di Calcolatori	2
1 Introduzione a Internet	3
2 Livello Fisico	5
2.1 Introduzione al livello fisico	5
2.1.1 Nozioni di fisica	5
2.1.2 Attenuazione e rumore	6
2.1.3 Teorema di Nyquist	8
2.1.4 Teorema di Shannon	9
3 Livello Data Link	10
3.1 Introduzione al livello Data Link	10
3.1.1 Notazione: Time-sequence diagrams (TSD)	10
3.1.2 Framing	11
3.1.3 Recupero dagli errori	11
3.1.4 Gestione Link Errors	12
3.1.5 Correzione degli errori	13
3.2 Reliable transport	16
3.2.1 Go-back-n	16
3.2.2 Selective repeat	18
3.2.3 Piggybacking e deferred ACKs	19
3.2.4 Dimensione della finestra - analisi personale	19
3.3 Condivisione di risorse	20
3.3.1 Topologie di rete	20
3.3.2 Media Access Control	23
3.3.3 Random Access: ALOHA e CSMA	24
4 Livello Rete	27
4.1 Introduzione al livello di rete	27
4.1.1 Livello di rete Datagram-oriented	27
4.1.2 Indirizzamento ed eterogeneità	28
4.1.3 Frammentazione	28
4.1.4 Hot Potato	28
4.1.5 DV - Distance Vector Routing	30
4.2 Limiti di convergenza in DVR	33
4.2.1 Count-to-Infinity	33
4.2.2 Split Horizon e Poison Reverse	34
4.2.3 Link State Routing	37

4.2.4	Analisi e confronto	39
5	Livello di Trasporto	40
5.1	Transport Layer	40
5.1.1	Modello connectionless	40
5.1.2	Modello connection-oriented	40
5.1.3	Three Way Handshake	42
5.1.4	Reliable Data Transfer	42
5.1.5	Rilascio della connessione	44
6	Livello Applicativo	46
6.1	Application Layer	46
6.1.1	Naming System	46
7	Sicurezza di Rete	47
7.1	Sicurezza di Rete	47
7.1.1	Servizi di sicurezza	47
7.1.2	Modello di sicurezza	48
7.1.3	Fondamenti di crittografia	49
7.1.4	Funzioni Hash	50
7.2	Autenticazione e Cifratura Simmetrica	52
7.2.1	Meccanismi Challenge-Response	52
7.2.2	Cifratura a chiave simmetrica	53
7.2.3	Encrypt-then-MAC	55
7.3	Cifratura asimmetrica - PKE	56
7.3.1	Aritmetica modulare	56
7.3.2	Diffie-Hellman	56
7.3.3	PKE - Public Key Encryption	57
7.3.4	Digital Signatures	58
7.3.5	Public Key Crypto Performance	59
7.3.6	RSA	60
7.4	PKI - Public Key Infrastructure	61
7.4.1	Distribuzione delle chiavi pubbliche	61
7.4.2	Web of Trust	61
7.4.3	PKI: Architettura e dettagli	62
7.4.4	PKE e PKI in pratica	65
II	Protocolli e Implementazioni	66
8	Protocolli a livello Applicativo	67
8.1	DNS - Domain Name System	67
8.1.1	Risoluzione DNS	68
8.1.2	DNS - Dettagli del protocollo	69
8.1.3	Reverse DNS	71
8.1.4	Gestione del traffico e performance	72
8.1.5	Sicurezza - slide extra	73

8.2	HTTP - HyperText Transfer Protocol	75
8.2.1	URI - Uniform Resource Identifier	75
8.2.2	HTML - HyperText Markup Language	75
8.2.3	HTTP - Dettagli del protocollo	76
8.2.4	Server Proxy	78
8.3	SMTP - Simple Mail Transfer Protocol	80
8.3.1	Formato mail	81
8.3.2	SMTP - Dettagli del protocollo	82
8.3.3	Relaying	82
8.3.4	POP3 - Post Office Protocol	84
8.4	Tecniche antispam	85
8.4.1	Tecniche IP-based	85
8.4.2	Tecniche DNS e Crypto-based	87
8.4.3	DKIM - DomainKeys Identified Mail	88
8.4.4	DMARC: Policy di autenticazione e reporting	89
8.4.5	Risultato finale	90
9	Protocolli di Trasporto e Sicurezza	91
9.1	TCP - UDP	91
9.1.1	UDP - User Datagram Protocol	91
9.1.2	TCP - Transmission Control Protocol	91
9.1.3	Reliability	94
9.1.4	Sender Side	95
9.1.5	Receiver Side	98
9.2	Gestione della congestione	99
9.2.1	Congestion Window	99
9.2.2	Slow Start Alghorithm	99
9.2.3	Eventi di congestione	99
9.2.4	Socket di rete	100
9.3	TLS - Transport Layer Security	102
9.3.1	SSL - TLS	102
9.3.2	TLS handshake protocol	102
9.3.3	TLS record protocol	103
9.3.4	Socket TLS	105
10	Protocolli di Rete	106
10.1	IPv4	106
10.1.1	Header IPv4	108
10.1.2	Indirizzi pubblici e privati - NAT	109
10.2	IPv6	112
10.2.1	ICMP	112
10.2.2	IPv6 - Dettagli del protocollo	112
10.2.3	Header IPv6	114
10.2.4	ICMPv6	115
10.2.5	Intradomain Routing - OSPF	115
10.3	BGP - Border Gateway Protocol	117
10.3.1	Autonomous Systems (AS)	117

10.3.2	Propagazione dei prefissi BGP	118
10.3.3	BGP - Dettagli del protocollo	119
11	Standard del Livello Data Link e Fisico	121
11.1	Ethernet - IEEE 802.3	121
11.1.1	Livello fisico	121
11.1.2	Livello Data Link	121
11.1.3	ARP - Address Resolution Protocol	124
11.1.4	DHCP - Dynamic Host Configuration Protocol	124
11.2	Wi-Fi - IEEE 802.11	126
11.2.1	Livello fisico	126
11.2.2	Livello MAC	126
11.2.3	Access Control	128
11.2.4	Meccanismo RTS/CTS	130
11.2.5	Stima bit-rate	131
III	Appendici	132

Prefazione

Questo documento è stato realizzato per fornire una dispensa a coloro che si stanno approcciando allo studio delle reti di calcolatori. Si tratta di una trascrizione dei concetti fondamentali trattati nel corso di *Reti di Calcolatori* tenuto dal Prof. *Leonardo Maccari* presso l'Università Ca' Foscari di Venezia nell'anno accademico 2025/2026.

Gli appunti sono tratti da lezioni frontali, slide, libri di testo del corso e analisi personali, che spero siano corrette e che possano essere utili per approfondire i temi trattati e chiarire eventuali dubbi. Le immagini e gli schemi presenti nel documento sono stati creati personalmente o sono tratti dalle slide del corso e dai libri di testo consigliati. Tali contenuti sono utilizzati esclusivamente a scopo didattico e illustrativo, restando di proprietà dei rispettivi autori.

Inoltre, ci tengo a precisare che il documento presentato non sostituisce lo studio dei testi consigliati, né tantomeno la frequenza delle lezioni, che permettono di comprendere appieno gli argomenti affrontati durante il corso. Spero che le spiegazioni siano chiare e che non siano presenti errori dovuti a interpretazioni personali, anche se non garantisco nulla.

Questo ebook è disponibile anche al link [Github](#) e può essere condiviso liberamente per usi non commerciali, purché non vengano apportate modifiche al contenuto originale e venga sempre citata la fonte.

Parte I

Principi delle Reti di Calcolatori

1 Introduzione a Internet

Reti a commutazione di circuito (circuit-switched): era necessario connettere tutti con tutti, quindi con $n * (n - 1)/2$ cavi.

Reti gerarchiche: nascono gli switch (accentratori). Colpire un nodo centrale causa un fallimento di tutta la rete.

Basata su statistical multiplexing: è improbabile che tutti vogliano comunicare contemporaneamente.

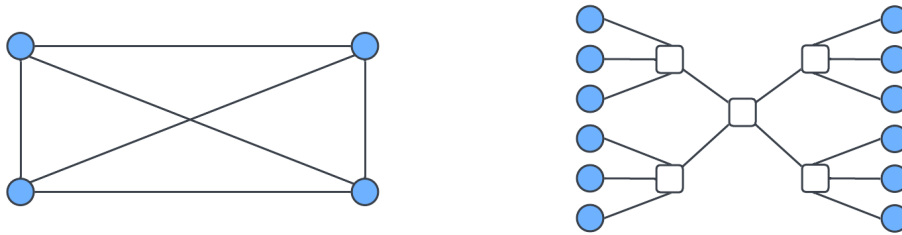


Figura 1.1: Rete magliata - Rete gerarchica

Reti a commutazione di pacchetto (packet switches): non usano circuiti fisici e ogni informazione è codificata in bit e scomposta in pacchetti.

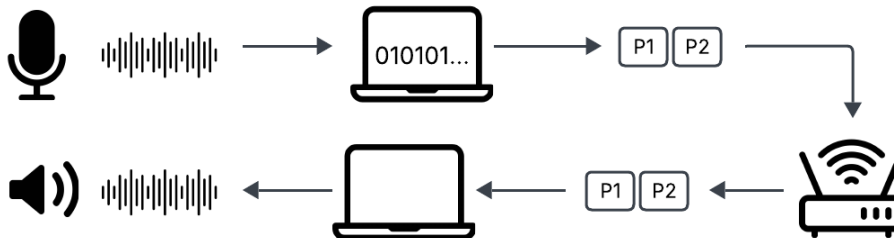


Figura 1.2: Rete a commutazione di pacchetto

- + Ogni link può usare una tecnologia diversa
- + Dati digitalizzati possono trasportare qualsiasi tipo di informazione
- + In caso di fallimento, i router possono trovare un percorso alternativo

Internet è quindi più robusto e non necessita di un coordinatore globale. È interoperabile grazie a layering, standard e protocolli

Gli **standard/protocolli** sono documenti tecnici che descrivono dettagli di comunicazione.

Gli standard sono generati da enti importanti e applicati ai livelli bassi (freq. radio e potenza (watt))

Internet layers

TCP/IP	ISO-OSI	Descrizione
Application	Application	comportamento applicazione (HTML)
	Presentation	come visualizzare i dati (CSS)
	Session	Gestione sessioni (cookies)
Transport	Transport	Gestione errori: ordinare i pacchetti, ritrasmetterli se persi
Internet	Network	Comunicazione a più link (routing)
	Data Link	Politiche di accesso, evita collisioni
Link	Physical	Comunicazione a un singolo link

IETF, che produce gli standard, usa TCP/IP, mentre IEEE usa ISO-OSI in quanto necessita di più dettagli per i livelli bassi, senza preoccuparsi dei livelli superiori.

Quando si sviluppa un protocollo/standard, basta preoccuparsi solo del livello in cui si sta operando e fornire un'interfaccia standard al livello superiore.

Encapsulation: Ogni livello aggiunge un header con le proprie informazioni scendendo di livello (aggiunge overhead).

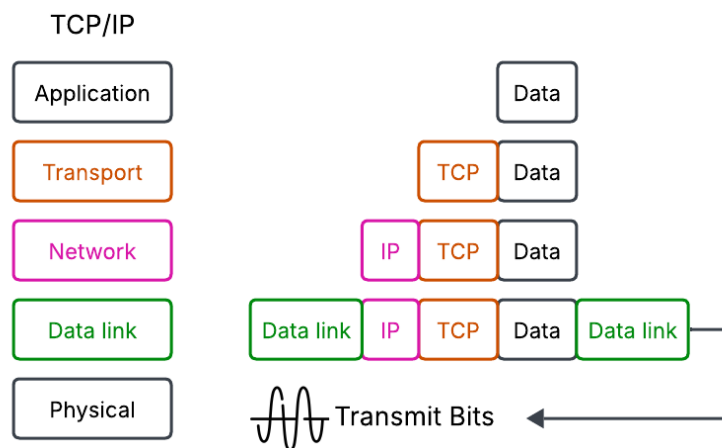


Figura 1.3: Encapsulation

Inoltre, i pacchetti passano attraverso router intermediari che possono leggere informazioni, quindi è necessario includere delle misure di sicurezza nei protocolli che usiamo.

2 Livello Fisico

2.1 Introduzione al livello fisico



bit-rate: velocità di un canale di comunicazione (bit/s)

Prefisso	milli	chilo	mega	Giga
Simbolo	m	k	M	G
Notazione	10^{-3}	10^3	10^6	10^9

Tabella 2.1: Prefissi del S.I.

Nelle comunicazioni (bit-rate) si usa il sistema decimale (1 kbps = 8000 bps), mentre per la memoria si usa il sistema binario (1 KB = 1024 B = 8192 bit)

2.1.1 Nozioni di fisica

Grandezza	Formula	Udm
Lavoro	$L = F s$	Joule: Newton * metri
Energia	$KE = \frac{1}{2} m v^2$	Joule: Newton * metri
Potenza	$P = \frac{\text{Energia}}{\text{tempo}}$	Watt: J/s
Corrente	$\frac{Q \text{ (carica elettrica)}}{\text{tempo}}$	Ampère: Coulombe / s

Lavoro: Risultato dell'applicazione di una forza che provoca lo spostamento di un corpo nella sua direzione. Il lavoro causa una variazione di energia.

Il lavoro è quindi la differenza tra KE a velocità v e KE a velocità 0

Potenza: Quanto velocemente un sistema fornisce energia

Corrente elettrica

Un elettrone è una particella con carica negativa e il generatore di corrente fornisce energia facendo muovere gli elettroni nel cavo.

Gli elettroni non vanno dritti, sbattono sul cavo trasferendo energia al materiale che è trasformata in calore (**resistenza**)

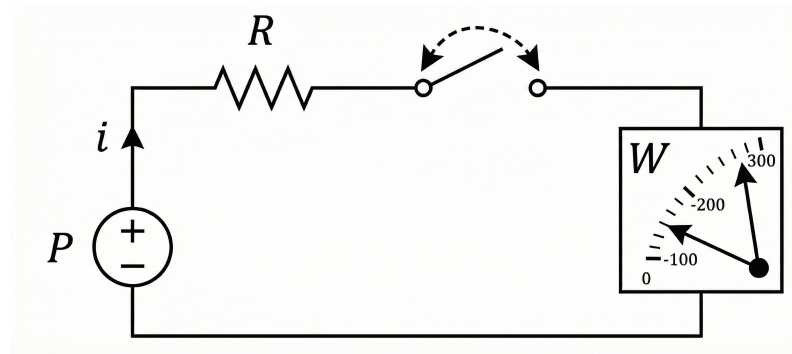


Figura 2.1: Circuito elettrico semplice

Il generatore fornisce potenza P al circuito, che è trasformata in corrente elettrica. La resistenza dissipa la potenza, scaldandosi.



Sensibilità: Quantità minima di potenza che il sensore deve ricevere per rilevare la presenza di corrente.

Se accendiamo e spegniamo lo switch, trasmettiamo un segnale.

2.1.2 Attenuazione e rumore

Un cavo ha una certa resistenza.



Attenuazione: perdita di potenza (intensità del segnale) che avviene lungo un cavo, principalmente a causa della sua resistenza

cavo + lungo = + resistenza = + perdita di segnale

Stesso segnale con ampiezza minore → serve un sensore con una sensibilità maggiore.

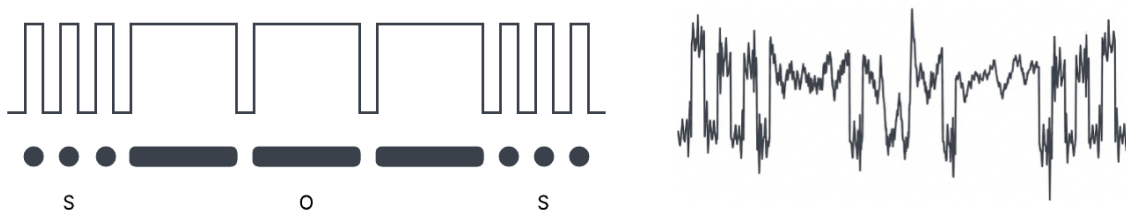


Figura 2.2: Esempi di segnali Morse con e senza rumore

L'attenuazione dipende dal tipo di cavo, lunghezza e frequenza di comunicazione. Un cavo di rame può ridurre la potenza del segnale di $\frac{1}{25}$ per km.



Delay: Il tempo che passa da quando il segnale parte dal trasmettitore a quando inizia a essere ricevuto dal ricevitore.

Gli elettroni non sono mai completamente fermi, anche quando non c'è corrente nel circuito.

Rumore: segnale di disturbo dovuto al movimento casuale degli elettroni (rumore termico) o interferenze di altri dispositivi.



SNR (Signal Noise Ratio - Rapporto Segnale/Rumore)

$$\frac{\text{Potenza del segnale ricevuto}}{\text{Potenza del rumore al ricevitore}}$$

Maggiore è il rapporto, meglio si riconosce il segnale inviato (consente un bit-rate più alto)



Sincronizzazione: accordo tra trasmettitore e ricevitore su quando inizia e finisce ogni simbolo (durata del simbolo)

Modulazione

Il segnale digitale non è inviato direttamente perché non si trasmette bene nel mondo reale a causa delle interferenze; per questo usiamo la modulazione.



Modulazione:

- Segnale portante (analogico): senoide con frequenza f_c (Hertz)
- Segnale modulante (digitale): segnale da trasmettere



Modulazione di frequenza (FM):

- Portante $p(t) = A \cos(2\pi f_p t)$, segnale digitale $m(t)$ da trasmettere
- Possiamo modificare la freq. della portante incrementandola di un valore Δ

$$\begin{cases} f'_p = f_p & \text{if } m(t) = 0 \\ f'_p = f_p(1 + \Delta) & \text{if } m(t) = 1 \end{cases}$$

- In questo modo possiamo comunicare un segnale digitale binario

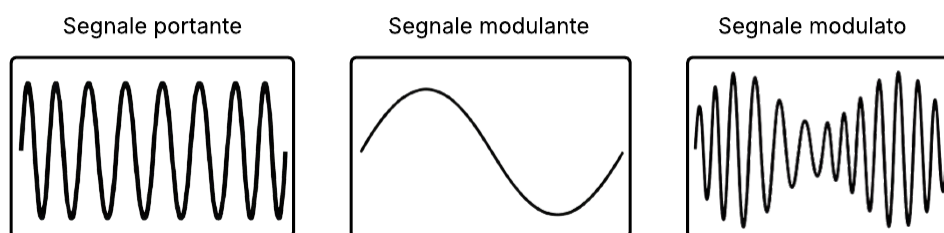


Figura 2.3: Modulazione di frequenza

In un circuito si usa un generatore AC per creare il segnale portante. In una comunicazione wireless si usa un'onda elettromagnetica che oscilla.

Baud rate: simboli trasmessi al secondo

Se $f_p = 4 \rightarrow 4$ oscillazioni al secondo e un simbolo è determinato da 4 oscillazioni $\rightarrow$ baud rate = 1

$$\text{bit-rate} = \text{baud-rate} * \text{bits-per-symbol}$$

Per aumentare il bit-rate posso:

1. Ridurre la durata di un simbolo per aumentare il baud-rate: Usare solo 2 oscillazioni per simbolo

SNR peggiore: più difficile distinguere il simbolo dal rumore

2. Codificare più bit in un simbolo: usare più livelli M

$$\begin{cases} f'_p = f_p & \text{if } m(t) = 00 \\ f'_p = f_p(1 + \Delta) & \text{if } m(t) = 01 \\ f'_p = f_p(1 + 2\Delta) & \text{if } m(t) = 10 \\ f'_p = f_p(1 + 3\Delta) & \text{if } m(t) = 11 \end{cases}$$

Stesso baud rate, ma ogni simbolo trasmette 2 bit. Il ricevitore deve distinguere tra le diverse frequenze.



Banda: Intervallo di frequenze usato per la comunicazione

Per aumentare il bit-rate, bisogna aumentare la banda

Non si può usare la stessa frequenza per più comunicazioni nello stesso momento. Le frequenze sono allocate per legge.

2.1.3 Teorema di Nyquist

Capacità massima teorica di trasmissione ideale, senza rumore, in funzione della banda e del numero di livelli di segnale M (2^n).

$$C = 2B \log_2(M) \text{ bps}$$

Esercizi

Frequenza [2401 - 2441 MHz], M = 4

$$\begin{aligned} B &= (2441 - 2401) = 40 \text{ MHz} \\ M = 4 &\rightarrow C = 2 \cdot 40 \cdot \log_2(4) = 160 \text{ Mb/s} \end{aligned}$$

Frequenza [2401 - 2441 MHz], $C \geq 300 \text{ Mb/s}$

$$C = 2B \log_2(M) \geq 300 \text{ Mb/s} \iff M \geq 2^{\frac{C}{2B}} = 2^{\frac{300}{80}} = 2^{3.75} \rightarrow M = 16$$

Analisi personale

bit-rate = baud-rate * bits-per-symbol

2B $\rightarrow$ baud-rate: Servono almeno 2 campionamenti per ogni oscillazione della sinusoide (min e max) (th di Fourier)

$\log_2(M)$: bits-per-symbol. $M = 8$, servono 3 bit (000-111)

2.1.4 Teorema di Shannon

$$C = B \log_2\left(1 + \frac{S}{N}\right)$$

Capacità massima teorica di un canale di comunicazione reale, con rumore. Trasmettere a velocità maggiore causa errori di decodifica.

Se si conosce SNR, non si può avere un numero arbitrario di livelli M . M va scelto in modo che $C_S \geq C_N$

Esercizio

$S = 100 \text{ mW}$, $N = 0.0001 \text{ mW}$, Attenuazione: $S/25$ ad ogni km, $B = 2.2 \text{ Mhz}$

$$S_2 = 100/25^2 = 0.16 \text{ mW} =$$

$$C_2 = 2.2 * \log_2(1 + 0.16/0.0001) = 23 \text{ Mb/s}$$

$$M \leq 2^{C/2B} = 2^{23/2*2.2} = 2^{5.23} \rightarrow M = 32$$

Se usassimo $M = 6 \rightarrow C_N = 26.4 \text{ Mb/s} > C_S$ e avremmo errori di decodifica



Bit Error Ratio (BER): probabilità che un bit ricevuto sia sbagliato

$$\frac{\text{Errors}}{\text{Received Bits}}$$

Nell'esempio: $\frac{26.4-23}{26.4} = \frac{3.4}{26.4} = 13\%$

3 Livello Data Link

3.1 Introduzione al livello Data Link

3.1.1 Notazione: Time-sequence diagrams (TSD)

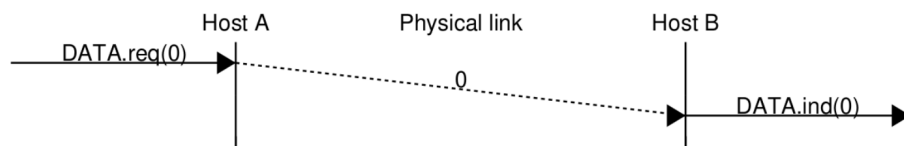


Figura 3.1: Schema TSD [1]

Errori di comunicazione (rumore e distorsione)

Bit flipping

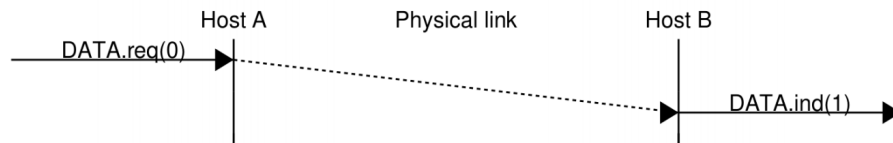


Figura 3.2: Bit flipping [1]

De-synchronization: il clock si disallinea e il ricevitore legge più o meno bit di quelli inviati.

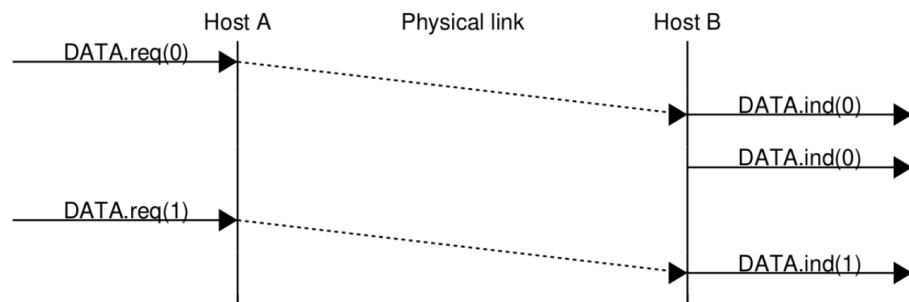


Figura 3.3: De-synchronization [1]

Per sincronizzare mittente e destinatario, il segnale cambia ad ogni simbolo (low $\iff$ high), semplificando la sincronizzazione.

3.1.2 Framing

PDU a livello data link.



Frame: sequenza di bit con lunghezza massima e sintassi e struttura specifiche

Come determinare l'inizio e la fine di un frame?

1. Livello fisico sincronizzato: mittente e destinatario mantengono il clock sempre sincronizzato. Le radio trasmettono sempre
2. Bit-stuffing: sequenza binaria che delimita il frame

Se la sequenza appare nei dati, si inserisce un bit extra per evitare ambiguità

3. Mix: bit-stuffing e indicazione della lunghezza del frame nell'header, così il ricevente sa esattamente quanti bit leggere



Protocol overhead: parte della capacità del link utilizzata per garantire il corretto funzionamento della comunicazione (preambolo, ritrasmissione, ACK)



Throughput: capacità effettivamente usata per trasmettere dati

3.1.3 Recupero dagli errori

SDU - Service Data Unit: dati ricevuti dal livello superiore.

Il livello datalink riceve un SDU che trasforma in frame. Ipotizziamo che non ci siano errori di trasmissione, i frame arrivano tutti e in ordine.

Possibili errori: receiver lento a processare i dati; bufferizza i frame e se sono troppi li dropa. Il mittente può inviare nuovi dati quando il destinatario ha finito di processare quelli già inviati.

ACK

Pacchetto che segnala che il dato precedente è stato ricevuto correttamente.

Dividiamo il frame In

- Header: 0 = dati, 1 = ACK
- Payload: dati da inviare

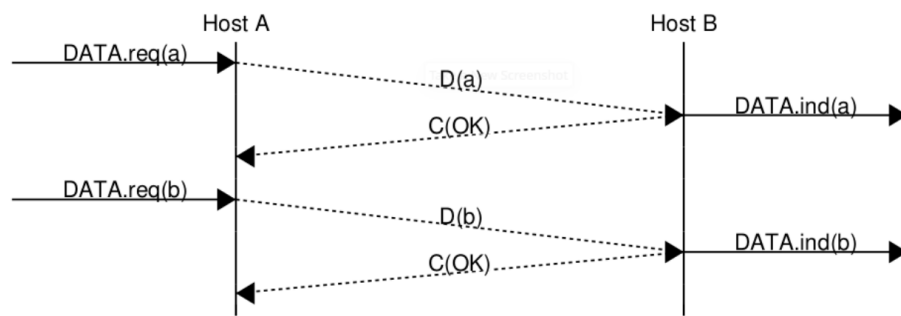


Figura 3.4: ACK [1]

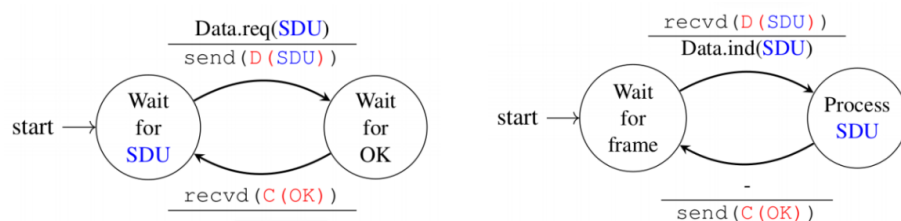


Figura 3.5: FSM Sender e Receiver del livello datalink [1]

In Figura 3.5 il numeratore indica l'evento che causa il cambio di stato, il denominatore l'azione eseguita (D = Data, C = ACK).

3.1.4 Gestione Link Errors

Flipped bits, missing bits, added bits

Error detection: rilevamento di errori nei frame ricevuti

Bit di parità

- Mittente e destinatario si accordano su parità pari/dispari
- Mittente aggiunge bit di parità al frame in modo che il numero totale di 1 rispetti la regola
- Il destinatario verifica la parità e rimuove l'ultimo bit

Esempio

Dato	Parità dispari	Parità pari
0011011	0011011 1	0011011 0

$$(0 + 0 + 1 + 1 + 0 + 1 + 1)_2 = 100_2 \implies 100_2 \pmod{2} = 0$$

Con parità dispari il destinatario si aspetta un numero di bit dispari

Se non ci sono errori rimuove l'ultimo bit e passa l'SDU al livello superiore. In caso di errore richiede una ritrasmissione o dropa il frame.

Se ci sono un numero pari di errori o il bit di parità è sbagliato.

Checksum

Il bit di parità si può estendere a checksum, CRC e hashing.



Checksum: complemento a uno (16 bit) della somma in complemento a uno di tutte le parole da 16 bit del messaggio.

Aggiungono un campo al frame → overhead → header allineati al byte e se meno di 1B si aggiunge pad.

Più grande è il checksum, maggiore è la probabilità di rilevare l'errore.

Probabilità di avere un errore non rilevato: 2^{-C} (C: dim del checksum)

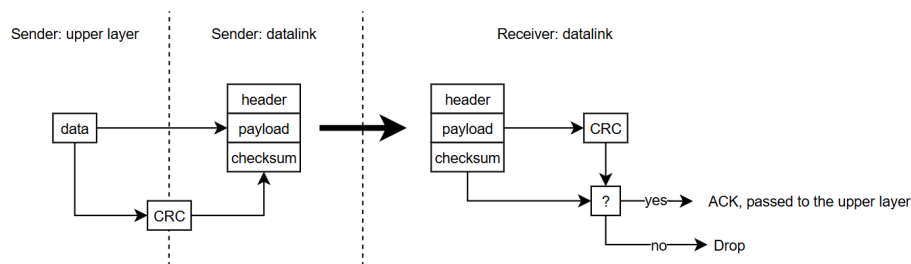


Figura 3.6: Calcolo CRC [2], basato su [3]

Il CRC è salvato alla fine del frame in modo che il recv possa calcolarlo e confrontarlo direttamente senza salvarlo due volte.

A livello data link si usa CRC, non checksum.

3.1.5 Correzione degli errori

Error correction: aggiunge ridondanza per rilevare e correggere un errore

Esempi: 0 → 000, 1 → 111, ma 010 potrebbe essere l'invio di 0 con 1/3 errori o 1 con 2/3 errori.

Sprechiamo 2/3 del bit-rate, quindi assumiamo che gli errori siano rari e sia più conveniente dropare i frame con errori e ritrasmettere.

Errore invio frame: Il mittente invia un frame e avvia un timer in attesa di un ACK. Se l'ACK non arriva, ritrasmette il frame.

Errore invio ACK: Il destinatario riceve il frame, invia l'ACK, ma viene perso. Il mittente ritrasmette il frame e il destinatario deve riconoscere che è un duplicato.

Sequence number - ABP

Alternating Bit Protocol: bit di sequenza alternato a ogni frame per distinguere i duplicati.

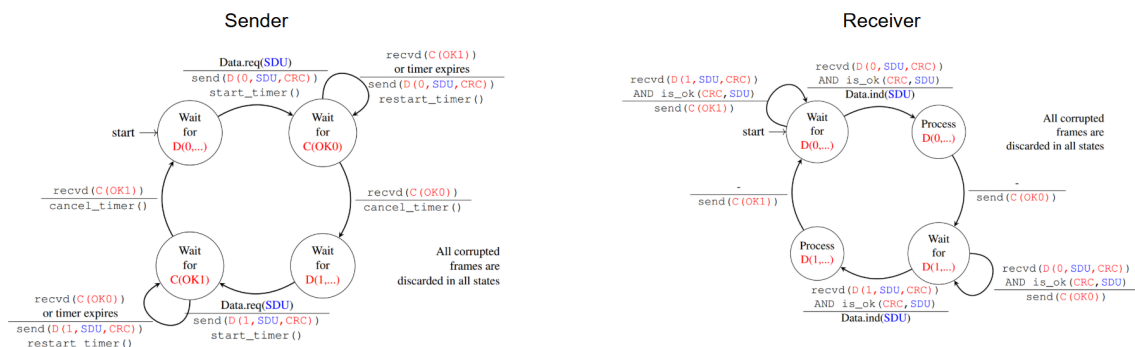


Figura 3.7: FSM Alternating Bit Protocol [1]

Il destinatario invia l'ACK solo dopo aver processato il frame, quindi il mittente può regolare il sending rate di conseguenza.

Le macchine a stati funzionano fintanto che mittente e destinatario sono sincronizzati.

Ora il livello datalink offre un nuovo servizio (reliable delivery) e non serve modificare gli altri livelli.

RTT - Round Trip Time: tempo tot impiegato da un pacchetto per andare dal mittente al destinatario e tornare indietro.

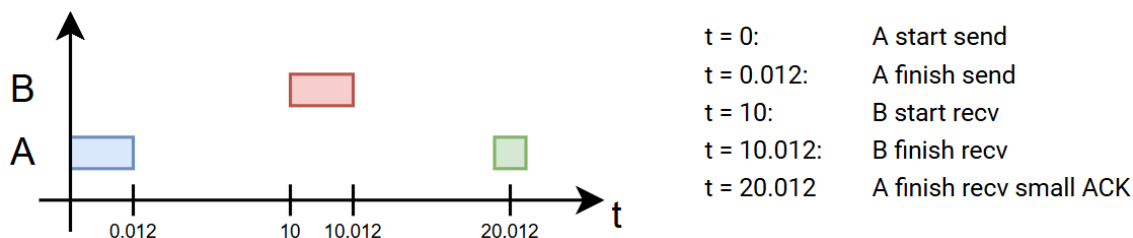
Esempio

Latenza: 10ms, Frame: 1500B, bit-rate: 10^9 b/s

Tempo per inviare: $\frac{1500 \cdot 8}{10^9} = 0.012$ ms

Un frame ACK ha 2 bit (header e seq_n): $\frac{2}{10^9}$ s (si può ignorare)

Nella realtà RTT dipende dal delay, buffering nei router e tempo di elaborazione.



Servono 20.012 ms per inviare un frame di 1500B che corrisponde a un bit-rate di $\frac{1500 \cdot 8}{0.020012} \simeq 0.6$ Mb/s

L'attesa degli ACK rallenta la trasmissione, anche se il link è veloce.

Esempio reale

I Satelliti geostazionari orbitano attorno alla Terra a 35,784,000 m dalla superficie terrestre, alla stessa velocità di rotazione della Terra, rimanendo fissi rispetto a un punto sulla superficie terrestre. Le onde elettromagnetiche viaggiano a circa $3 \cdot 10^8$ m/s.

$$\text{RTT} = 2 \cdot \frac{35.784.000}{3 \cdot 10^8} \simeq 200 \text{ ms}$$

Per ridurre il delay, possiamo diminuire la distanza, aumentando il numero di satelliti.

ABP è semplice da implementare ma aumenta il delay. Se i link sono troppo lontani si usa pipelining.

3.2 Reliable transport

Pipelining: Il mittente può inviare più frame in un flusso senza attendere l'ACK di ciascuno di essi.

Sliding window: Il mittente può inviare fino a N frame senza attendere l'ACK, dove N è la dimensione della finestra. Quando il frame con il seqNum più basso viene ACKed, la finestra scorre in avanti.



Figura 3.8: Sliding window [1]

Sequence number finiti: N bit $\rightarrow \text{seqNum} \in [0, 2^n - 1]$

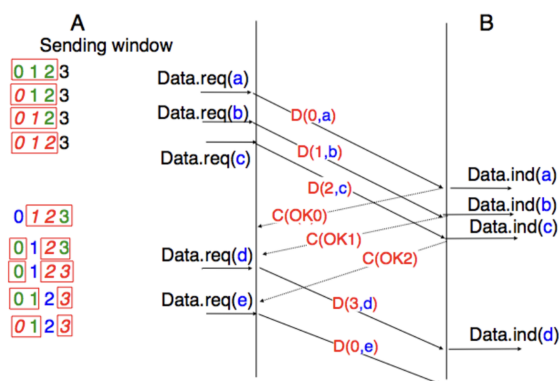


Figura 3.9: Sliding window con wrap up $N = 3$ [1]

Il flusso, controllato dal delay, riduce l'impatto del RTT. Ora è necessaria una policy corretta, ma principalmente efficiente, che gestisca la perdita dei frame.

3.2.1 Go-back-n

Il destinatario:

- Accetta solo i frame in ordine, scarta gli altri.
- Invia ACK cumulativi con il seqNum dell'ultimo frame ricevuto in ordine (conferma tutti i precedenti).
- I frame sono processati uno alla volta, non c'è buffering.
- Si salva solo i seqNum `lastack` e `next` (expected)

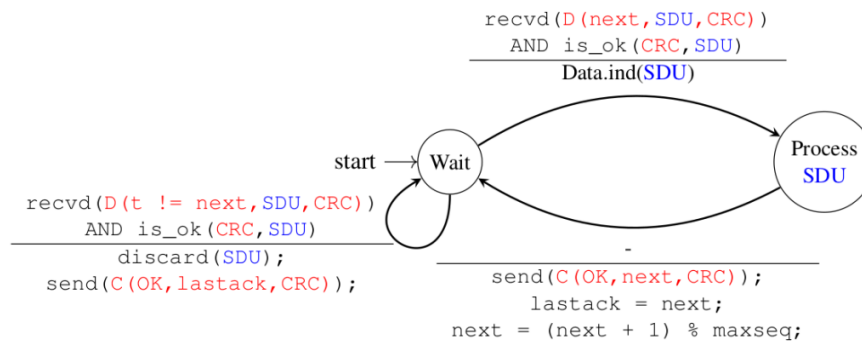


Figura 3.10: Go-back-n - FSM destinatario [1]

Il mittente:

- Mantiene un buffer di frame di dimensione N (dimensione della finestra).
- Invia i frame in ordine, fino a riempire il buffer. Poi attende gli ACK.
- Usa un solo timer per il frame più vecchio non ancora ACKed.
- Memorizza `size(buffer)`, `seq` (ultimo seqNum inviato), `unack` (seqNum del frame più vecchio non ancora ACKed) e un timer.

Quanto il mittente riceve un ACK:

- Rimuove i frame acked dal buffer (l'ack è cumulativo).
- Riavvia il timer se ci sono ancora frame non acked.

Se il timer scade, ritrasmette tutti i frame nel buffer (non acked) e riavvia il timer.

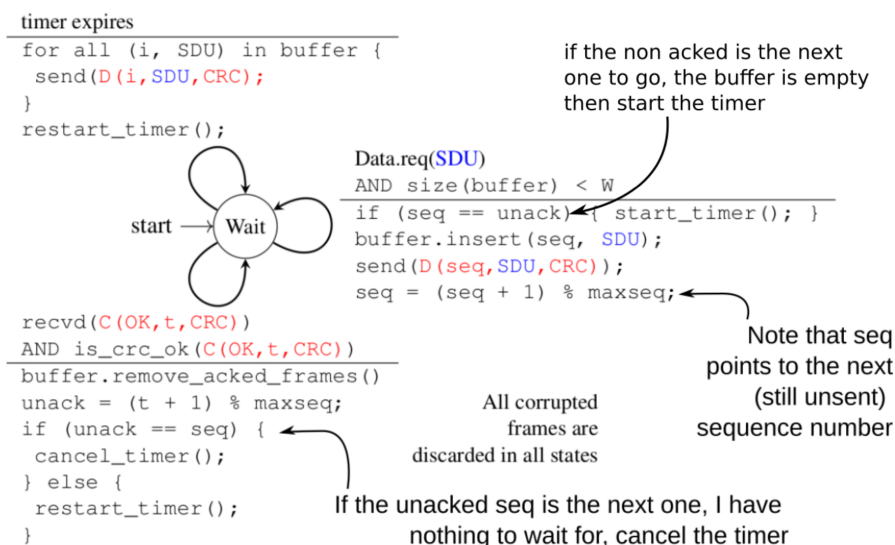


Figura 3.11: Go-back-n - FSM mittente [1]

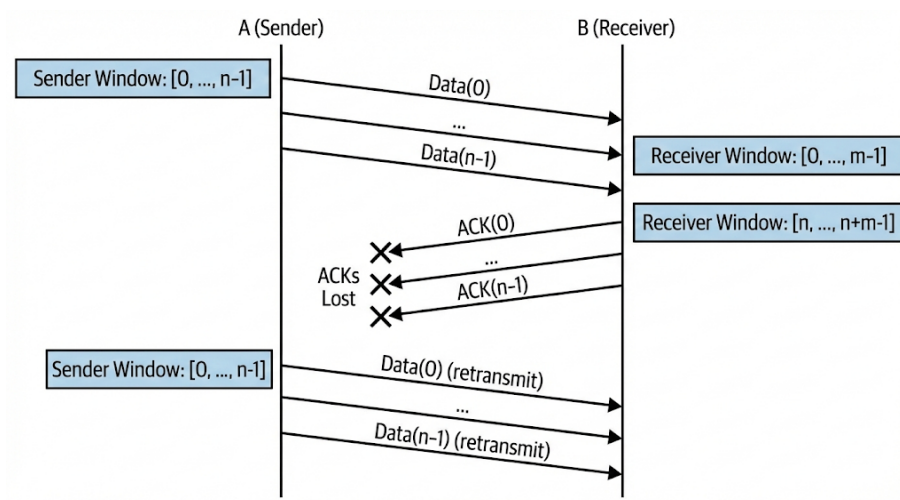
3.2.3 Piggybacking e deferred ACKs

Nel mondo reale gli scambi di dati sono bidirezionali. Conviene quindi inviare gli ACK insieme ai dati (piggybacking) per ridurre l'overhead e attendere prima di inviare un ACK (deferred ACKs) in attesa di eventuali dati in uscita.

3.2.4 Dimensione della finestra - analisi personale

Consideriamo un seq. number di N bit $\rightarrow 2^N$ valori, $\text{seqNum} \in [0, 2^N - 1]$.

$$W_s = n, W_r = m$$



Affinché non si verifichi un wrap up dei sequence number:

$$n + m - 1 < 2^N \iff n + m \leq 2^N$$

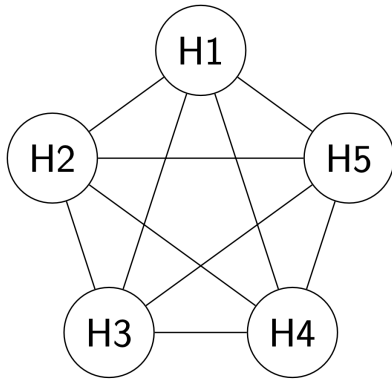
In Go-back-n: $m = 1 \rightarrow n + 1 \leq 2^N \iff n \leq 2^N - 1$

In Selective Repeat $n = m \rightarrow 2n \leq 2^N \iff n \leq 2^{N-1}$

3.3 Condivisione di risorse

3.3.1 Topologie di rete

Rete magliata



- + Capacità e robustezza
- Troppi cavi $\frac{n(n-1)}{2}$ (costi per installare i cavi)

Figura 3.14: Rete magliata [1]

Rete bus

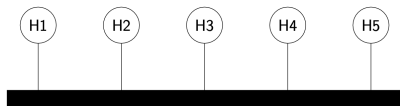


Figura 3.15: Rete bus [1]

- Non scalabile, la capacità diminuisce con più nodi
- Comunicano uno alla volta (come gestire l'accesso al canale?)

Rete ad anello

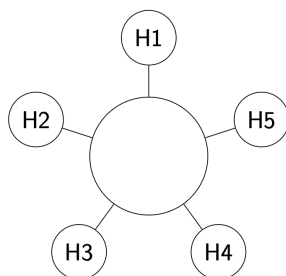
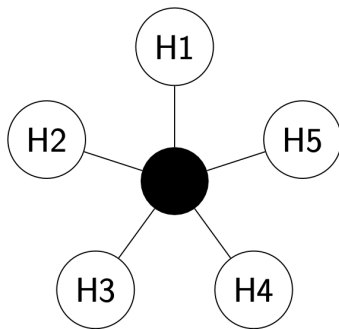


Figura 3.16: Rete ad anello [1]

- Resiliente alla rottura di un link
- Nelle metropolitane (ridondanza)
- Non scalabile

Rete a stella

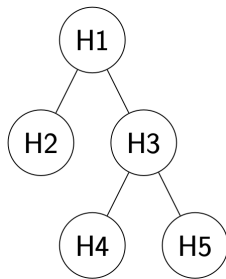


- $n - 1$ cavi
- Un punto controlla tutto (se si rompe si rompe tutto)

Figura 3.17: Rete a stella
[1]

Le reti wireless sono organizzate logicamente a stella (access point centrale), ma fisicamente a bus (tutti vedono tutto).

Rete ad albero



- $n - 1$ cavi
- Scalabile, estende una rete a stella
- Multiplexing quando il traffico va verso l'alto, demultiplexing verso il basso

Figura 3.18:
Rete ad albero
[1]

Bottleneck

Comunicazione multi-hop: il traffico passa attraverso più link.

 **Bottleneck:** link più lento nel percorso.

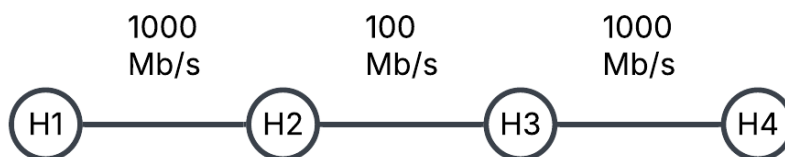


Figura 3.19: Bottleneck: H1 - H4 limitata a 100 Mb/s a causa di H2 - H3

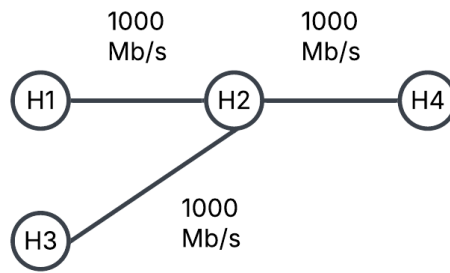


Figura 3.20: Bottleneck: se H1, H3 comunicano con H4 contemporaneamente, il loro bit-rate è limitato a 500 Mb/s

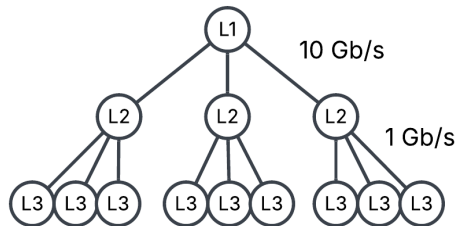


Figura 3.21: Rete gerarchica: capacità dei nodi vicini alla radice maggiore rispetto a quelli più lontani

Data center

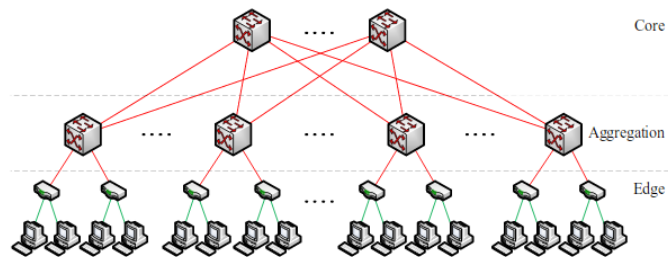


Figura 3.22: Data center [4]

Link vicini alla radice hanno capacità maggiore e la ridondanza degli switch core aumenta la tolleranza ai guasti. Sono molto costosi e di solito si usano i Fat tree.

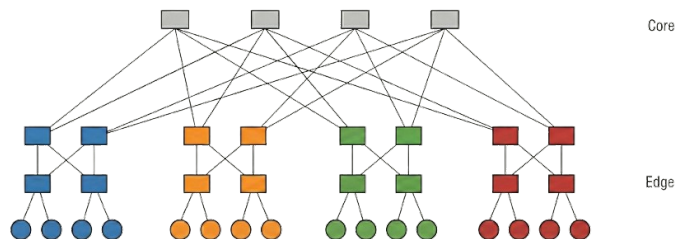


Figura 3.23: Fat tree

Ridondanza di switch edge (molto meno costosi dei core), ma hanno meno capacità e saturano prima. Bisognerebbe gestire più percorsi, ma è complesso.

3.3.2 Media Access Control

Multiplexing: tecnica che consente a più host di condividere lo stesso canale di comunicazione (link).

Gestione delle collisioni: In reti a stella: cavi ethernet con circuiti separati per trasmissione e ricezione (full-duplex), in reti bus: scheduling dei frame.

FDMA - Frequency Division Multiple Access

Suddividere la banda in canali di frequenza separati. Ad esempio WiFi con 2.4 GHz e 5 GHz.

Comunicando sulla stessa frequenza si hanno interferenze e la comunicazione dipende dal SNR.

Ma gli host connessi ad un wifi comunicano sulla stessa frequenza in momenti diversi.

TDMA - Time Division Multiple Access

Il tempo è suddiviso in slot. Gli slot vengono ripetuti in cicli e sono assegnati a dei terminali. Ogni terminale trasmette o riceve nel suo slot assegnato. L'access point trasmette a tutti nel suo slot.

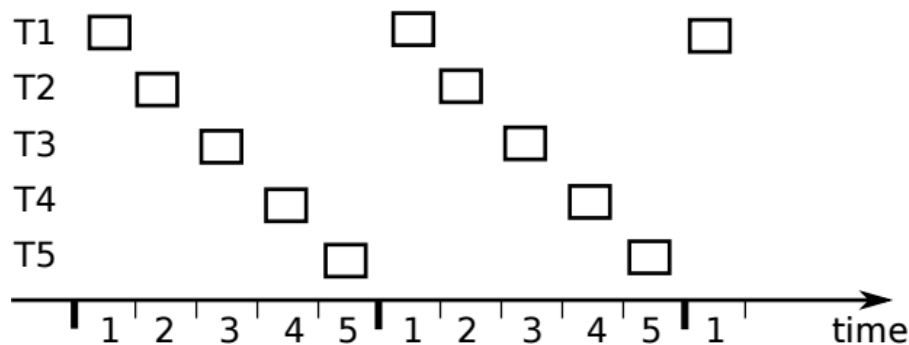


Figura 3.24: Esempio di TDMA

Efficienza massima se la rete è satura (situazione non desiderata).



Efficienza livello datalink: percentuale di tempo usata per inviare dati utili.

Se un solo terminale deve comunicare? TDMA dinamico: il terminale chiede più risorse all'AP.

Semplice da implementare e deterministico (reti cellulari 2G, bluetooth)

CDMA (Non in programma)

Analogia del cocktail party: tutti trasmettono contemporaneamente sulla stessa frequenza, ma ogni trasmissione è codificata con un codice unico. Ogni ricevitore conosce il codice del trasmettitore desiderato e può estrarre il segnale utile.

Codici ortogonali: Moltiplicando un segnale per un codice errato si ottiene un vettore nullo.

Esercizio

Sistema TDMA con: bit rate $b = 100 \text{ Mb/s}$, $N = 5$ terminali, slot $s = 1 \text{ ms}$.
Calcolare:

- Dati trasmessi in uno slot da un terminale: $b * s = 10^8 * 10^{-3} = 100 \text{ Kb}$
- Dati trasmessi in 1s da un terminale
 $Slot_{tot} = 200 \rightarrow (b * s) * 200 = 100 \text{ Kb} * 200 = 20 \text{ Mb/s}$
- Se un terminale deve inviare 250 Kb, quanto tempo impiega?
Al tempo t_i invia 100 Kb, al tempo t_{i+5} invia altri 100 Kb, al tempo t_{i+10} invia gli ultimi 50 Kb in 0.5 ms.
Best case: $i = 0 \rightarrow t_{10} \rightarrow 10.5 \text{ ms}$
Worst case: $i = 4 \rightarrow t_{14} \rightarrow 14.5 \text{ ms}$
Tempo medio: 12.5 ms

3.3.3 Random Access: ALOHA e CSMA

I terminali cercano di trasmettere e se si verifica una collisione, ritrasmettono dopo un tempo casuale.

ALOHA

Sviluppato negli anni '70 all'università delle Hawaii per collegare computer su isole remote e antenato del CSMA/CA usato nel WiFi.

S-ALOHA (Slotted ALOHA)

Terminali sincronizzati che trasmettono a un singolo ricevitore solo all'inizio di uno slot. Trasmette sempre in una rete con poco traffico, in caso di collisione (non ricevono ack) si ritrasmette con una certa probabilità.

q_a : probabilità di trasmissione

Se si verifica una collisione, i terminali entrano in uno stato **backlogged** e ritrasmettono con probabilità q_b ad ogni slot successivo. Se ricevono altri frame dai livelli superiori, li scartano.

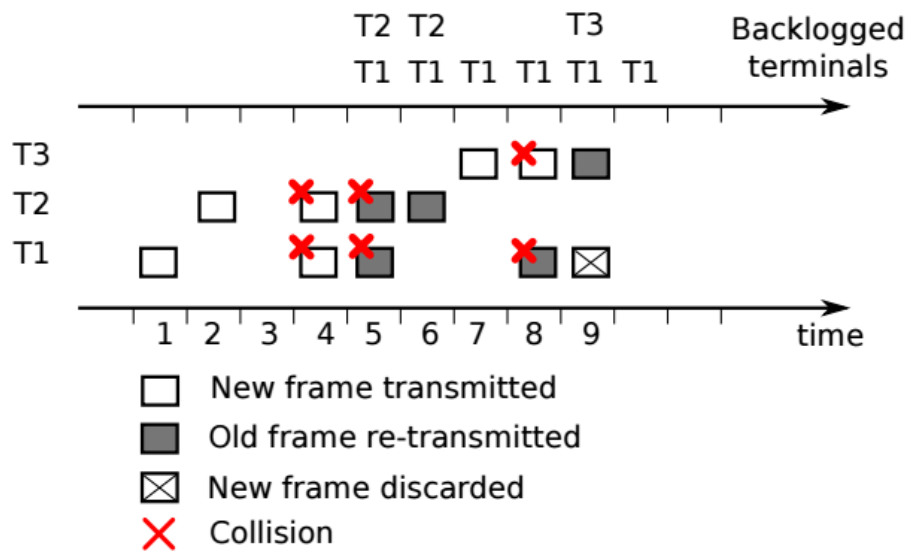


Figura 3.25: Aloha

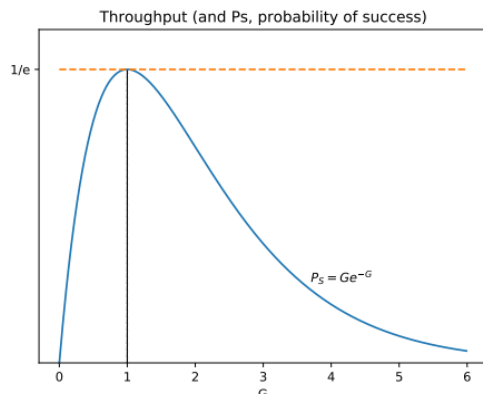


Carico sulla rete (G): numero medio di frame trasmessi da tutti i terminali per slot di tempo.



P_S : probabilità che una trasmissione abbia successo.

$$P_S = Ge^{-G}$$



$G = 1$: max throughput $\frac{1}{e}$, $q_a = \frac{1}{N}$ (ogni terminale trasmette una volta ogni N slot)

Se < 1 : rete scarica, se > 1 : rete congestionata.

Figura 3.26: ALOHA - Carico sulla rete

Per l'esame: Conoscere come funziona S-ALOHA e spiegare la curva del throughput



Congestion collapse: si verifica quando l'incremento del carico di rete provoca un calo improvviso delle prestazioni del sistema.

TDMA non collassa mai, anche quando la rete è congestionata (carico alto $\Rightarrow$ meno risorse per tutti).

ALOHA è instabile

In conclusione, TDMA è semplice da implementare, ma spreca risorse quando la rete è scarica. S-ALOHA è più efficiente quando la rete è scarica, ma instabile quando è congestionata. (possibile soluzione: reti ibride)

Il livello datalink non è affidabile, può scartare frame, quindi i livelli superiori devono gestire la ritrasmissione.

4 Livello Rete

4.1 Introduzione al livello di rete

Il livello di rete gestisce la comunicazione tra host che non appartengono alla stessa rete fisica, attraverso router intermediari.

Assume che il livello datalink è *almost reliable*: perde frame raramente e il suo PDU (Protocol Data Unit) è il **pacchetto/datagram**.

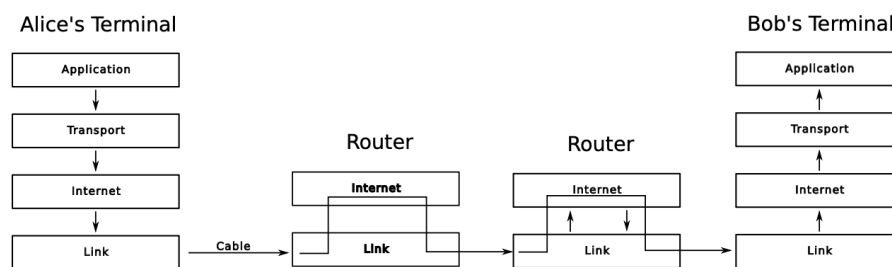


Figura 4.1: Esempio router

4.1.1 Livello di rete Datagram-oriented

Ogni host è identificato da un indirizzo di rete e i router usano un instradamento *hop by hop*: ogni router decide indipendentemente dove mandare i pacchetti.



Forwarding table: struttura dati che mappa ogni indirizzo di destinazione a un'interfaccia di uscita in cui inviare un pacchetto.

Le tabelle devono essere consistenti (devono esserci tutti i percorsi aggiornati), ma possono verificarsi degli errori:



Black hole: un router scarta un pacchetto, perché non ha un'entrata valida per la destinazione nella sua forwarding table.



Routing loop: ciclo infinito in cui i router si inoltrano a vicenda un pacchetto

I router implementano:

- Data plane: inoltro dei pacchetti secondo la forwarding table.
- Control plane: costruzione e aggiornamento della forwarding table con scambio di pacchetti di controllo.

La forwarding table può essere costruita manualmente, ma questa soluzione non è scalabile. Una soluzione automatica può prevedere un controller centralizzato (reti piccole) o distribuito (internet).

Il circuit-oriented network layer (es. ATM) è un altro tipo di livello di rete.

4.1.2 Indirizzamento ed eterogeneità

Flat Addressing

Ogni macchina mantiene lo stesso indirizzo, non cambia. È semplice da configurare, ma non è scalabile: ogni router deve mantenere un'entry per ogni host nella sua forwarding table.

Addressing gerarchico

Si divide la rete in sottoreti, ogni host ha un indirizzo composto da: indirizzo di rete + indirizzo host. Le forwarding table memorizzano solo gli indirizzi di rete.

4.1.3 Frammentazione

Una rete può essere composta da link con diverse capacità massime di trasmissione (MTU, Maximum Transmission Unit). Se un pacchetto è più grande della capacità supportata da un link, il router può:

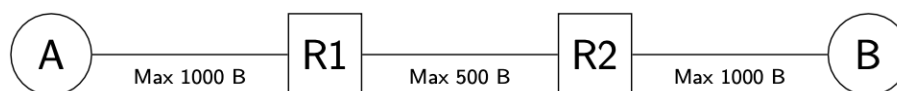


Figura 4.2: Frammentazione dei pacchetti

Drop and notify: R1 scarta il pacchetto e notifica A. A ritrasmette in pacchetti più piccoli.

Fragment and Reassemble at Next Hop: R1 frammenta il pacchetto in pezzi più piccoli e li invia a R2. R2 riassume i pezzi.

Fragment and Reassemble at Destination: R1 frammenta il pacchetto in pezzi più piccoli indipendenti e li invia. B riassume i pezzi (miglior trade-off).

4.1.4 Hot Potato

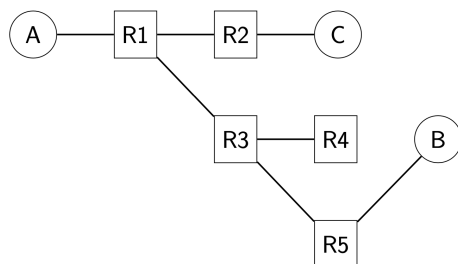


Figura 4.3: Hot potato

Se A vuole comunicare con B

1. Se sono la destinazione non inviare
2. Se conosco la destinazione, inoltra unicast
3. Altrimenti broadcast a tutti i vicini, tranne a chi me l'ha inviato

1. A invia il pacchetto al router R1 vicino.
2. R1 aggiorna la FT ($A \rightarrow \text{West}$) e inoltra il pacchetto ai router vicini (broadcast).
3. R2 aggiorna la FT ($A \rightarrow \text{West}$) e inoltra il pacchetto a C, che non è la destinazione e lo scarta.
4. R3 aggiorna la FT ($A \rightarrow \text{North-West}$) e inoltra a R4, R5.
5. R4 aggiorna la FT.
6. R5 aggiorna la FT e inoltra a B.

Ora tutti i router conoscono il percorso per raggiungere A.

Non necessita di control packet aggiuntivi, le FT sono costruite con i pacchetti dati.

È necessario che la rete sia un albero, altrimenti possono crearsi dei loop.

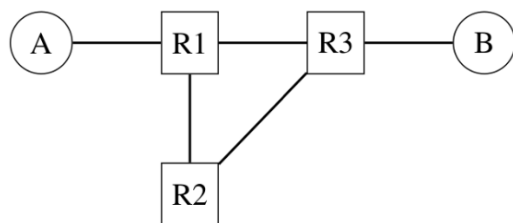


Figura 4.4: Loop

1. R1 invia a R2, R3
2. R2 inoltra a R3, R3 inoltra a R1, B
3. R1 inoltra a R2 che inoltra a R3, che inoltra a R1, B, ...
4. B continua a ricevere copie del pacchetto

Circuiti virtuali: Un altro approccio è stabilire un percorso fisso tra mittente e destinatario che dura per tutta la comunicazione. Ogni pacchetto specifica il circuito nell'header.

Control Plane



Algoritmo: sequenza finita di istruzioni non ambigue per risolvere un problema.



Protocollo di routing: insieme di specifiche (formato pacchetti, macchine a stati) per implementare un algoritmo di routing.



Daemon: Processo che implementa il protocollo di routing su un router. Crea e gestisce i pacchetti di controllo, gestisce la routing table e la traduce in FT.

Lo scopo di un protocollo di routing è costruire una **routing table** R che associa ogni destinazione d a un **link** (interfaccia di uscita), un **cost** e un **time** (tempo dall'ultimo aggiornamento).

Una RT ha più informazioni di una FT e un router può avere più RT (una per ogni protocollo di routing usato per comunicare con reti diverse) che vengono mergeate in un'unica FT.

```
$ ip r # routing table in linux
```

4.1.5 DV - Distance Vector Routing

Efficiente, semplice da implementare, leggero, ma lento a convergere.

Ogni link ha un costo e i router inizializzano la propria RT settando il proprio indirizzo a costo 0

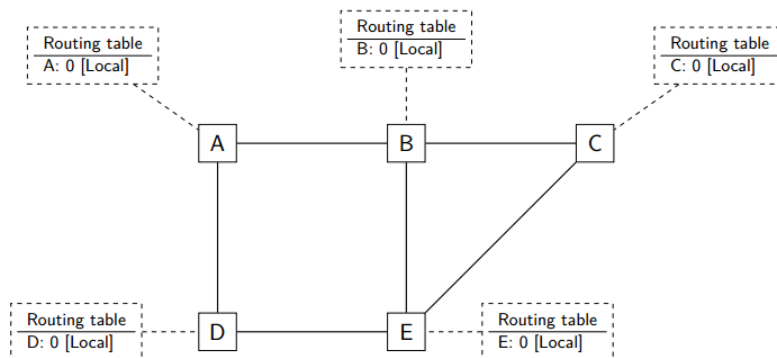


Figura 4.5: Inizializzazione Distance Vector [1]

Periodicamente ogni router invia la propria RT (distance vector) a tutti i vicini. L'ordine di arrivo è indefinito.

Listing 4.1: Invio DV

```

1 Every N seconds: # period of generation
2     v = {} # vector
3     for d in R: # for each destination in RT
4         v[d] = R[d].cost
5     for i in interfaces: # for each neighbor
6         send(v, i)
  
```

Listing 4.2: Ricezione DV

```

1 # V{:} received DV, l: incoming interface, R[:]: current RT
2 def received(V, l):
3     for d in V:
4         if d not in R: # new route
5             update(V, d, l)
6         elif V[d].cost + l.cost < R[d].cost or R[d].link == l:
7             # better route or same link
8             update(V, d, l)
9
10 def update(V, d, l):
11     R[d].link = l
12     R[d].cost = V[d].cost + l.cost
13     R[d].time = now()
  
```

Riga 6: Se conosco d, aggiorno se ho un percorso migliore o se il percorso arriva dallo stesso link (utile in caso di errore) (Es. A passa per B per arrivare a d, se B invia il suo DV ad A, A deve aggiornare il costo per arrivare a d passando per B)



Convergenza: Stato in cui tutte le RT hanno next hop per un percorso minimo verso ogni destinazione.

Esempio di convergenza, assumendo che i link abbiano tutti costo 1.

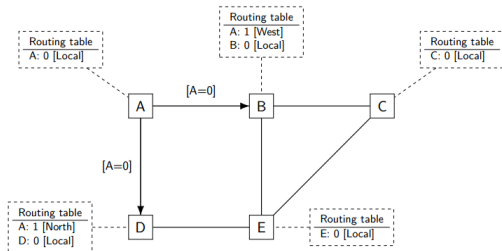


Figura 4.6: A invia il DV [1]

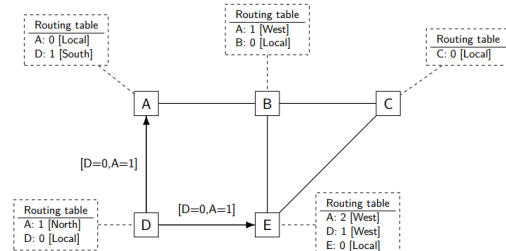


Figura 4.7: D invia il DV [1]

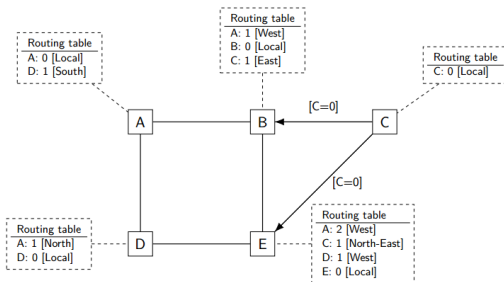


Figura 4.8: C invia il DV [1]

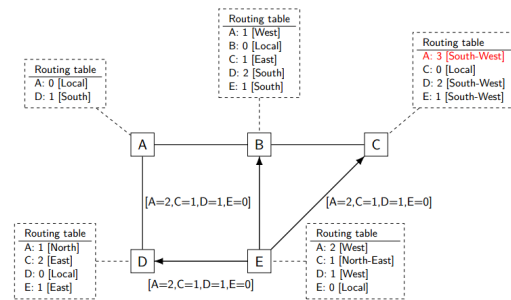


Figura 4.9: E invia il DV, B converge [1]

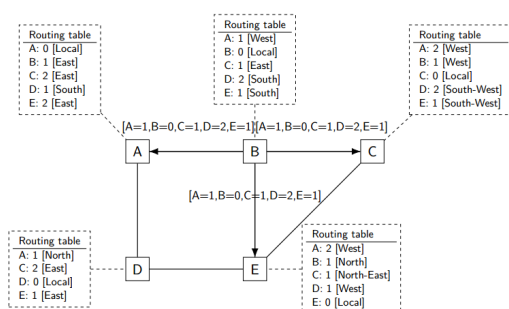


Figura 4.10: B invia il DV [1]

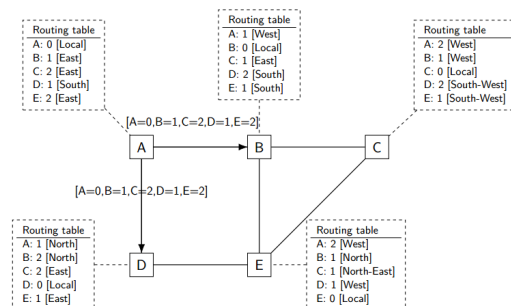


Figura 4.11: A invia il DV, di nuovo e la rete converge [1]

I pacchetti di controllo non sono propagati, i router inviano il proprio DV periodicamente.

Failure Recovery

Se una rotta non viene aggiornata dopo $3N$ secondi, il suo costo è impostato a infinito, in modo che i router vicini possano cercare un percorso alternativo. Dopo altri $3N$ secondi l'entry è rimossa dalla RT.

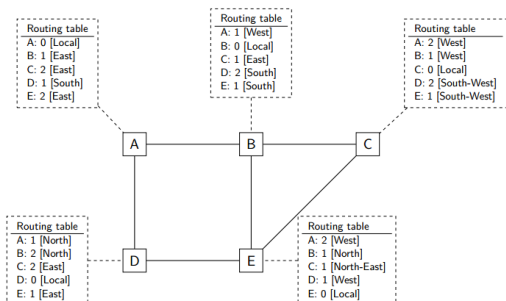


Figura 4.12: C invia il DV [1]

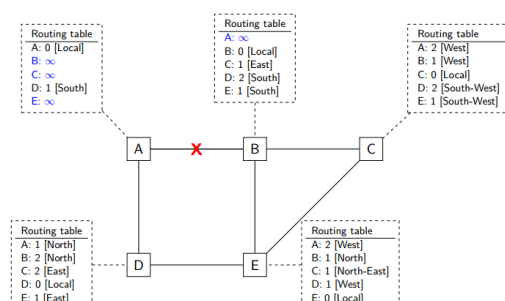


Figura 4.13: Il link A-B fallisce [1]

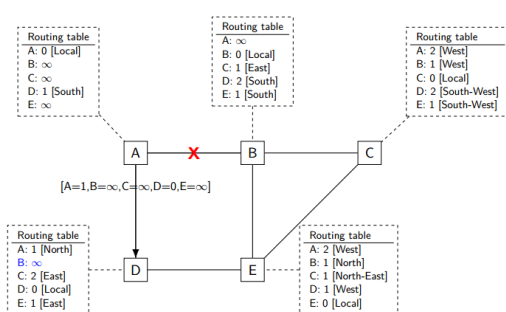


Figura 4.14: A invia il DV [1]

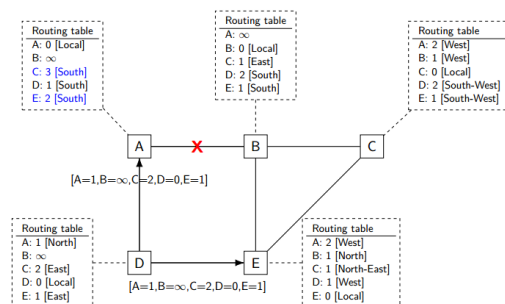


Figura 4.15: D invia il DV (black hole: $C \rightarrow A$) [1]

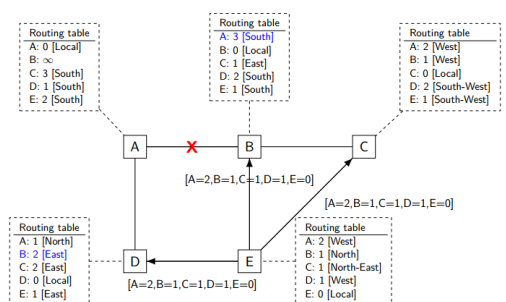


Figura 4.16: E invia il DV [1]

Recupero sub-ottimale

4.2 Limiti di convergenza in DVR

4.2.1 Count-to-Infinity

A causa di un guasto, i router aggiornano le loro RT comunicando solamente con i vicini, senza conoscere la condizione generale della rete

Si consideri l'esempio seguente:

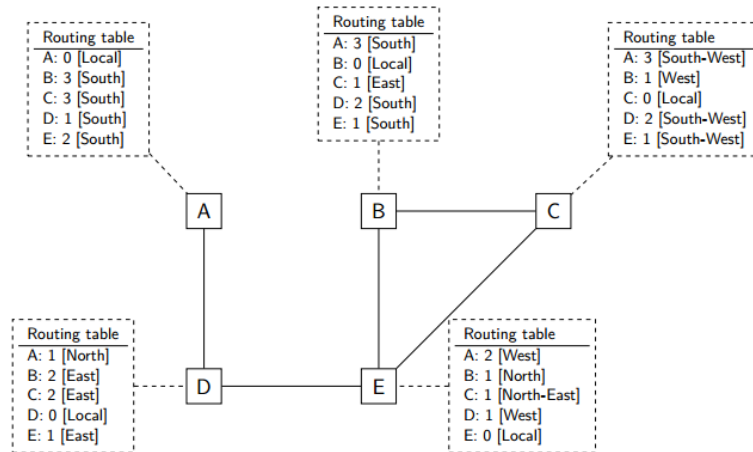


Figura 4.17: Situazione iniziale [1]

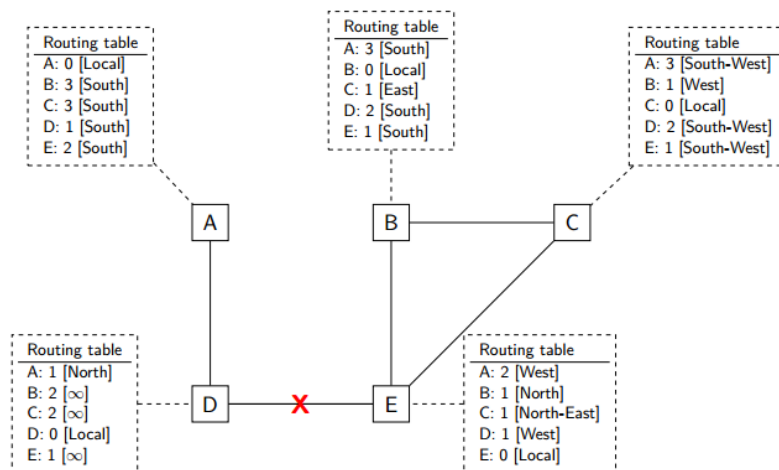


Figura 4.18: Il link D-E fallisce. [1]

In figura 4.18 la rete risulta partizionata in due componenti e se A invia il suo DV prima di D, D aggiornerà la sua RT erroneamente come in figura 4.19.

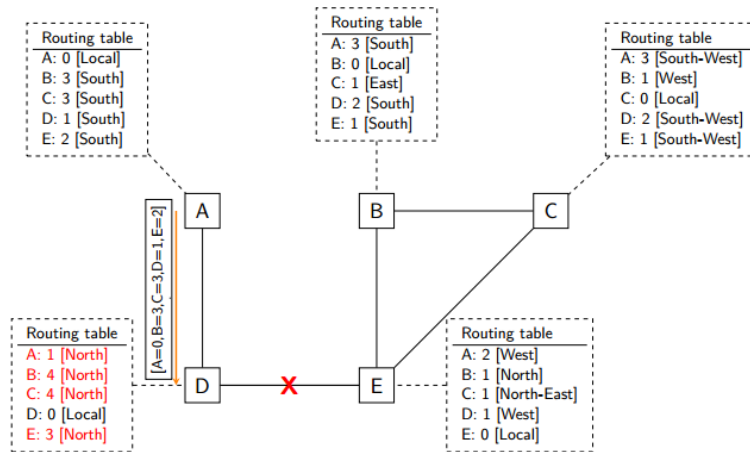


Figura 4.19: A invia il suo DV. [1]

Successivamente, D invierà il suo DV ad A, che aggiornerà la sua RT come in figura 4.20, in quanto risulterà vera la condizione $R[d].link == 1$.

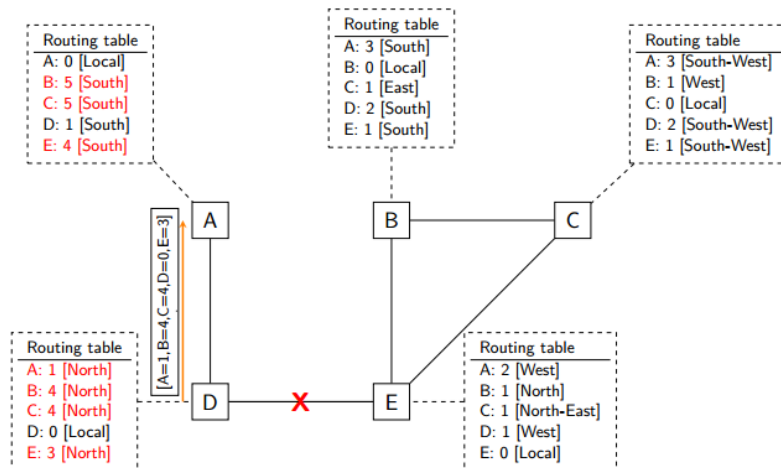


Figura 4.20: D invia il suo DV [1]

I due router continueranno a incrementare le distanze all'infinito.

Una possibile soluzione è quella di usare dei **Triggered updates**: inviare il DV ai vicini non appena un link fallisce. Ma nell'esempio precedente, A potrebbe comunque inviare il DV prima di D, oppure il DV di D potrebbe andare perso.

4.2.2 Split Horizon e Poison Reverse

Listing 4.3: Split Horizon - Poison Reverse

```

1 Every N seconds:
2   for ifc in interfaces: # a vector for each interface
3     v = {} # vector
4     for d in R:
5       v[d] = INFINITY if R[d].link == ifc else R[d].cost
6     send(v, ifc)

```

Split Horizon: suddividere i router vicini in: router usati come next hop ($R[d].link == ifc$) e gli altri.

Poison Reverse: inviare una distanza infinita ai router usati come next hop per una certa destinazione.

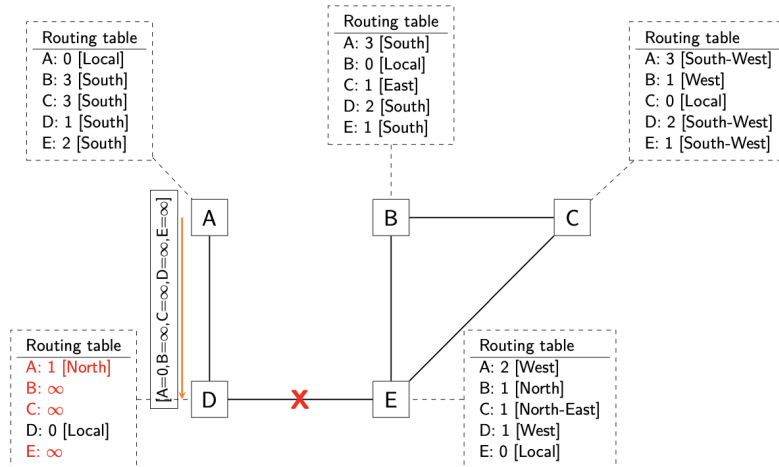


Figura 4.21: A invia il suo DV con Poison Reverse [1]

Questa soluzione aggiunge overhead e non risolve tutti i casi di Count-to-Infinity.

Si consideri l'esempio seguente:

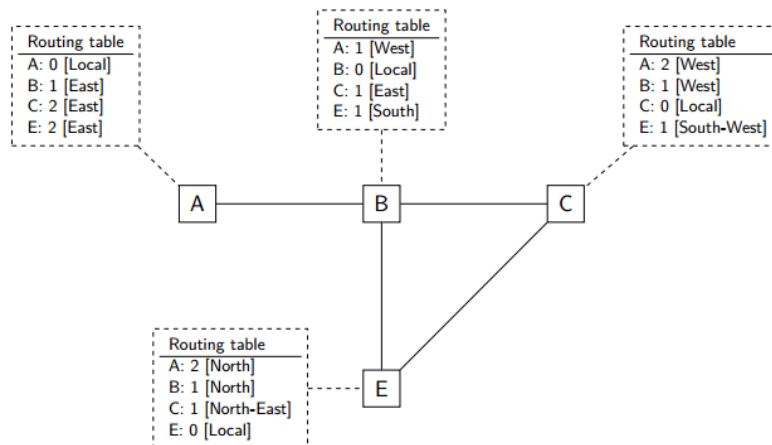


Figura 4.22: Stato iniziale [1]

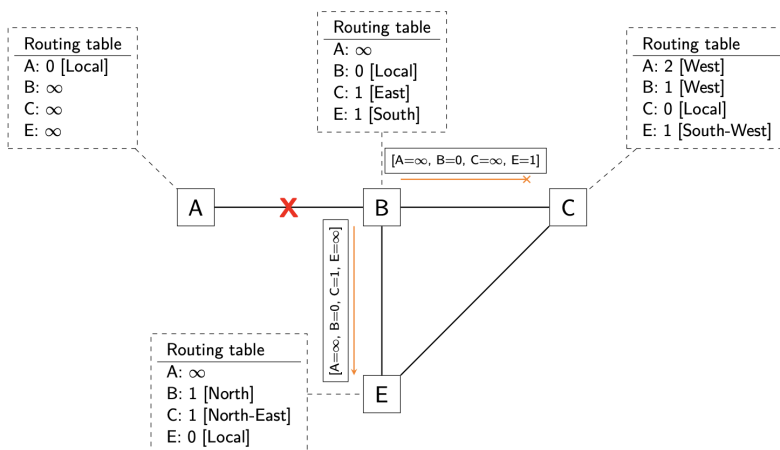


Figura 4.23: B invia il suo DV, ma viene perso il pacchetto per C [1]

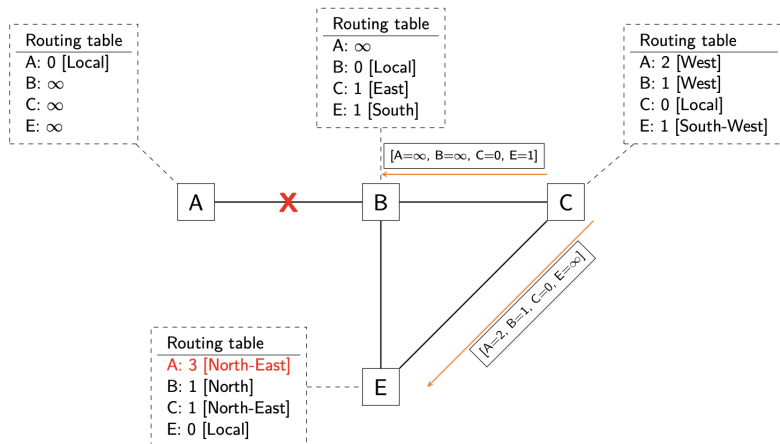


Figura 4.24: C invia il suo DV ed E aggiorna la sua RT erroneamente [1]

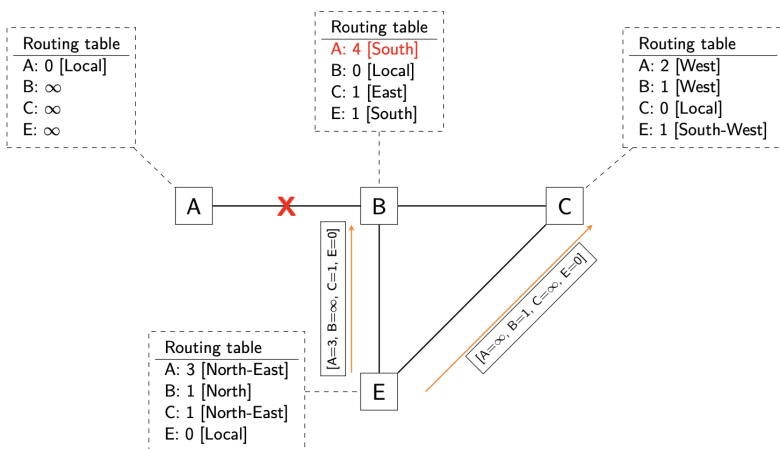


Figura 4.25: E invia il suo DV e si crea un loop [1]

Una possibile soluzione è usare un timer di sospensione (Hold-down Timer) che impedisca a un router di aggiornare la sua RT se ha appena settato un costo ∞ per una rotta. Ciò previene singoli eventi di perdite di pacchetti DV come in figura 4.23

Split Horizon e Poison Reverse risolvono il problema del Count-to-Infinity solo per loop di 2 router e i timer di sospensione rallentano la convergenza della rete.

Per risolvere definitivamente il problema, i router dovrebbero esportare l'intero percorso per la destinazione (come in BGP).

4.2.3 Link State Routing

In questo protocollo, ogni router costruisce una mappa completa della rete e calcola il percorso più breve per ogni destinazione usando l'algoritmo di Dijkstra.

La rete è rappresentata come un grafo orientato pesato, dove i nodi sono i router, gli archi sono i link e i pesi possono essere unitari, latenza, $\frac{C}{link_capacity}$ con C costante.

Messaggi HELLO con indirizzo del router, inviati periodicamente ai vicini per rilevare i link attivi ed eventuali errori (da non propagare).

Link State Packet (LSP): routerID, age, seq number, Links[] (ID vicino, cost).

I router inviano LSP a tutti i nodi della rete in modo affidabile, usando un algoritmo di **flooding** per non generare troppi pacchetti. Ogni router mantiene un DB con gli LSP più recenti e propaga gli LSP ricevuti se hanno un seq number più grande di quello memorizzato nel DB.

Listing 4.4: LS routing - update

```

1 if newer(LSP, LSDB(LSP.routerID)):
2     LSDB.add(LSP) # implicitly remove older LSP
3     # send to all neighbors except the one it came from
4     for i in links - {1}:
5         send(LSP, i)

```

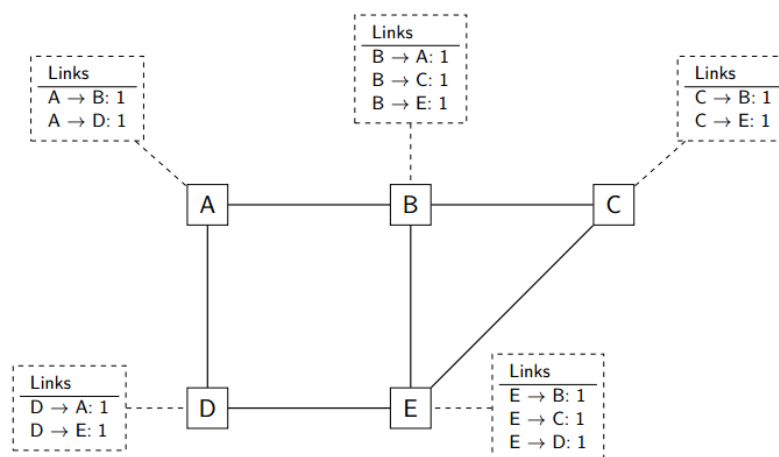


Figura 4.26: Invio di messaggi HELLO [1]

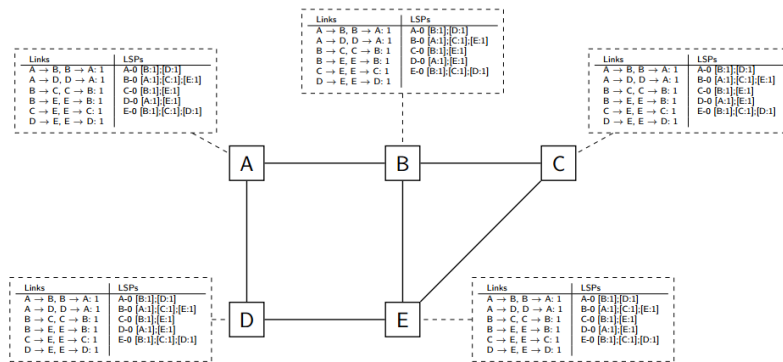


Figura 4.27: Flooding degli LSP [1]

Quando un link fallisce, i router adiacenti non ricevono più messaggi HELLO per $k \times N$ secondi e generano un nuovo LSP, con il link rimosso, che viene propagato in tutta la rete.

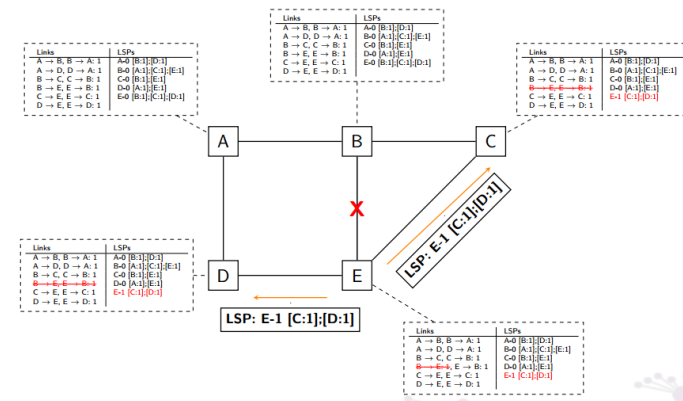


Figura 4.28: Il link B-E fallisce, E rileva un fallimento e genera un nuovo LSP [1]

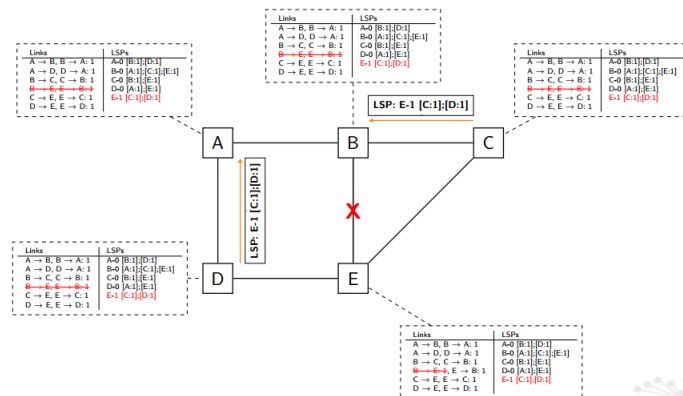


Figura 4.29: Propagazione del nuovo LSP [1]

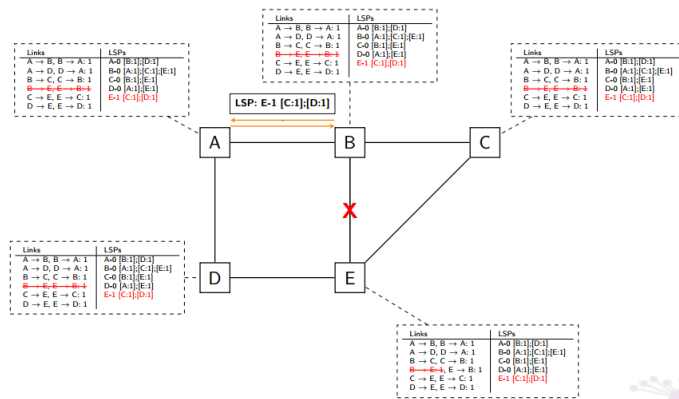


Figura 4.30: Propagazione del nuovo LSP e convergenza [1]

Nell'esempio in figura 4.28 E rileva il fallimento del link prima di B e cancella $B \rightarrow E$, ma gli altri router cancellano anche $E \rightarrow B$, perché un link funzionante in un'unica direzione è considerato non funzionale.

In questa fase transitoria, possono crearsi loop o black hole.

4.2.4 Analisi e confronto

LSR fornisce più informazioni ai router, rispetto a DVR, converge velocemente in caso di cambiamenti, ma richiede più potenza di calcolo e genera più traffico. I messaggi HELLO non sono propagati e possono essere generati frequentemente, migliorando il rilevamento dei guasti.

Assumiamo che il percorso da D ad A sia D-G-F-A e che il link A-B fallisca come in figura 4.31.

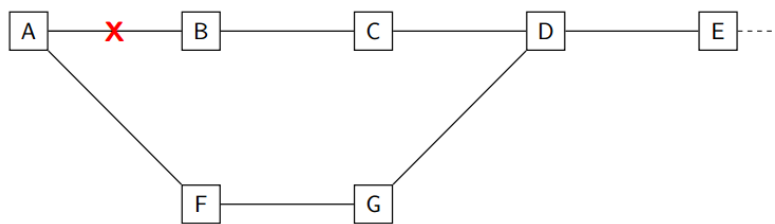


Figura 4.31: Confronto tra Link State e Distance Vector Routing [1]

Con DVR: A,B inviano i loro DV, poi C,F e infine G,D e la rete converge scambiando solo pacchetti locali.

Con LSR: A,B iniziano un flooding di LSP, che viene propagato in tutta la rete e ogni router ricalcola il percorso più breve.

Una soluzione con LSR non è praticabile su una rete molto grande come Internet.

5 Livello di Trasporto

5.1 Transport Layer

Il livello di rete è **connectionless** (non orientato alla connessione), ovvero non garantisce che i pacchetti inviati da un mittente raggiungano il destinatario, né che arrivino nell'ordine corretto.

Consente la trasmissione di pacchetti fino a 65535 Byte.

Il livello di trasporto offre:

- Solo multiplexing (UDP - connectionless)
- Affidabilità e multiplexing (TCP - connection-oriented)

Le applicazioni condividono un **buffer** con il livello di trasporto con uno stream di dati.

PDU: **Segmento** (TCP) o **Datagramma** (UDP).

5.1.1 Modello connectionless

Stateless e offre error detection (checksum).

Non è un problema perdere alcuni dati e non ci interessa ritrasmettere (streaming, gaming).

Multiplexing: più applicazioni possono usare lo stesso servizio di trasporto, distinguendosi tramite le **porte**.

5.1.2 Modello connection-oriented

Stateful: è necessario gestire una connessione tra mittente e destinatario.

Connection set-up

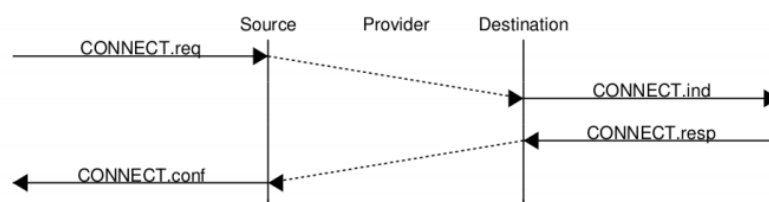


Figura 5.1: Connection set-up [1]

Il livello di rete non è affidabile, potrebbero esserci richieste di connessione duplicate.

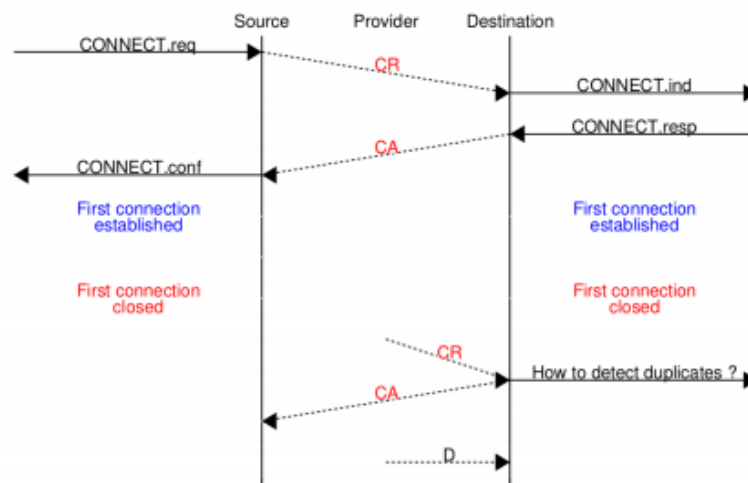


Figura 5.2: Connection set-up: pacchetti duplicati [1]

Potremmo identificare ogni richiesta con un ID, ma il server dovrebbe ricordare tutti gli ID ricevuti.

Assumendo che i pacchetti duplicati dipendano da loop temporanei, definiamo:

 **Maximum Segment Lifetime (MSL):** tempo massimo di vita di un pacchetto nella rete (120s - RFC 793).

Ci basta quindi distinguere le richieste inviate negli ultimi 2 MSL (240s), usando un **Initial Sequence Number (ISN)**, generato a partire da un *transport clock* locale (sopravvive ai reboot).

Inoltre, ISN deve wrappare in un tempo molto maggiore rispetto a MSL per evitare che vecchi pacchetti vengano interpretati come nuovi.

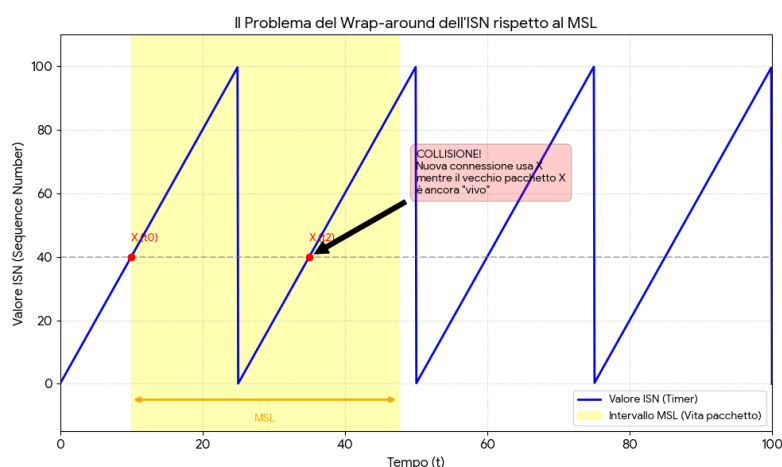


Figura 5.3: Problema del wrapping di ISN rispetto a MSL

5.1.3 Three Way Handshake

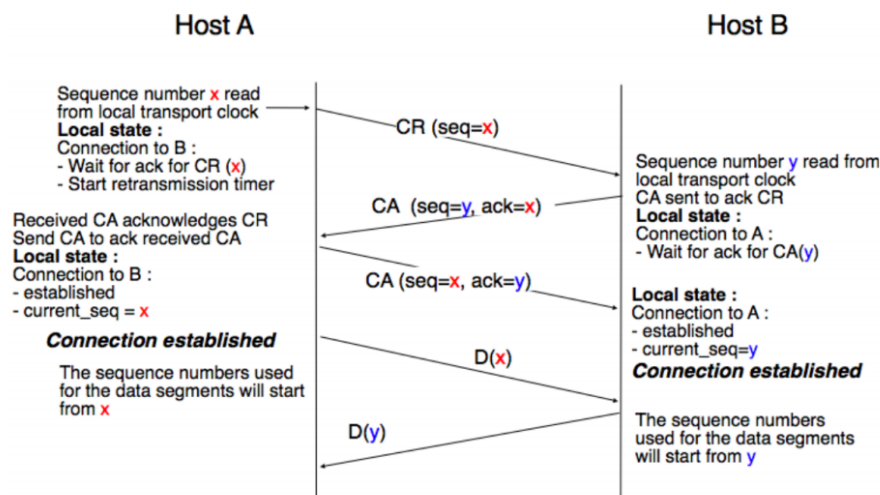


Figura 5.4: Three Way Handshake [1]

5.1.4 Reliable Data Transfer

Con i sequence number, possiamo usare checksum, go-back-n/selective repeat per garantire affidabilità.

- A livello trasporto i sequence number indicano i byte nel flusso continuo.
- Il ricevitore deve avere un buffer, perché la quantità di dati in arrivo non è nota a priori.
- `Data.ind()` non è bloccante; il ricevitore può essere saturato se l'applicazione è lenta.
- Il ricevitore comunica la sua RWIN negli ACK. I dati del mittente $\text{Unacked} \leq \min(\text{swin}, \text{rwin})$

Se la capacità è alta, di solito siamo in una LAN, quindi il delay è basso e i loop sono limitati $\Rightarrow$ MSL basso (problema mitigato)

Se il delay aumenta, MSL aumenta e se la capacità è elevata, il wrap time aumenta (problema).

Esercizio: Calcolare il max throughput per $MSL = 120s$ e sequence number a 32 bit: $\frac{2^{32} * 8}{120} \simeq 286 Mb/s$

5.1.5 Rilascio della connessione

Graceful

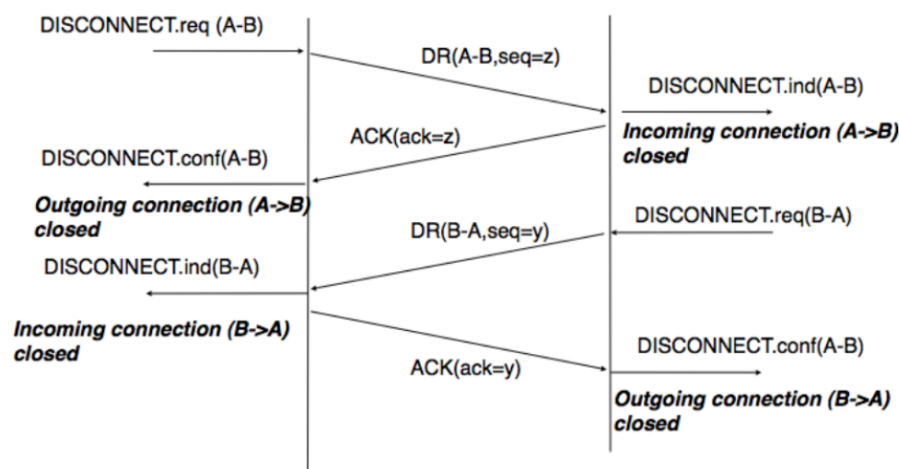


Figura 5.7: Rilascio della connessione in modo graduale [1]

Abrupt

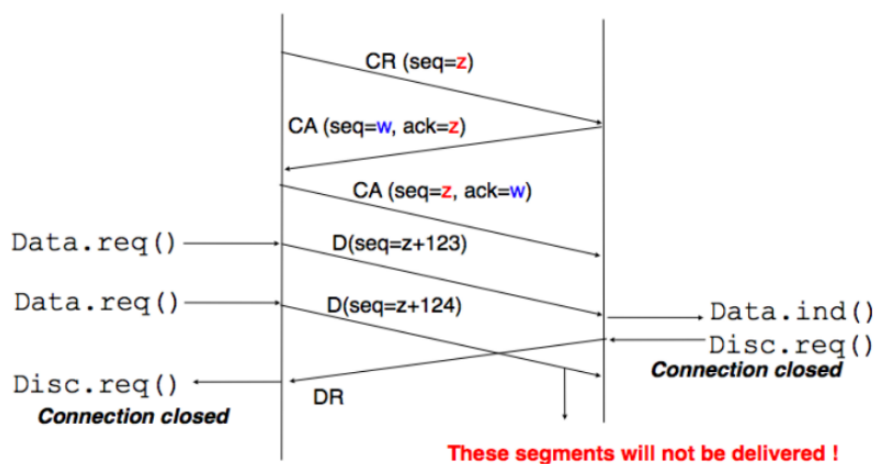


Figura 5.8: Rilascio della connessione in modo brusco [1]

N.B.: Modificare il livello di rete è più complesso rispetto a modificare il livello di trasporto, perché bisognerebbe cambiare tutti i router su internet.

6 Livello Applicativo

6.1 Application Layer

Le applicazioni sono molto diverse tra loro, ma tendono a usare un modello request-response.

Il testo è codificato in ASCII/UTF-8 e i dati binari in big-endian (standard di internet).

6.1.1 Naming System

Gli host sono identificati da indirizzi IP, ma per gli utenti è più semplice usare nomi simbolici (DNS).

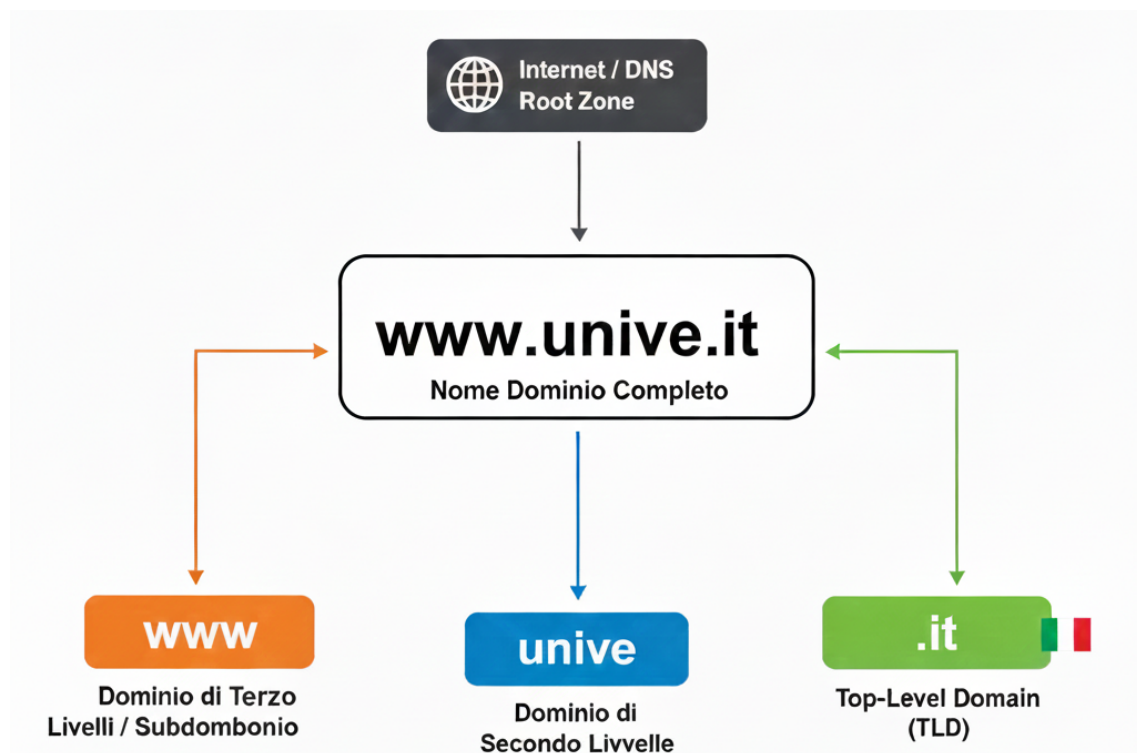


Figura 6.1: Esempio di un nome di dominio

7 Sicurezza di Rete

7.1 Sicurezza di Rete

Sicurezza: processo per rendere una rete "sicura"

7.1.1 Servizi di sicurezza

Service availability: un servizio deve essere disponibile.

- Le risorse limitate sono soggette ad attacchi DoS

Secrecy: i dati scambiati tra due entità non devono essere accessibili da terze parti.

- Si ottiene tramite crittografia.

Integrity: i dati non devono essere alterati durante la trasmissione.

- Un attaccante può intercettare e modificare i dati (bit flipping)
- Si ottiene con checksum, hash functions, MAC

Authentication

Data origin Authentication: il destinatario deve essere in grado di verificare l'identità del mittente.

Peer entity Authentication: due entità che comunicano devono essere in grado di verificare l'identità reciproca.

- Si ottiene con digital signatures

Access Control: solo utenti autorizzati possono accedere a determinati servizi o risorse.

Non repudiation: un'entità non può negare di aver effettuato una certa azione.

- Si ottiene con digital signatures

Anonymity: abilità di comunicare senza rivelare la propria identità.

7.1.2 Modello di sicurezza

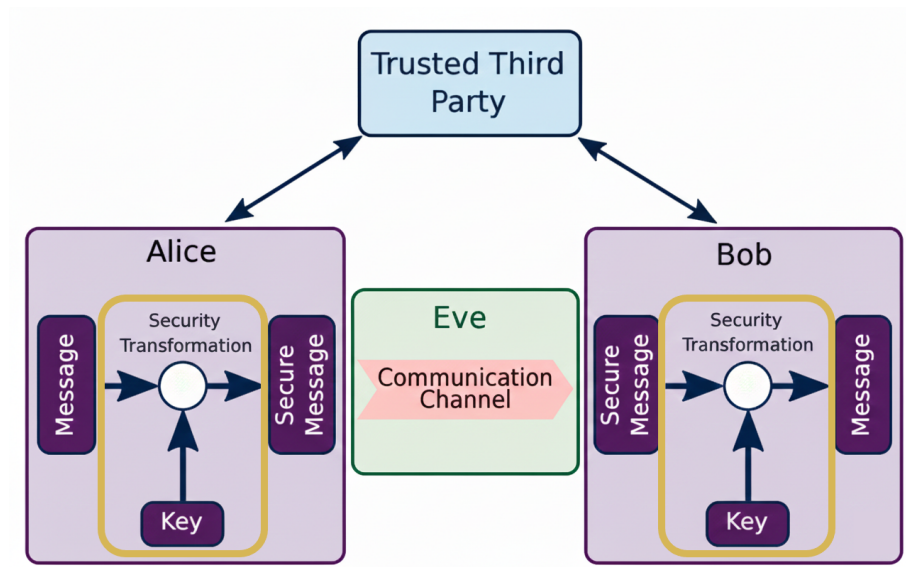


Figura 7.1: Modello di sicurezza di rete

Il modello in figura 7.1 mostra come un sistema di sicurezza di rete protegge le comunicazioni tra due entità.

Mittente e destinatario: entità che comunicano tra loro.

Communication Channel (rete, lettera, ecc.)

Opponent: entità che tenta di compromettere la comunicazione tra mittente e destinatario.

Transformations: operazioni che modificano i dati per proteggerli dagli attacchi dell'opponent.

Trusted Third Party: entità di fiducia che aiuta mittente e destinatario a stabilire una comunicazione sicura.

Trade-off: maggiore sicurezza = maggiore overhead (tempo, risorse, complessità).

Caso studio: Telegram

Due utenti si scambiano messaggi. Il server invia i messaggi da un utente all'altro come trusted third party.

I servizi di sicurezza offerti sono: segretezza, integrità, autenticazione e access control; consideriamo solo la segretezza.

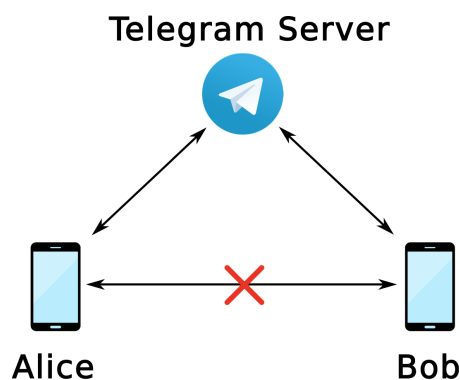


Figura 7.2: Modello di sicurezza di Telegram

- Eve può interagire con Alice/Bob al di fuori di Telegram e può intercettare e modificare i messaggi scambiati.
- Mallory può controllare i server di Telegram.

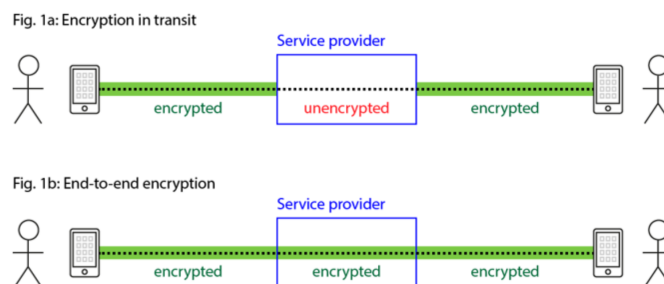


Figura 7.3: Modelli di crittografia

Per garantire la segretezza dei messaggi, possiamo usare uno dei due modelli in figura 7.3:

- **Client-server encryption:** i messaggi sono cifrati tra client e server. Mallory può leggere i messaggi sui server di Telegram.
- **End-to-end encryption:** solo Alice e Bob possono leggere i messaggi.

Purtroppo, il secondo modello è più sicuro, ma è meno usabile (niente chat di gruppo, chiavi salvate su un solo dispositivo, ...). Telegram usa quindi il primo modello di default.

7.1.3 Fondamenti di crittografia

Encryption (kryptos = nascosto, graphein = scrivere): la scienza che si occupa di rendere segrete le informazioni.

- Data integrity (Hash functions)

- Secrecy (Encryption)
- Not repudiation (Digital signatures)
- Authentication, Access control, Anonymity (con una combinazione di tecniche)

Cryptographic Algorithm: sequenza di operazioni matematiche applicate a un messaggio M

- RSA, AES, SHA-256, ...

Cryptographic function: implementazioni di un algoritmo per fornire un servizio di sicurezza (segretezza tramite cifratura)

- AES per segretezza, SHA-256 per integrità, ...

Secure Protocols: insiemi di regole che specificano come usare le funzioni crittografiche per fornire servizi di sicurezza

- TLS usa AES e RSA per fornire segretezza e autenticazione

7.1.4 Funzioni Hash

Funzione unidirezionale che mappa un qualsiasi input di lunghezza variabile a un digest di lunghezza fissa.

Verifica dell'integrità

- Alice calcola il digest del messaggio M : $H(M) = D$ e invia M , D a Bob
- Bob riceve $\{M, D\}$ e calcola $H(M) = D'$
- Se $D = D'$, M non è stato alterato

Esempio: scarico l'iso di ubuntu da un sito secondario, calcolo l'hash e lo confronto con quello ufficiale.

Proprietà delle Funzioni Hash

1. Input M di lunghezza qualsiasi, output D di lunghezza fissa (Ammette collisioni: $H(M1) = H(M2)$)
2. Veloce da calcolare
3. One way: Dato D , è computazionalmente impossibile trovare X tale che $H(X) = D$
4. Weak collision resistance: Dato X , è computazionalmente impossibile trovare Y tale che $H(X) = H(Y)$

5. Strong collision resistance: è computazionalmente impossibile trovare X, Y tali che $H(X) = H(Y)$



Computazionalmente impossibile: un problema per cui non esiste un algoritmo polinomiale ($O(n^k)$) per risolverlo

Brute force: se l'hash è di n bit, ci sono 2^n possibili output.

Se $H(M1) = H(M2)$, $M1$ e $M2$ devono essere molto diversi tra loro (avere un'alta Hamming distance).

H deve essere un PRNG e $H(M)$ deve essere imprevedibile.

7.2 Autenticazione e Cifratura Simmetrica

In 7.1.4 Eve potrebbe intercettare il messaggio M , modificarlo in M' ricalcolando l'hash e inviare a Bob $(M', H(M'))$ e Bob non se ne accorgerebbe.

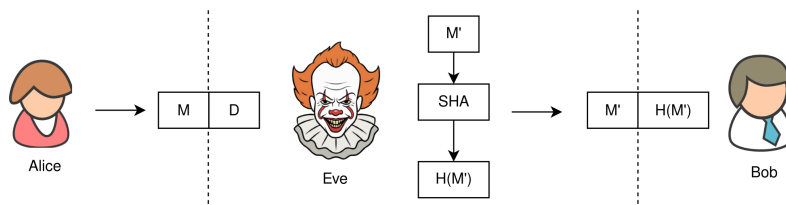


Figura 7.4: Dummy hash attack

Si potrebbero usare due canali separati per inviare M e $H(M)$, ma Eve potrebbe intercettare entrambi.

7.2.1 Meccanismi Challenge-Response

HMAC

Hashed Message Authentication Code: algoritmo crittografico che usa una chiave segreta condivisa K e una funzione hash per calcolare un digest $HMAC(K, M)$ del messaggio M .

$$D = HMAC(M, K) = H((K \oplus opad) \parallel H((K \oplus ipad) \parallel M))$$

$ipad$ e $opad$ sono sequenze di padding costanti che dipendono dalla funzione hash usata. Se K è più lunga del digest si usa $H(K)$.

- + comunicazione sullo stesso canale, autenticazione e integrità
- condivisione della chiave segreta K su un canale sicuro

CRAM-MD5

Challenge-Response Authentication Mechanism: protocollo di autenticazione che usa HMAC per autenticare un client a un server.

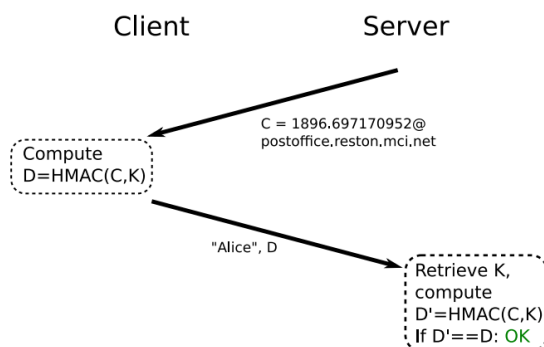


Figura 7.5: Autenticazione con CRAM-MD5

In figura 7.5 Alice riceve dal server un challenge `<timestamp@domain>`, calcola l'HMAC e lo invia al server per autenticarsi come Alice.

Ora deprecato, perché:

1. MD5 non è sicuro
2. Alice non sa se sta comunicando veramente con il server (potremmo ripetere CRAM due volte)

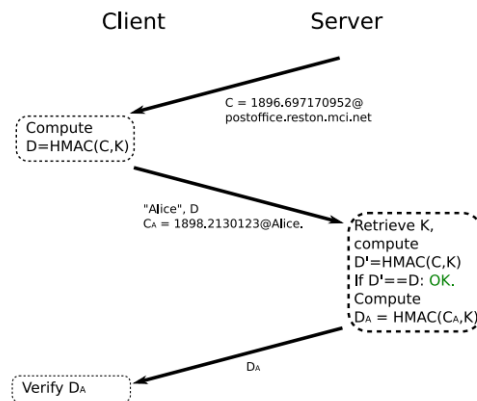


Figura 7.6: Autenticazione bidirezionale con CRAM-MD5

3. Vulnerabile a brute-force offline se Eve intercetta `C, D`

7.2.2 Cifratura a chiave simmetrica

In un sistema di crittografia, il **cipher** è l'algoritmo che, usando una chiave `k`, trasforma il **plaintext** `P` nel **ciphertext** `C` e viceversa.

Principio di Kerchoffs: la sicurezza di un sistema crittografico deve dipendere solo dalla segretezza della chiave `K`, non dall'algoritmo.

I cifrari pubblici sono sicuri, perché molte persone hanno cercato di romperli, ma non ci sono riuscite. Un cifrario segreto è *snake oil*.

Tecniche di Cifratura di Base

Sostituzione: sostituire un simbolo con un altro simbolo

- Cifrario di Cesare, ROT13, Vigenère
- Vulnerabili a un'analisi statistica delle frequenze dei simboli

Trasposizione: cambiare l'ordine dei simboli.

CERCATE LE LUCI. CREDETE NELLE LUCI

RAETCC EULCEL CE.DIR EETLEN LCEILU

5	3	1	6	2	4
C	E	R	C	A	T
E	L	E	L	U	C
I	.	C	R	E	D
E	T	E	N	E	L
L	E	L	U	C	I

1	2	3	4	5	6
R	A	E	T	C	C
E	U	L	C	E	L
C	E	.	D	I	R
E	E	T	L	E	N
L	C	E	I	L	U

Figura 7.7: Esempio di trasposizione ($K = 531624$)

One-Time-Pad

Cifrario perfetto che usa una chiave K lunga quanto il messaggio M , generata casualmente e usata una sola volta.

1. Alice genera una chiave K di n bit e la condivide con Bob in un canale sicuro
2. Per cifrare un messaggio M di $m < n$ bit, Alice usa una porzione K' di K di m bit e calcola $C = M \oplus K'$
3. K' viene usata una sola volta
4. Bob decifra C calcolando $M = C \oplus K'$

Non c'è correlazione tra M e C , OTP è teoricamente impossibile da rompere.

Esempio

$K = 1100100111010110$, $M = 00101101$

$C = 00101101 \oplus 11001001 = 11100100$

Il prossimo messaggio userà la porzione successiva di K .

Perché OTP è "perfetto"?

Se $C = 101$, K è lunga 3 bit e ogni possibile messaggio M di 3 bit è equiprobabile, come mostrato in tabella 7.1.

c	101							
k	000	001	010	011	100	101	110	111
$m' = k \oplus c$	101	100	111	110	001	000	011	010

Tabella 7.1: Brute-force su OTP

Limitazioni

- Se porzioni di K sono ripetute, diventa un algoritmo di sostituzione
- K deve essere condivisa in un canale sicuro e deve essere lunga quanto il messaggio.

Algoritmi moderni

Usano chiavi di lunghezza fissa e applicano numerose sostituzioni e trasposizioni al messaggio per ridurre al minimo la correlazione tra M e C.

Usano un **Feistel scheme**: dividono il messaggio in blocchi e applicano sostituzioni e trasposizioni in più round (DES).

Non sempre gli algoritmi sono stati computazionalmente impossibili da rompere (RC4 per il Wi-Fi, non era pubblico).

7.2.3 Encrypt-then-MAC

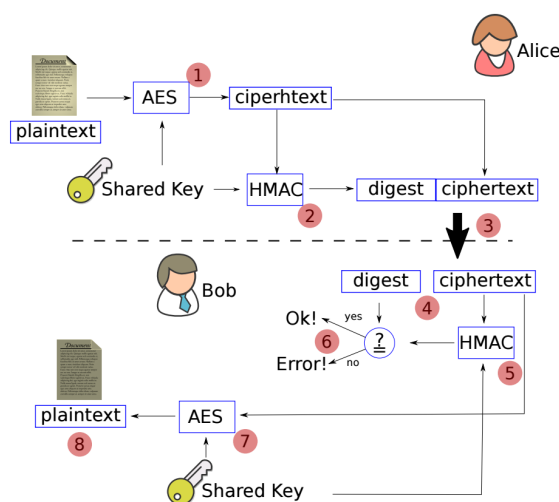


Figura 7.8: Encrypt-then-MAC

Lo schema in figura 7.8 garantisce segretezza, integrità e data origin authentication.

7.3 Cifratura asimmetrica - PKE

La cifratura a chiave asimmetrica usa una chiave pubblica per cifrare i messaggi e una chiave privata per decifrarli.

Questo approccio risolve il problema della distribuzione sicura delle chiavi, poiché la chiave pubblica può essere condivisa liberamente senza compromettere la sicurezza della comunicazione.

Aggiunge overhead, quindi è usata per scambiare chiavi simmetriche per comunicare in modo efficiente.

7.3.1 Aritmetica modulare

$$a \bmod n = a \% n$$

- $a, n \in \mathbb{Z}$, n solitamente è un numero primo grande

Problema della fattorizzazione intera: Dato un numero m di l cifre, è computazionalmente impossibile trovare i suoi fattori primi $p, q : m = p \times q$ ($\nexists$ algoritmo $O(l^k)$ per fattorizzare).

Problema del logaritmo discreto: Dati $g, n \in \mathbb{Z}$ e $g^a \bmod n$ è computazionalmente impossibile trovare a .

7.3.2 Diffie-Hellman

Protocollo per lo scambio di chiavi segrete su un canale insicuro.

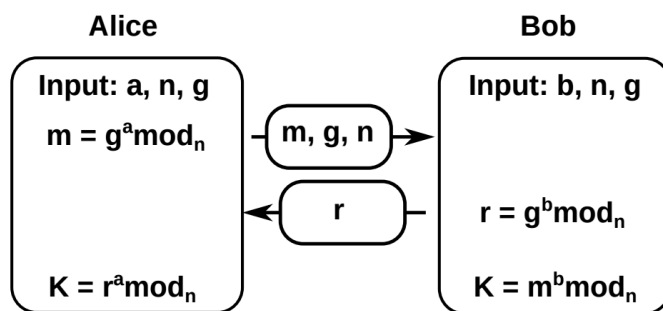


Figura 7.9: Schema del protocollo Diffie-Hellman

a, b numeri casuali, n primo, g radice primitiva modulo n (genera tutti i numeri da 1 a $n - 1$).

Le due parti generano la stessa chiave:

$$\begin{aligned}
 K &= r^a \bmod n = (g^b \bmod n)^a \bmod n \\
 &= g^{ab} \bmod n \\
 &= (g^a \bmod n)^b \bmod n = m^b \bmod n
 \end{aligned} \tag{7.1}$$

Eve potrebbe intercettare la comunicazione e generare una propria chiave segreta come mostrato in figura 7.10.

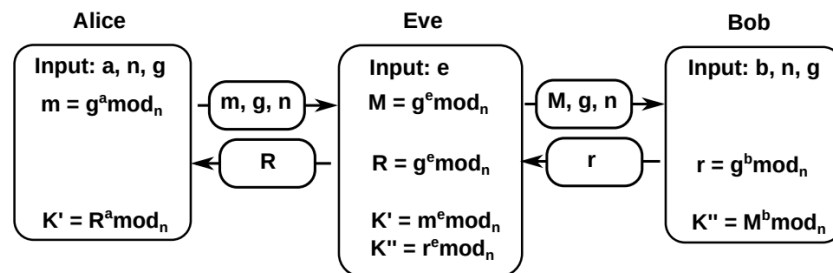


Figura 7.10: Attacco MiTM al protocollo Diffie-Hellman

Per uno scambio di chiavi sicure, il canale di comunicazione deve garantire integrità e autenticazione.

7.3.3 PKE - Public Key Encryption

Alice e Bob hanno una coppia di chiavi: una pubblica (condivisa) e una privata (segreta).

Se Alice cifra un messaggio con pub_B , solo Bob può decifrarlo con $priv_B$.

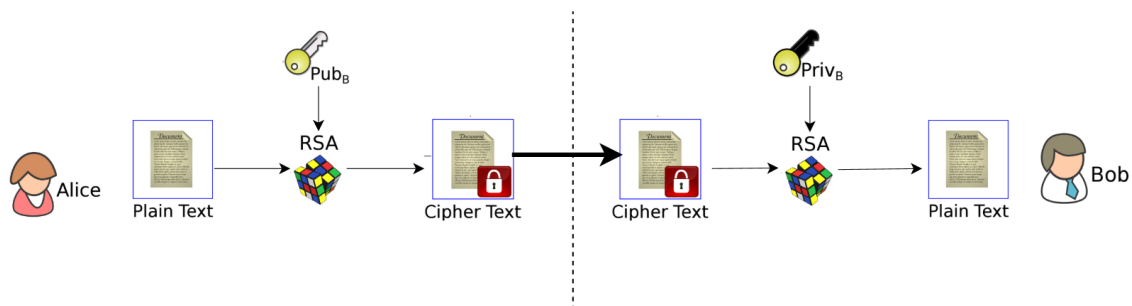


Figura 7.11: Schema PKE con RSA

+ Fornisce segretezza e non serve un canale sicuro per distribuire le chiavi

Vogliamo una funzione $f : D \rightarrow R$ con un parametro t (private key) che sia facile da calcolare e da invertire solo se si conosce t , computazionalmente impossibile altrimenti.

In matematica una funzione o è invertibile o non lo è. Non esiste una funzione che "cambia natura" solo se si conosce un parametro segreto t .

Ci basiamo quindi su delle approssimazioni per ottenere il risultato voluto.

Inoltre, vogliamo che:

Scambio di chiavi con firme digitali

Possiamo firmare le chiavi pubbliche scambiate come mostrato in figura 7.14.

1. $A \rightarrow B : GET \text{ PubB}$
2. $B \rightarrow A : \text{PubB}, \{\text{PubB}\}_{\text{PrivB}}, GET \text{ PubA}$
3. $A \rightarrow B : \text{PubA}, \{\text{PubA}\}_{\text{PrivA}}$
4. $A \rightarrow B : \{M0\}_{\text{PubB}}$
5. $B \rightarrow A : \{M1\}_{\text{PubA}}$

Figura 7.14: Scambio di chiavi con firme digitali

Ma può ancora verificarsi un attacco MiTM.

Prima di verificare la firma, Alice deve ricevere pubB.

1. $A \rightarrow B : GET \text{ PubB}$
2. $B \rightarrow E : \text{PubB}, \{\text{PubB}\}_{\text{PrivB}}, GET \text{ PubA}$
3. $E \rightarrow A : \text{PubE}, \{\text{PubE}\}_{\text{PrivE}}, GET \text{ PubA}$
4. $A \rightarrow E : \text{PubA}, \{\text{PubA}\}_{\text{PrivA}}$
5. $E \rightarrow B : \text{PubE}, \{\text{PubE}\}_{\text{PrivE}}$
6. $A \rightarrow E : \{M0\}_{\text{PubE}}$
7. $E \rightarrow B : \{M0\}_{\text{PubB}}$
8. $B \rightarrow E : \{M1\}_{\text{PubE}}$
9. $E \rightarrow A : \{M1\}_{\text{PubA}}$

Figura 7.15: Attacco MiTM nello scambio di chiavi con firme digitali

7.3.5 Public Key Crypto Performance

Le chiavi pubbliche sono più piccole rispetto alle chiavi private e quindi cifrare/verificare è più veloce rispetto a decifrare/firmare.

Un attaccante può causare un DoS inviando molti dati da decifrare/firmare.

Modalità ibrida: RSA per scambiare una chiave simmetrica, usata poi con cifratura simmetrica e HMAC per comunicare in modo efficiente e sicuro.

Firma degli Hash: Firma il digest del messaggio invece del messaggio intero per migliorare le prestazioni.

Esempio - Dummy key negotiation protocol

1. $A \rightarrow B : \text{PubA}$
2. $B \rightarrow A : \text{PubB}$

3. $A \rightarrow B : \{N_1\}_{\text{PubB}}$
4. $B \rightarrow A : \{N_2\}_{\text{PubA}}$
5. $K_{AB} = H(N_1, N_2); A \rightarrow B : \{M\}_{K_{AB}}$

Esempio - Authentication and Integrity Protocol

Alice invia una copia autenticata di M a Bob. Bob conosce già pubA .

1. $A \rightarrow B : M, D = H(M), C = \{D\}_{\text{PrivA}}$.
2. Bob:
 - calcola $D' = \{C\}_{\text{PubA}}$
 - verifica $D' = D$. la firma è valida, il messaggio è autenticato (altrimenti abortisce).
 - calcola $D'' = H(M)$
 - verifica $D'' = D$. allora il digest è valido, il messaggio è integro.

7.3.6 RSA

Generazione delle chiavi

- Scegli p, q numeri primi grandi
- Calcola $n = pq$ e $\phi(n) = (p-1)(q-1)$
- Scegli $e : 2 < e < \phi(n) : \gcd(e, \phi(n)) = 1$ (coprimi)
- Calcola $d : de \bmod \phi(n) = 1$ (d, e sono inversi modulo $\phi(n)$)

Chiave pubblica: (e, n) , **Chiave privata:** (d, n)

Cifratura: $C = M^e \bmod n$, **Decifratura:** $M = C^d \bmod n$

Esempio

$$\begin{aligned}
 p &= 7, q = 11 \\
 n &= 77, \phi(n) = 60 \\
 e &= 13 \text{ (primo e } \gcd(13, 60) = 1) \\
 d &= 37 \left(d = \frac{k \cdot \phi(n) + 1}{e} \right)
 \end{aligned}$$

Se $M = 20$:

$$C = 20^{13} \bmod 77 = 69$$

$$M = 69^{37} \bmod 77 = 20$$

Risorse aggiuntive: Attacchi su RSA [5], Common modulus attack [6].

7.4 PKI - Public Key Infrastructure

7.4.1 Distribuzione delle chiavi pubbliche

Una chiave è un file con una sequenza di byte casuali.

Per associare una chiave a un'identità:

1. **Key fingerprint**: pubblicare l'hash della pubKey su un sito web sicuro.
2. **Keyserver**: server per chiavi pubbliche, ma non sicuro.
3. **Web of Trust**: social network di utenti che validano l'identità altrui.

7.4.2 Web of Trust

Usato in contesti informali (GPG)



Trust: relazione unidirezionale tra due entità. Se Alice si fida di Carl:

- Alice conosce pubC
- Alice pensa che Carl non si comporterà male

Alice si fida di Carl, che fornisce $S = \{(Pub_A, "Alice")\}_{Priv_C}$. Ora chiunque si fidi di Carl può fidarsi di Alice.

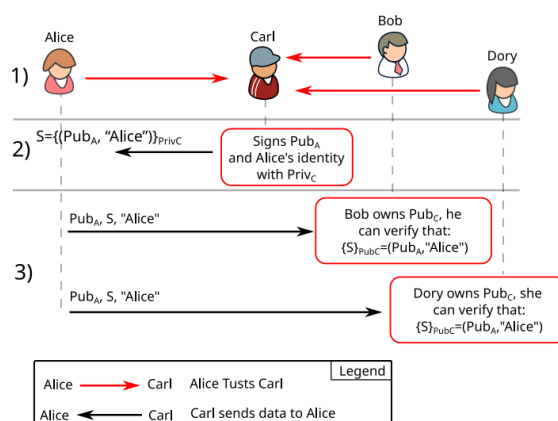


Figura 7.16: Relazioni di fiducia

Si può creare una rete di fiducia. La fiducia è **unidirezionale**.

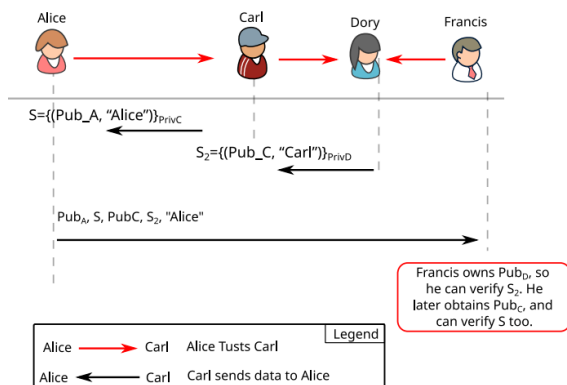


Figura 7.17: Trust chain

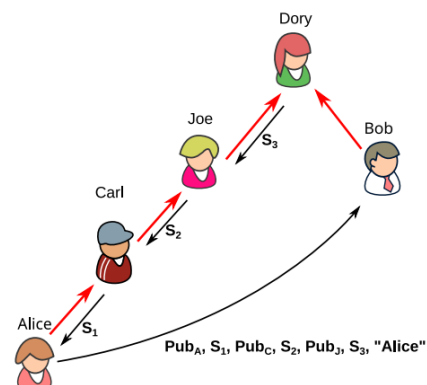


Figura 7.18: Trust path

In figura 7.18, Bob può inviare dati a Alice in sicurezza, ma non viceversa.

C, J sono **certifiers**.

Anche se Dory certifica Bob, Alice non può verificare la firma, perché non ha pubD.



Key signature: Un'entità C può firmare la pubKey di A con l'identità di A per certificare che pubA appartiene ad A.



Trust delegation: Se A e B hanno un *trust anchor* comune D, possono scambiarsi le chiavi in sicurezza.



Fiducia asimmetrica: La fiducia ha una direzione e i dati si muovono nella direzione opposta.

7.4.3 PKI: Architettura e dettagli

Infrastruttura in cui c'è un'entità di cui tutti si fidano (*trusted third party*) e di cui conoscono la pubKey: **Certification Authority (CA)**.

Può essere locale o globale.

Formato certificato X.509

Certificato: chiave firmata con metadati.

Issuer Name
Cert. Serial Number
Validity Period
Subject Name
Subject Public Key
Certificate Signature by the CA

- **Issuer name:** Id CA
- **Serial number:** Id certificato
- **Subject name:** Id proprietario (persona, host ecc.)
- **Certificate signature:** firma dell'hash del certificato con $Priv_{CA}$

Figura 7.19: Certificato

Richiesta certificato

1. Alice genera la coppia di chiavi (pubA, privA).
2. Alice invia un **Certificate Signing Request (CSR)** alla CA

$$CSR = (PubA, "Alice", \{PubA, "Alice"\}_{PrivA})$$

3. La CA verifica l'identità di Alice (con pubA ed eventuali documenti)
4. La CA crea il certificato

$$Cert = (CSR, metadata, \{(CSR, metadata)\}_{Priv_{CA}})$$

Questo processo è richiesto una volta sola per **validity period**. La CA non è coinvolta nelle comunicazioni successive e non conosce le chiavi private.

Alla fine il contenuto del certificato è:

$$Data = (PubA, "Alice", Issuer, validity \dots)$$

$$D = H(Data)$$

$$Cert = (Data, \{D\}_{Priv_{CA}})$$

Il certificato PKI è una key signature del WoT.

Verifica certificato

Bob riceve il certificato di Alice:

- Check identità (Subject name = "Alice"), scadenza e CA (mi fido e conosco pubCA)

- Autenticità: decifro la firma con pubCA per ottenere D
- Integrità: calcolo $D' = H(Data)$ e verifico che $D = D'$



Certificato valido: Bob può usare pubA se il certificato passa la verifica

Certificare nomi di dominio

Le CA possono certificare nomi di dominio per i web server.

- I browser memorizzano una lista di CA fidate (Root CA)
- Le root CA si autocertificano (Microsoft, Apple)

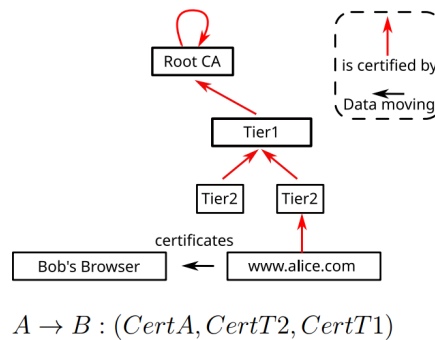


Figura 7.20: CA che certifica un dominio

Come mostrato in figura 7.20, quando Bob visita `www.alice.com`, Alice fornisce tutti i certificati fino alla Root CA.

Alla fine il browser di Bob conosce pubA del web server di Alice, ma il server non conosce Bob (fiducia asimmetrica).

Revoca certificati

Certificate revocation list (CRL): lista di certificati revocati pubblicata periodicamente dalla CA. Firmata con $Priv_{CA}$ e scaricata dal browser.

Sicurezza

Se una CA è compromessa, può emettere certificati falsi per qualsiasi dominio e impersonare qualsiasi sito web.

DigiNotar (2011)

7.4.4 PKE e PKI in pratica

- Con i certificati scambiamo almeno una pubKey tra server e browser
- Si genera una session key con DH
- La session key è usata con cifratura simmetrica e HMAC per mettere in sicurezza il traffico
- Il server può richiedere al client una password verification

Esercizio

`example.com` deve essere certificato da una CA tier 2

$$\begin{aligned} A \rightarrow CA : \text{CSR} &= (\text{Pub}_A, \text{"example.com"}, \{\text{Pub}_A, \text{"example.com"}\}_{\text{Priv}_A}) \\ CA \rightarrow A : \text{Cert} &= (\text{Pub}_A, \text{"example.com"}, \text{metadata}, \{\text{H}(\text{Pub}_A, \dots)\}_{\text{Priv}_{CA}}) \end{aligned}$$

Se Bob visita `example.com`: $A \rightarrow B : (\text{Cert}_A, \text{CertT2}, \text{CertT1})$

Risorse aggiuntive: Let's Encrypt CA hierarchy [7], AWS CA hierarchy [8]

Parte II

Protocolli e Implementazioni

8 Protocolli a livello Applicativo

8.1 DNS - Domain Name System

Protocollo che mappa nomi di dominio a indirizzi IP.

Nome di dominio: stringa separata da punti (`www.unive.it`)

DNS Query: `www.unive.it`

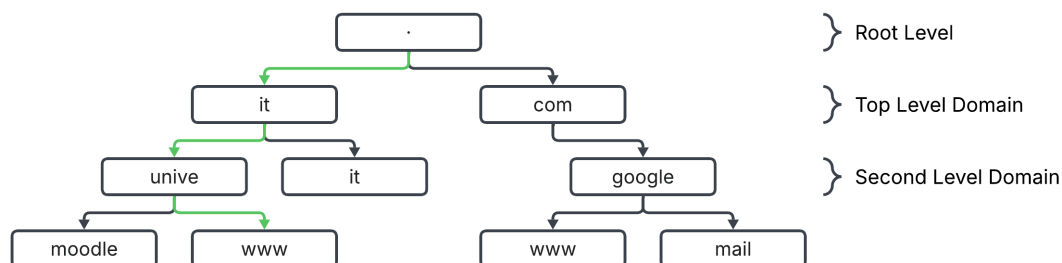


Figura 8.1: Gerarchia DNS

- Gerarchia dei nomi di dominio con TLD (`.it`, `.com`, `.org`)
- ICANN organizza i TLD e assegna i nomi di dominio
- I TLD delegano la gestione dei sottodomini a Domain Name Registrars (es. Aruba, GoDaddy)
- Chi è delegato sotto può aggiungere dei domini senza che chi sta sopra lo sappia

Fully Qualified Domain Name (FQDN): nome di dominio completo, include tutti i livelli della gerarchia (`www.unive.it.`)

Chi possiede un dominio è **autoritativo** (responsabile) per tutti i suoi sottodomini (es. `unive.it` possiede `www.unive.it`, `mail.unive.it`, ecc.)

Zone: sottoalbero dell'albero dei domini gestito da un server DNS autoritativo.

`www.example.com`, `mail.example.com`, `ns.example.com` sono nella stessa zona.

Un server autoritativo per una zona può essere al di fuori della zona stessa (`google.com` autoritativo per `example.com`).

Registrazione di un nome di dominio

- Creare una pagina web e configurare un web server su un IP pubblico.
- Registrare un nome di dominio tramite un Domain Name Registrar.
- Configurare un server DNS autoritativo per il dominio (example.com → 1.2.3.4.5).
- Il registrar comunica al TLD (.com) il nuovo sottodominio.

8.1.1 Risoluzione DNS

I browser conoscono i 13 root server DNS, i quali forniscono gli indirizzi dei server DNS autoritativi per i TLD.

Dig: programma per interrogare i server DNS.

Cerchiamo l'indirizzo IP di `www.unive.it` partendo da un root server.
(198.41.0.4 root server iterativo di Verisign inc, 8.8.8.8 Google Public DNS ricorsivo)

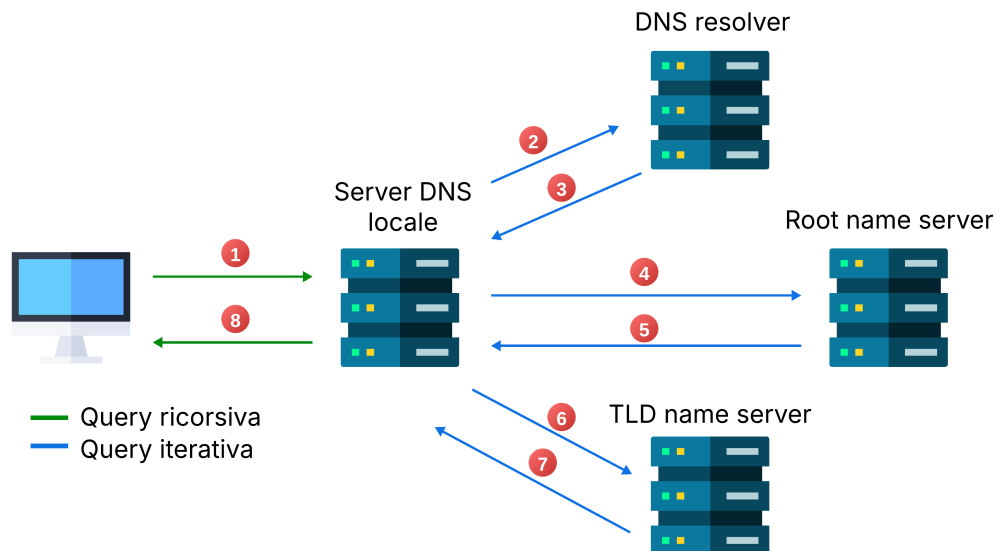
```
1 # -- Query al root server --
2 $ dig @198.41.0.4 www.unive.it
3 ;; QUESTION SECTION:
4 ;www.unive.it.                IN      A
5
6 # no answer section
7 # ask these servers, authoritative for .it
8 ;; AUTHORITY SECTION:
9 it.                          172800  IN      NS      a.dns.it.
10 it.                          172800  IN      NS      m.dns.it.
11 ...
12
13 # IP addresses of the authoritative servers
14 ;; ADDITIONAL SECTION:
15 a.dns.it. 172800 IN A      194.0.16.215
16 a.dns.it. 172800 IN AAAA  2001:678:12:0:194:0:16:215
17 m.dns.it. 172800 IN A      217.29.76.4
18 m.dns.it. 172800 IN AAAA  2001:1ac0:0:200:0:a5d1:6004:2
19 ...
20
21 # -- Query a un server autoritativo per .it --
22 $ dig @194.0.16.215 www.unive.it
23 # still no answer section
24
25 ;; AUTHORITY SECTION:
26 unive.it. 3600 IN NS      algal.unive.it.
27 ...
28
29 ;; ADDITIONAL SECTION:
30 algal.unive.it. 3600 IN A      157.138.1.8
31
32 # -- Query a un server autoritativo per unive.it --
33 $ dig @157.138.1.8 www.unive.it
34
35 ;; ANSWER SECTION:
36 www.unive.it. 86400 IN A      157.138.7.88
```

L'indirizzo di `www.unive.it` è 157.138.7.88.

Risoluzione iterativa: se il server interrogato non conosce la risposta, fornisce una risposta parziale, indicando un altro server da interrogare per avvicinarsi alla risposta.

Risoluzione ricorsiva: il client delega l'intera risoluzione al server DNS, che si occupa di fare tutte le richieste necessarie per trovare la risposta finale.

La risoluzione ricorsiva non è ammessa dai root server e di solito è consentita solo tra server DNS locali.



Caching DNS: i server DNS memorizzano le risposte alle query per un certo periodo (TTL) per ridurre il carico e velocizzare le risoluzioni future.

8.1.2 DNS - Dettagli del protocollo

DNS usa principalmente una comunicazione **UDP:53** con modello client-server: invio di pacchetti piccoli, non serve stabilire una connessione.

Usa TCP per zone transfer (AXFR) e messaggi lunghi (>512 byte).

Formato messaggio

Header DNS

0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5		
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+																	
ID																	
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+																	
QR		Opcode				AA TC RD RA		Z		RCODE							
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+																	
QDCOUNT																Number of questions	
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+																	
ANCOUNT																Number of answer	
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+																	
NSCOUNT																Number of authority	
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+																	
ARCOUNT																Number of additional	
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+																	

- Transaction ID: match query-answer (random per evitare spoofing)
- Flags:
 - QR: query (0) o risposta (1)
 - AA: Authoritative Answer (risposta da un server autoritativo)
 - RD: Recursion Desired (il client richiede risoluzione ricorsiva)
 - RA: Recursion Available (il server supporta risoluzione ricorsiva)
 - Z: non usato (0)
 - RCODE: codice di errore, OK (0), errori (1-5)

Body - Record Sections

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+															
Question Question for the NS															
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+															
Answer RRs answering the question															
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+															
Authority RRs pointing toward an authority															
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+															
Additional RRs with additional information															
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+															

Resource Record

1	1	1	1	1	1										
0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+															
/ NAME /															
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+															
TYPE															
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+															
CLASS															
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+															
TTL															
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+															
RDLENGTH															
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+															
/ RDATA /															
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+															

#	name	TTL	class	type	RDATA
	google.com.	300	IN	A	142.250.184.206
	google.com.	300	IN	AAAA	2a00:1450:4002:402::200e
	google.com.	800	IN	NS	ns1.google.com.
	google.com.	600	IN	MX	10 smtp.google.com.
	google.com.	3600	IN	TXT	"v=spf1 include:_spf.google.com ~all"

- NAME: nome di dominio
- TYPE: tipo di record
 - A: indirizzo IPv4
 - AAAA: indirizzo IPv6
 - NS: nameserver autoritativo
 - MX: mail exchange
 - CNAME: canonical name (alias)
 - SOA: info sulla zona
 - TXT: testo libero
- TTL: time to live per caching (in secondi)
- RDLENGTH: lunghezza del campo RDATA
- RDATA: dati del record

Gli header a basso livello hanno dimensione fissa, per essere elaborati e trasmessi velocemente. Quindi anche gli header DNS sono fissi e hanno bit riservati per usi futuri, mentre il body può variare in lunghezza e formato.

8.1.3 Reverse DNS

Reverse lookup: dato un indirizzo IP, trovare il nome di dominio associato.

Un nome di dominio può essere registrato con un record PTR che mappa un indirizzo IP a un nome di dominio

5.3.2.1.in-addr.arpa

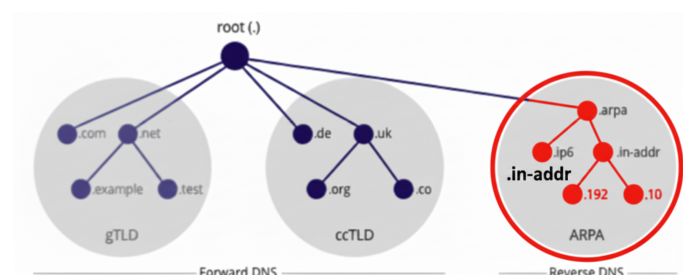


Figura 8.2: Gerarchia DNS - Risoluzione inversa

Come mostrato in figura 8.2, la risoluzione inversa parte dal dominio di primo livello `in-addr.arpa` e procede verso il basso.

Listing 8.1: Reverse lookup con dig

```
1 # -- Query NS autoritativi per in-addr.arpa --
2 $ dig in-addr.arpa NS
3 ;; ANSWER SECTION:
4 in-addr.arpa. 3600 IN NS a.in-addr-servers.arpa.
5
6 # -- Query server autoritativo per in-addr.arpa --
7 dig @a.in-addr-servers.arpa 88.7.138.157.in-addr.arpa PTR
8 ;; AUTHORITY SECTION:
9 157.in-addr.arpa. 86400 IN NS r.arin.net.
10
11 # -- Query server autoritativo per 157.in-addr.arpa --
12 $ dig @r.arin.net 88.7.138.157.in-addr.arpa PTR
13 ;; AUTHORITY SECTION:
14 7.138.157.in-addr.arpa. 86400 IN NS ns1.garr.net.
15
16 # -- Query server autoritativo per 7.138.157.in-addr.arpa --
17 $ dig @ns1.garr.net 88.7.138.157.in-addr.arpa PTR
18 ;; ANSWER SECTION:
19 88.7.138.157.in-addr.arpa. 86400 IN PTR www.unive.it.
```

Protocollo WHOIS - RDAP

Protocollo per interrogare database che contengono informazioni sui nomi di dominio registrati (proprietario, data di registrazione, data di scadenza, nameserver associati, ecc.)

```
1 $ whois unive.it
2 ...
3 $ host -v unive.it
4 ...
```

8.1.4 Gestione del traffico e performance

Ridondanza dei server per garantire Availability e bilanciamento del carico: il Round Robin distribuisce le query variando l'ordine della lista Authority.

Un TTL alto riduce il traffico sul server, ma rallenta l'aggiornamento della cache e limita l'efficacia del Round Robin (i client non ruotano gli IP finché il record non scade)

```
1 # -- Monitoraggio TTL (seconda colonna) --
2 $ dig google.com
3 ;; ANSWER SECTION:
4 google.com. 250 IN A 142.250.180.174
5
6 $ sleep 30; dig google.com
7 ;; ANSWER SECTION:
8 google.com. 220 IN A 142.250.180.174
9
10 # -- TTL scaduto, nuova risoluzione --
11 $ sleep 220; dig google.com
12 ;; ANSWER SECTION:
13 google.com. 300 IN A 142.250.180.174
```

Possono esserci più server DNS con lo stesso IP (IP anycast - BGP)

Record non-terminali: un server autoritativo per un dominio risponde con un altro dominio.

```
1 $ dig @157.138.1.8 moodle.unive.it
2 ;; ANSWER SECTION:
3 moodle.unive.it. 86400 IN CNAME MT2KLB7-1993779370.eu-west-1.elb.amazonaws.com.
```

8.1.5 Sicurezza - slide extra

DNS non è sicuro di design ed è ancora utilizzato così.

vspacel em

Esistono security upgrades:

- DNSSEC (DNS Security Extensions): firme digitali per autenticare le risposte DNS (auth, integrity)
- DoH (DNS over HTTPS) e DoT (DNS over TLS): trasmissione su canali sicuri (segretezza, le informazioni sono pubbliche ma potremmo volerle nascondere) (soluzione preferibile)

DNS poisoning (Kaminsky attack): attaccante risponde a una query DNS prima del server legittimo, cercando di indovinare l'ID della query e inserendo una risposta falsa in cache.

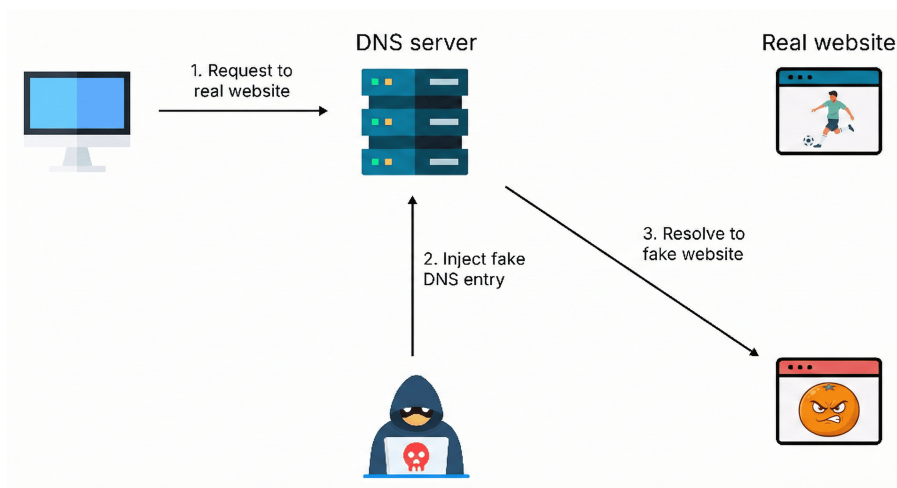


Figura 8.3: DNS poisoning

DNS DoS: attaccante inonda un server DNS con richieste, rendendolo non disponibile.

Reflection attack: attaccante invia richieste DNS con indirizzo IP sorgente falsificato (IP della vittima) a server DNS aperti, che rispondono alla vittima con risposte più grandi, sovraccaricandola.

- Il protocollo è stateless
- Le risposte sono più grandi delle richieste

Cookie DNS: meccanismo per mitigare attacchi di spoofing, usa cookie per autenticare i client. Il server genera il cookie combinando l'IP del client, il suo client cookie e un segreto del server tramite un hash. È un sistema stateless, in quanto il server memorizza solo il segreto e non i cookie dei client.

8.2 HTTP - HyperText Transfer Protocol

8.2.1 URI - Uniform Resource Identifier

Stringa di caratteri che identifica una risorsa su una rete.

URL (Uniform Resource Locator) e URN (Uniform Resource Name).

Formato URI

`scheme` ":" "//" `authority` [`path`] ["?" `query`] ["#" `fragment`]

- **scheme**: protocollo (http, https, ftp, mailto)
- **authority**: dominio o indirizzo IP del server
può essere preceduto da "user:password@" e seguito da ":port"
- **path**: percorso della risorsa sul server
- **query**: stringa di parametri (opzionale)
- **fragment**: riferimento a una sezione specifica della risorsa (opzionale)

Esempi

`http://tools.ietf.org/html/rfc3986.html`

`mailto:infobot@example.com?subject=current-issue`

`https://facebook.com.js12fweijejn12i3j1i2j3211kfreej49.ru`

`ftp://cnn.example.com&story=breaking_news@10.0.0.1/top_story.htm`

IDN Homograph Attack

Gli IDN (Internationalized Domain Names) permettono l'uso di caratteri non ASCII nei nomi di dominio. Ciò può essere sfruttato per creare nomi di dominio ingannevoli.

8.2.2 HTML - HyperText Markup Language

Linguaggio di markup standard per creare pagine web.

```
1 <html>
2   <head>
3     <title>Vault 111 - Terminal 01</title>
4   </head>
5   <body>
6     <h1>War never changes</h1>
7     <a href="https://www.google.com">Where is Shaun?</a>
8     <br><br>
9     
10   </body>
11 </html>
```

Possiamo rendere la pagina dinamica con JS (client-side) o PHP (server-side).

Client-side: codice eseguito nel browser dell'utente.

Server-side: codice eseguito sul server web prima di inviare la pagina al browser. È usato per inviare query al DB, autenticazione, ecc.

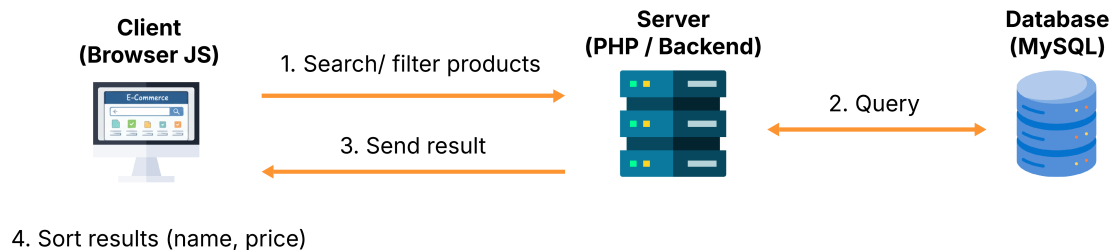


Figura 8.4: Interazione client-server

Cascading Style Sheets (CSS): linguaggio usato per modificare l'aspetto delle pagine web.

```
1 <html data-year="1983">
2   <head>
3     <link href="style.css" rel="stylesheet" type="text/css">
4   </head>
5   <body>
6     <p>Friends-dont-lie</p>
7   </body>
8 </html>
```

```
1 p {
2   color: #F3C010; /* Eggo Waffle Gold */
3   font-size: 11px;
4 }
```

I siti caricano spesso librerie JS, stili CSS e immagini da siti esterni.

8.2.3 HTTP - Dettagli del protocollo

Protocollo di comunicazione client-server per il trasferimento di risorse web (pagine HTML, immagini, video, ecc.).

Usa **TCP:80**

Stateless: ogni richiesta è indipendente dalle altre.

Formato messaggi

Request

- Metodo, URI, versione HTTP
- Header
- MIME (doc. opzionale)

Response

- Status code
- Header
- MIME (doc. opzionale)

Metodi

GET, POST (invia dati al server come risorsa MIME), PUT, DELETE, HEAD, OPTIONS.

Esempio GET: `http://www.w3.org/MarkUp/`

- Apre una connessione TCP:80 a `www.w3.org`
- Invia la richiesta: `GET /MarkUp/ HTTP/1.0`

Headers

Simili a SMTP, forniscono informazioni aggiuntive sulla richiesta o risposta.

- **User-Agent:** informazioni sul client (browser, OS)
- **Referrer:** URL della pagina precedente
- **Host:** dominio del server
- **Server:** informazioni sul server (non usato per problemi di sicurezza)
- **Connection: keep-alive:** mantiene la connessione aperta per più richieste
- **Set-Cookie:** richiede al client di memorizzare un cookie
- **Cookie:** invia i cookie memorizzati al server

Status Codes

- 1xx: Informational
- 2xx: Success (200 OK)
- 3xx: Redirection (res moved)
- 4xx: Client Error
- 5xx: Server Error

Esempio

```
GET / HTTP/1.1
Host: www.kame.net
User-Agent: Mozilla/5.0
Connection: Keep-Alive
```

```
HTTP/1.1 200 OK
Date: Fri, 19 Mar 2010 09:23:37 GMT
Server: Apache/2.0.63 (FreeBSD) PHP/5.2.12
Keep-Alive: timeout=15, max=100
Connection: Keep-Alive
Content-Length: 3462
Content-Type: text/html
<html>...</html>
```

Cookies: dati salvati lato client per mantenere lo stato tra richieste HTTP.

Possono contenere informazioni di sessione, preferenze utente, dati di tracciamento, ecc.

HTTP 2.0

- Richieste e risposte in formato binario e non più testuale
- Pagine divise in frame e inviate in parallelo sulla stessa connessione TCP
- HoL (Head of Line) blocking ridotto: se un frame è bloccato, gli altri possono essere processati
- Il server può inviare risorse al client senza una richiesta esplicita (server push)

8.2.4 Server Proxy

Intermediario tra client e server che inoltra le richieste e le risposte e può salvare in cache le risorse per migliorare le prestazioni.

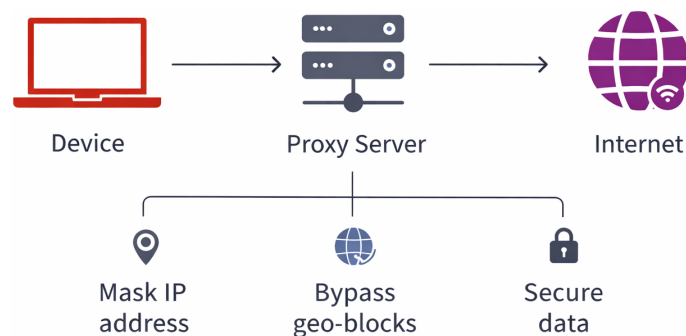


Figura 8.5: Server proxy

Header cache-control: direttive di caching per client e proxy.

- **no-store:** il server non può memorizzare la risorsa in cache
- **max-age = N:** la risorsa è valida per N secondi
- **no-cache:** il client deve verificare con il server prima di usare la risorsa in cache

Il server invia la risorsa

```
HTTP/1.1 200 OK
Cache-Control: no-cache
Last-Modified: Wed, 01 Jan 2025 10:00:00 GMT
```

Il client chiede al server se la risorsa è cambiata

```
GET /news.html HTTP/1.1
If-Modified-Since: Wed, 01 Jan 2025 10:00:00 GMT
```

Se la risorsa non è cambiata, il server risponde con:

```
HTTP/1.1 304 Not Modified
```


Reverse Proxy

Intermediario che riceve le richieste dai client e le inoltra ai server interni, spesso usato per bilanciare il carico, migliorare la sicurezza e gestire la cache.

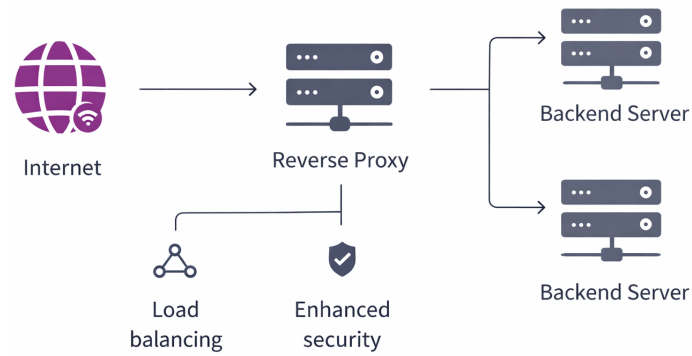


Figura 8.6: Reverse proxy

8.3 SMTP - Simple Mail Transfer Protocol

In un architettura di posta elettronica, il client interagisce con un server di posta per inviare e ricevere mail.

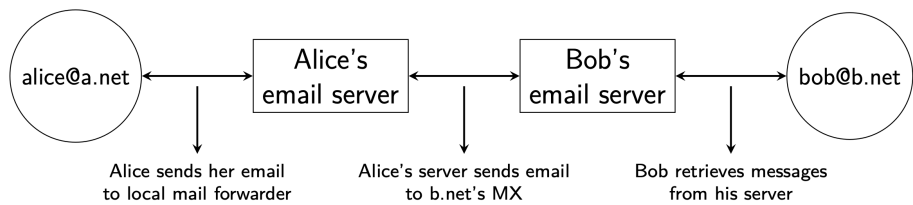


Figura 8.7: Architettura mail [1]

Mail User Agent (MUA): client di posta che permette all'utente di comporre l'email.

Mail Transfer Agent (MTA): server di posta che si occupa di inoltrare l'email al server di destinazione.

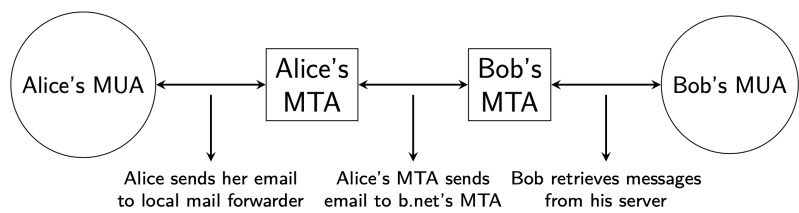


Figura 8.8: Architettura mail con MUA e MTA [1]

Internet Message Format (**IMF**) e Multipurpose Internet Mail Extensions (**MIME**): standard che definiscono il formato delle mail.

Standard MIME

Estende il formato delle mail per supportare file binari, HTML, ecc.

MIME-Version	1.0
Content-Type	text/plain multipart/mixed (testo + allegato) multipart/alternative (stesso msg in formati diversi)
Content-Transfer-Encoding	ASCII, UTF-8, Base64...

Struttura **multipart**s ad albero divise da boundary.

Mail 8.2: Mail MIME

```
Date: Mon, 1 Jan 2024 10:00:00 +0000
From: Alice <alice@mail.com>
To: Bob <bob@mail.com>
Subject: Meeting Reminder
```

```
MIME-Version: 1.0
```

```
Content-Type: multipart/mixed; boundary="step"

--step
Content-Type: text/plain; charset="UTF-8"

Hello World!

--step
Content-Type: image/png; name="image.png"
Content-Disposition: attachment; filename="image.png"
Content-Transfer-Encoding: base64
Content-ID: <image1>

iVBORw0KGgoAAA...==
```

8.3.1 Formato mail

Header

Message-Id	Id univoco mail
From, To, Subject, Date, ...	/
In-reply-to	Id mail a cui si risponde
Received	Traccia i server attraversati
X-	Campi personalizzati

Corpo: Messaggio di testo con righe che terminano con CRLF.

Esempi

Mail 8.3: Mail semplice

```
From: Alice <alice@mail.com>
To: Bob <bob@mail.com>, Carol <carol@mail.com>
Subject: Meeting Reminder
Date: Mon, 1 Jan 2024 10:00:00 +0000
Hi Bob and Carol,
Just a reminder about our meeting tomorrow at 10 AM.
```

Mail 8.4: Mail con Received

```
Received: from smtp3.sgsi.ucl.ac.be (Unknown [10.1.5.3]) by mmp.sipr-dc.ucl.ac.be
(Sun Java(tm) System Messaging Server 7u3-15.01 64bit (built Feb 12 2010)) with ESMTPE id <0
KYY00L85LI5JLE0@mmp.sipr-dc.ucl.ac.be>; Mon, 08 Mar 2010 11:37:17 +0100 (CET)
Received: from mail.ietf.org (mail.ietf.org [64.170.98.32]) by smtp3.sgsi.ucl.ac.be (Postfix) with ESMTPE
id B92351C60D7; Mon, 08 Mar 2010 11:36:51 +0100 (CET)
Received: from [127.0.0.1] (localhost [127.0.0.1]) by core3.amsl.com (Postfix) with ESMTPE id F066A3A68B9
; Mon, 08 Mar 2010 02:36:38 -0800 (PST)
Received: from localhost (localhost [127.0.0.1]) by core3.amsl.com (Postfix) with ESMTPE id A1E6C3A681B
for <rrg@core3.amsl.com>; Mon, 08 Mar 2010 02:36:37 -0800 (PST)
Received: from mail.ietf.org ([64.170.98.32]) by localhost (core3.amsl.com [127.0.0.1]) (amavisd-new,
port 10024) with ESMTPE id erw8ih2v8VQa for <rrg@core3.amsl.com>; Mon, 08 Mar 2010 02:36:36 -0800 (
PST)
Received: from gair.firstpr.com.au (gair.firstpr.com.au [150.101.162.123]) by core3.amsl.com (Postfix)
with ESMTPE id 03E893A67ED for <rrg@irtf.org>; Mon, 08 Mar 2010 02:36:35 -0800 (PST)
Received: from [10.0.0.6] (wira.firstpr.com.au [10.0.0.6]) by gair.firstpr.com.au (Postfix) with ESMTPE
id D0A49175B63; Mon, 08 Mar 2010 21:36:37 +1100 (EST)
Date: Mon, 08 Mar 2010 21:36:38 +1100
From: Robin Whittle <rw@firstpr.com.au>
Subject: Re: [rrg] Recommendation and what happens next
In-reply-to: <C7B9C21A.4FAB%tony.li@tony.li>
To: RRG <rrg@irtf.org>
Message-id: <4B94D336.7030504@firstpr.com.au>
```

Nella mail 8.4, **from** è il mittente e **by** è il destinatario

La mail parte dall'australia alle 21:36 (UTC+11), passa per un IRTF mailing list americano, dove è inviata a vari server interni (localhost) per dei controlli e arriva in Belgio alle 11:37 (UTC+1).

8.3.2 SMTP - Dettagli del protocollo

Protocollo text-based standard per gestire l'invio delle mail tra MUA/MTA.

Invia comandi al server SMTP sulla porta **25** usando **TCP**

Il server risponde con codici di stato:

2xx	Successo
3xx	In attesa di ulteriori informazioni
4xx	Errore temporaneo (es. server sovraccarico, riprovare più tardi)
5xx	Errore permanente (es. indirizzo inesistente o sintassi errata)

220: server connesso, 250: mail accettata, 450: mailbox piena, 550: mailbox inesistente

Sessione SMTP

```
S: 220 smtp.example.com ESMTP MTA information
C: EHLO mta.example.org # avvio sessione
S: 250 Hello mta.example.org, glad to meet you
C: MAIL FROM:<alice@example.org> # mittente
S: 250 Ok
C: RCPT TO:<bob@example.com> # destinatario
S: 250 Ok
C: DATA # inizio corpo mail
S: 354 End data with <CR><LF>.<CR><LF>
C: From: "Alice Doe" <alice@example.org>
C: To: Bob Smith <bob@example.com>
C: Date: Mon, 9 Mar 2010 18:22:32 +0100
C: Subject: Hello
C:
C: Hello Bob
C: This is a small message containing 4 lines of text.
C: Best regards,
C: Alice
C: . # fine corpo mail
S: 250 Ok: queued as 12345
C: QUIT # termina sessione
S: 221 Bye
```

8.3.3 Relaying

Il MUA è configurato con il dominio di un server SMTP (Gmail, protonmail)

Relay locale: MTA accetta mail destinate al dominio che gestisce.

Closed relay: MTA non accetta mail destinate a domini esterni da utenti non autenticati.

Autenticazione SMTP

```
S: 220-smtp.example.com ESMTP Server
C: EHLO client.example.com # avvio sessione
S: 250-smtp.example.com Hello client.example.com
# metodi di autenticazione supportati
S: 250-AUTH DIGEST-MD5 CRAM-MD5
S: 250-ENHANCEDSTATUSCODES
S: 250 STARTTLS
C: AUTH CRAM-MD5 # autenticazione con CRAM-MD5 (MSA - TCP:587)
# challenge: timestamp in base64
S: 334 PDQxOTI5NDIzNDEuMTIOMjgONzJAc291cmNlZm91ci5hbmRyZXC1Y211LmVkdT4=
# username | H(password, challenge)
C: cmpzMyB1YzNhNTlmZWQzOTVhYmExZW2Mzy3YzRmNGIOMWFjMA==
S: 235 2.7.0 Authentication successful
```

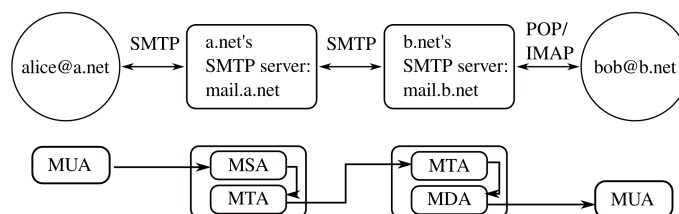


Figura 8.9: Autenticazione SMTP con Mail Submission Agent (MSA) [1]

Risoluzione DNS

Un MTA effettua una risoluzione DNS per un record **MX** per individuare il server di posta del dominio destinatario e una risoluzione DNS per conoscere l'indirizzo IP associato e inoltrare la mail.

```
$ dig gmail.com MX
# non-terminal response with scores (lowest better)
;; ANSWER SECTION:
gmail.com. 3600 IN MX 20 alt2.gmail-smtp-in.1.google.com.
gmail.com. 3600 IN MX 5 gmail-smtp-in.1.google.com.
gmail.com. 3600 IN MX 40 alt4.gmail-smtp-in.1.google.com.

$ dig gmail-smtp-in.1.google.com
;; ANSWER SECTION:
gmail-smtp-in.1.google.com. 300 IN A 74.125.143.27
```

L'MTA mittente si connette all'MTA destinatario, il quale accetta solo le mail per il dominio che gestisce.

Il MUA potrebbe connettersi direttamente all'MTA destinatario e inviare mail di spam, per questo ci sono tecniche antispam per mitigare il problema.

Alla fine l'MTA destinatario consegna la mail al **Mail Delivery Agent (MDA)** che la memorizza nella mailbox del destinatario.

8.3.4 POP3 - Post Office Protocol

Protocollo text-based utilizzato dai MUA per recuperare le mail dai server di posta. Scarica le mail e le elimina dal server.

```
S: +OK POP3 server ready
# autenticazione
C: USER alice
S: +OK
C: PASS 12345pass
S: +OK alice maildrop has 2 messages (620 octets)
# status mailbox e lista nuovi messaggi
C: STAT
S: +OK 2 620
C: LIST
S: +OK 2 messages (620 octets)
S: 1 120
S: 2 500
S: .
# recupero msg 1 e cancellazione
C: RETR 1
S: +OK 120 octets
S: <the POP3 server sends message 1>
S: .
C: DELE 1
S: +OK message 1 deleted
C: QUIT
S: +OK POP3 server signing off (1 message left)
```

Oggi l'autenticazione POP3/IMAP è cifrata.

IMAP (Internet Message Access Protocol): consente la sincronizzazione delle mail tra client e server, mantenendo le mail sul server.

8.4 Tecniche antispam

Lo spam è una delle principali minacce alla sicurezza e alla privacy degli utenti di posta elettronica. Si tratta di messaggi indesiderati inviati in massa a un gran numero di destinatari.

Se lo spammer è un MUA, il suo MSA rileverà l'invio di molte mail e lo bloccherà. Se lo spammer finge di essere un MTA, gli altri MTA dovranno implementare delle tecniche per riuscire a rilevare lo spam.

Closed relay: relay locale, open relay solo previa autenticazione.

8.4.1 Tecniche IP-based

FQDN check

Avvio sessione HELO con FQDN valido (MTA) o un indirizzo IP [x.x.x.x] (MUA).

1. Risoluzione DNS: FQDN → IP
2. Reverse DNS: IP → FQDN
3. Confronto FQDN e controllo che l'IP ottenuto sia lo stesso che ha avviato la connessione.

Un MUA non ha un FQDN valido, quindi non supera il controllo.

Mail 8.5: Mail con FQDN check

```
1 Delivered-To: bob@mail.com
2 Received: from mail.ietf.org (mail.ietf.org. [50.223.129.194]) by mx.google.com ...
3 for <bob@mail.com>
4 ...
5 From: Alice <alice@mail.com>
6 To: Bob <bob@mail.com>
```

Nella mail 8.5 a riga 2, l'MTA che avvia la connessione si presenta come `mail.ietf.org` e l'IP associato è `50.223.129.194`.

```
1 # 1. Risoluzione DNS
2 $ dig mail.ietf.org
3 ;; ANSWER SECTION:
4 3 mail.ietf.org. 60 IN A 50.223.129.194
5
6 # 2. Reverse DNS
7 $ dig -x 50.223.129.194
8 ;; ANSWER SECTION:
9 194.129.223.50.in-addr.arpa. 3600 IN PTR mail.ietf.org.
```

L'FQDN e l'IP corrispondono, quindi l'MTA è verificato.

IP blacklisting

Dei server mantengono liste di indirizzi IP noti per inviare spam (Spamhaus).

Gli MTA verificano se l'IP mittente è in blacklist inviando una richiesta DNSBL (DNS block list) / RBL (Realtime block list) a tali server.

```
1 # Query DNSBL per 127.0.0.2 simile a un reverse lookup
2 $ dig 2.0.0.127.zen.spamhaus.org
3 # A record - codici di stato (127.0.0.x)
4 # Se la risposta e' vuota, l'IP non e' in blacklist
5 ;; ANSWER SECTION:
6 2.0.0.127.zen.spamhaus.org. 60 IN A 127.0.0.4
7 2.0.0.127.zen.spamhaus.org. 60 IN A 127.0.0.2
8 2.0.0.127.zen.spamhaus.org. 60 IN A 127.0.0.10
9
10 # Query TXT per dettagli sul motivo della blacklist
11 $ dig 2.0.0.127.zen.spamhaus.org TXT
12 ;; ANSWER SECTION:
13 2.0.0.127.zen.spamhaus.org. 60 IN TXT "Listed by XBL, ..."
14 2.0.0.127.zen.spamhaus.org. 60 IN TXT "Listed by SBL, ..."
15 2.0.0.127.zen.spamhaus.org. 60 IN TXT "Listed by PBL, ..."
16
17 # Verifichiamo mail.irtf.org
18 # Nessuna ANSWER - mail.irtf.org non e' in blacklist
19 $ dig 194.129.223.50.zen.spamhaus.org
20 ;; AUTHORITY SECTION:
21 zen.spamhaus.org. 10 IN SOA need.to.know.only. hostmaster.spamhaus.org. ...
```

Inoltre, i provider assegnano IP dinamici inserendoli preventivamente in blacklist per impedire che le connessioni domestiche vengano usate per inviare spam.

Greylisting

Un server legittimo rispetta le RFC SMTP e può ritardare la risposta o ritornare un errore temporaneo per motivi di congestione, gestione spam, ecc.

- Un MTA riceve una mail da un mittente sconosciuto.
- Risponde con un errore temporaneo (4**) e memorizza il suo IP in una whitelist
- Il client legittimo riprova e rimane in whitelist, mentre lo spammer passa al prossimo server target e finirà in **greylist**.

Questa tecnica aggiunge del delay alla prima mail inviata da un MTA legittimo. Inoltre, gli IP in whitelist vengono rimossi per inattività.

Gli MTA più usati sono sempre in whitelist (Gmail), mentre MTA usati poco possono essere soggetti a delay (MTA per password reset).

Gli spammer potrebbero implementare la ritrasmissione, riducendo l'efficacia della tecnica.

8.4.2 Tecniche DNS e Crypto-based

Comandi SMTP

-  | **HELO/EHLO:** Identifica il nome di dominio del server SMTP mittente.
-  | **MAIL FROM:** Mail a cui inviare i messaggi di errore.
-  | **From:** Indirizzo del mittente visualizzato al destinatario e usato per Reply-to.

Non si può imporre il match tra questi campi, perché l'MTA (HELO) può inviare mail per conto di domini diversi (FROM, MAIL FROM)

SPF - Sender Policy Framework

SPF è un protocollo che usa DNS per proteggere i comandi HELO e MAIL FROM.

Assume che un attaccante non possa modificare i record DNS.

Record SPF

Record TXT nel DNS del dominio mittente che specifica quali server sono autorizzati a inviare mail per quel dominio.

```
# Qualifiers: + (pass), - (fail), ~ (softfail, MTA decides), ? (neutral)
<FQDN> <ttl> IN TXT "v=spf1<version> ip4:<address/mask> ip6:<address/mask>
a:<domain> mx:<mailhost> include:<external_domain> <qualifier>all"
```

Esempi record SPF

```
# one IP allowed, others are forbidden
unive.it. 86400 IN TXT "v=spf1 ip4:17.18.7.120 -all"
```

```
$ dig ietf.org TXT
;; ANSWER SECTION:
ietf.org. 300 IN TXT "v=spf1 ip4:50.223.129.192/26 ip6:2001:559:c4c7::/48
a:ietf.org mx:mail.ietf.org ip4:192.95.54.32 include:spf.google.com ~all"
```

Procedura di verifica SPF

1. L'MTA destinatario fa una risoluzione DNS del dominio in MAIL FROM
2. Controlla il record SPF e verifica se l'IP MTA mittente è autorizzato
3. E' CONSIGLIATO fare lo stesso per HELO

Esempio

Mail 8.6: Mail header con SPF

```
1 Delivered-To: leonardo.maccari@unive.it
2 Received: by 2002:a9a:69c5:0:b0:299:9b06:7765 with SMTP id c5csp1571665lkg;
3 Wed, 6 Nov 2024 01:21:37 -0800 (PST)
4 Received: from mail.ietf.org (mail.ietf.org. [50.223.129.194]) by mx.google.com ...
5 for <leonardo.maccari@unive.it> (version=TLS1_3 cipher=TLS_AES_256_GCM_SHA384
   bits=256/256);
6 Wed, 6 Nov 2024 01:21:37 -0800 (PST)
7 Received-SPF: pass (google.com: domain of forwardingalgorithm@ietf.org designates
   50.223.129.194 as
   permitted sender) client-ip=50.223.129.194;
8 ...
9 ...
10 # [MAIL FROM]: @ietf.org
```

Nella mail 8.6, l'header a riga 7 è stato aggiunto dall'MTA di Google dopo aver controllato il record SPF del dominio `ietf.org`.

Esercizio - regola SPF

Scrivere una regola SPF per il dominio `ietf.org` che autorizzi `1.2.3.4`, `1.2.3.5` e gli host ammessi da `example.com`

```
ietf.org. 300 IN TXT "v=spf1 ip4:1.2.3.4 ip4:1.2.3.5 include:example.com -all"
```

8.4.3 DKIM - DomainKeys Identified Mail

Protocollo implementato dall'MSA per autenticare le mail.

Header DKIM

```
DKIM-Signature: v=<version>; a=<algorithm>; d=<domain>; s=<selector>;
                h=<signed headers>; bh=<body hash>; b=<signature>
```

```
DKIM-Signature: v=1; a=rsa-sha256; d=ietf.org; s=ietf1;
                h=Date:From:To:Subject; bh=gKK...uw=; b=ILdy...6E=;
```

Firma e verifica della mail

1. L'MSA genera un hash della mail (body + headers importanti)
2. Firma l'hash e lo aggiunge come header

L'MSA ha una coppia di chiavi asimmetriche. La `pubKey` è pubblicata nel DNS come **record TXT**, con dominio `selector._domainkey.domain`.

3. L'MTA destinatario recupera la `pubKey` con una query DNS e verifica la firma.

```
$ dig ietf1._domainkey.ietf.org TXT
;; ANSWER SECTION:
ietf1._domainkey.ietf.org. 300 IN TXT "k=rsa; p=MIG...QAB"
```

Questo metodo fornisce integrità e origin authentication (rispetto a `d=`) alla mail.

8.4.4 DMARC — Domain-based Message Authentication, Reporting & Conformance

Protocollo che usa SPF e DKIM per per validare l'autenticità delle mail e specificare una policy di gestione dei fallimenti.

Alignment check: il dominio in **From:** deve corrispondere a quello in SPF (MAIL FROM) o DKIM (d=).

	CHECK	FROM: (DMARC)	d= (DKIM)	MAIL FROM: (SPF)
Full Alignment	✓	@client.net	@client.net	@client.net
DKIM Only	✓	@client.net	@client.net	@sample.net
SPF Only	✓	@client.net	@sample.net	@client.net
Fail	✗	@client.net	@sample.net	@sample.net

Tabella 8.4: DMARC Alignment [9]

Return-Path: <rocket@sample.net>
Delivered-To: <groot@example.com>
Received: from ...
DKIM Signature: v=1; a=rsa-sha256; d=criminal.net;
s=april2023; q=dns/txt;
Date: Fri, 7 Apr 2023 15:09:21 +0000
From: "Rocket" <rocket@sample.net>
To: "Groot" <groot@example.com>
Subject: Blaster Needed!

Diagram description: Red boxes highlight 'sample.net' in Return-Path, From, and DKIM d=. A green arrow points from the Return-Path box to the From box. A red arrow points from the From box to the DKIM d= box.

Figura 8.10: SPF alignment [9]

Return-Path: <rocket@criminal.net>
Delivered-To: <groot@example.com>
Received: from ...
DKIM Signature: v=1; a=rsa-sha256; d=sample.net;
s=april2023; q=dns/txt;
Date: Fri, 7 Apr 2023 15:09:21 +0000
From: "Rocket" <rocket@sample.net>
To: "Groot" <groot@example.com>
Subject: Blaster Needed!

Diagram description: Red boxes highlight 'criminal.net' in Return-Path and 'sample.net' in DKIM d=. A red arrow points from the Return-Path box to the DKIM d= box. A green arrow points from the From box to the DKIM d= box.

Figura 8.11: DKIM alignment [9]

```

Return-Path: <rocket@sample.net>
Delivered-To: <groot@example.com>
Received: from ...
    DKIM Signature: v=1; a=rsa-sha256; d=sample.net;
    s=april2023; q=dns/txt;
Date: Fri, 7 Apr 2023 15:09:21 +0000
From: "Rocket" <rocket@sample.net>
To: "Groot" <groot@example.com>
Subject: Blaster Needed!

```

Figura 8.12: Full DMARC alignment [9]

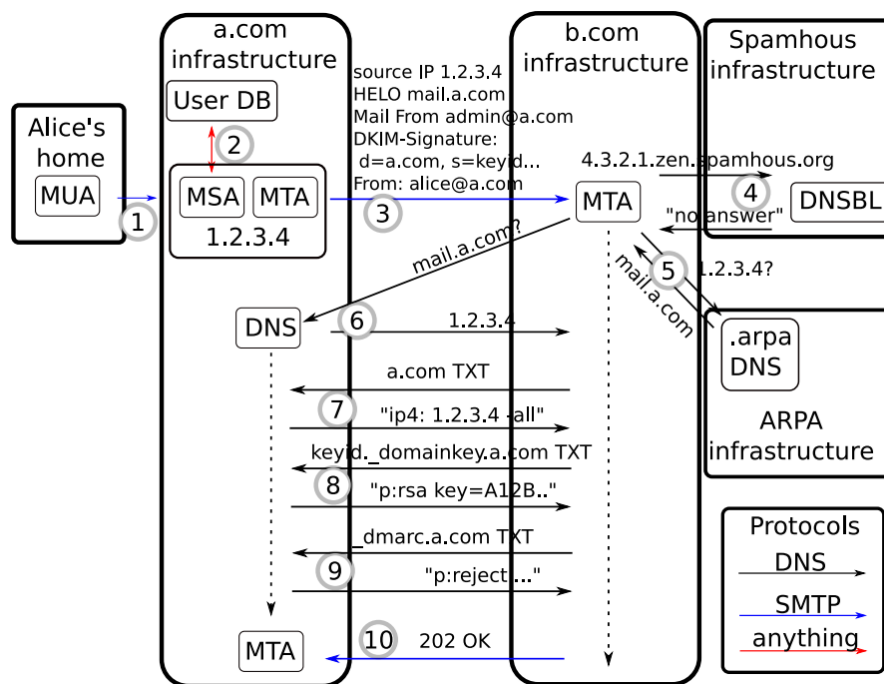
Se l'allineamento fallisce, l'MTA destinatario applica la policy DMARC specificata nel record TXT del dominio in **From**:

```

1 # p (policy): action when alignment fails - none (do nothing), quarantine (mark as
2   spam), reject (drop it)
3 # rua: email to send aggregate reports
4 $ dig _dmarc.ietf.org TXT
   _dmarc.ietf.org. 300 IN TXT "v=DMARC1; p=none;
   rua=mailto:dmarc_agg@vali.email,mailto:dmarc-report@ietf.org"

```

8.4.5 Risultato finale



Altri protocolli: BIM (loghi certificati), ARC (autenticazione per mail inoltrate).

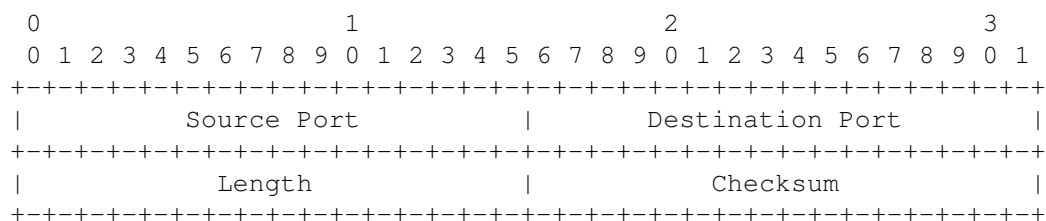
9 Protocolli di Trasporto e Sicurezza

9.1 TCP - UDP

9.1.1 UDP - User Datagram Protocol

Protocollo connectionless e unreliable, fornisce solo multiplexing ed error detection. SDU fino a 65467B (65535 - 8B header - 60B header IPv6).

Header UDP



Il checksum è calcolato sui campi dell'header UDP, uno pseudo-header IP (indirizzi IP, ...) e il payload.

Porte: campi di 16 bit (0-65535).

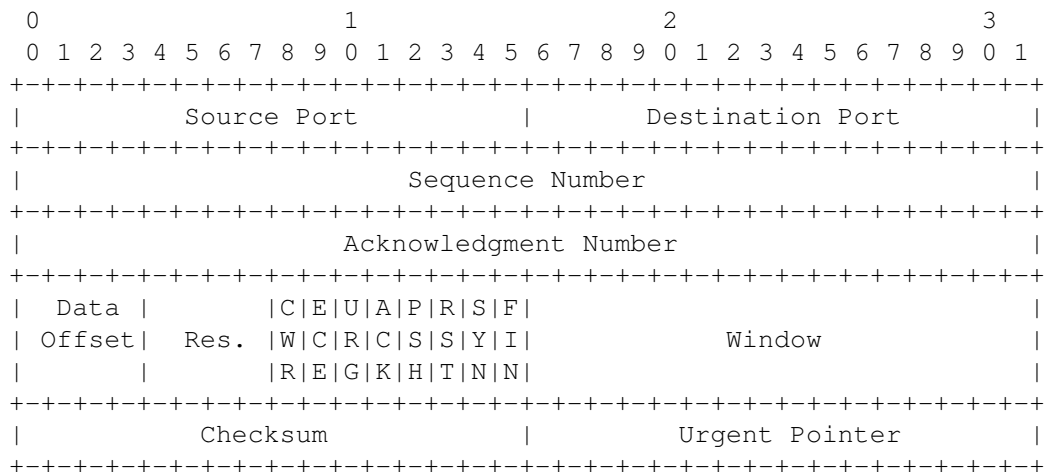
- 0-1023: privileged port numbers (well-known ports)
- 1024-49151: registered port numbers (protocolli, registrate da IANA)
- 49152-65535: ephemeral port numbers (usate dinamicamente dalle app)

9.1.2 TCP - Transmission Control Protocol

Protocollo connection-oriented e reliable.

Header TCP

- THL (TCP Header Length) / Data Offset (4 bit): lunghezza header TCP in parole di 32 bit
- Reserved bits, CWR, ECE: usati per gestire la congestione
- Window: dimensione finestra del mittente
- Checksum: calcolato su header TCP, pseudo-header IP e payload



- SYN: inizio connessione
- FIN: fine connessione
- RST: reset connessione
- ACK: ack valido

Il destinatario può bufferizzare i dati prima di passarli all'applicazione.

- PSH: dati da inviare subito all'app, rispettando l'ordine di arrivo.
- URG, urgent pointer: dati urgenti da inviare senza rispettare l'ordine (es. segnale di interruzione - CTRL-C).

Non sono usati, perché è l'applicazione che decide quando leggere dal buffer.

L'header TCP può essere esteso con opzioni: MSS (obbligatorio), Window Scaling, Timestamp (opzionali).

Connection Set-Up

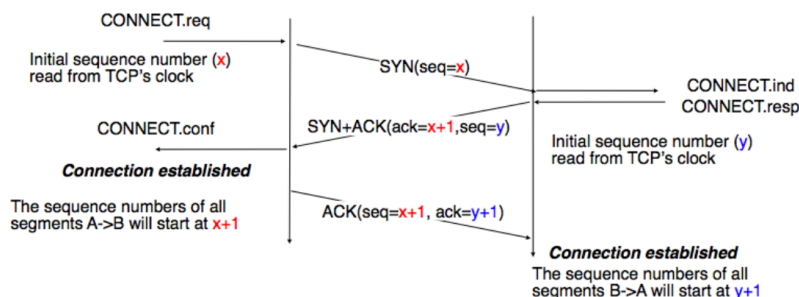


Figura 9.1: TCP connection set-up [1]

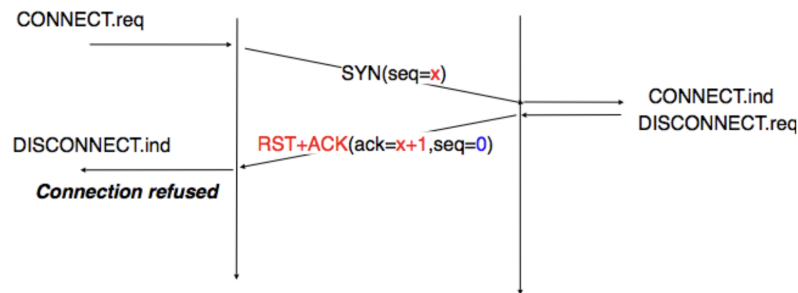
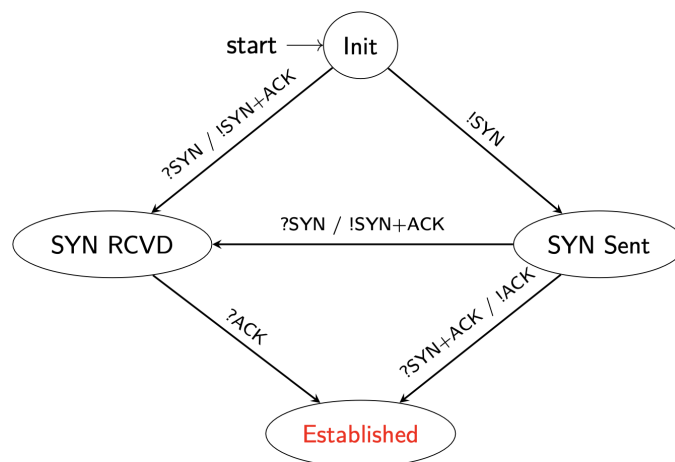


Figura 9.2: TCP connection refusal [1]



Here “?” means receive and “!” means send.

Figura 9.3: TCP connection start - FSM [1]

In figura 9.3, la transizione da **SYN SENT** a **SYN RCVD** avviene quando due dispositivi avviano la connessione contemporaneamente sulla stessa porta.

Durante questa fase, viene anche stabilito l'**MSS (Maximum Segment Size)**: dimensione massima del segmento TCP (escluso header TCP e IP) che il mittente è disposto a ricevere.

Di solito è calcolata come MTU (dimensione massima del frame per evitare frammentazione) - 40B (header IP + header TCP).

TCB - Transmission Control Block: stato mantenuto da entrambi gli endpoint per ogni connessione attiva.

Connection Release

Graceful release: entrambi gli endpoint inviano un segmento **FIN(x)**, finiscono di inviare dati e aspettano l'ack dell'altro.

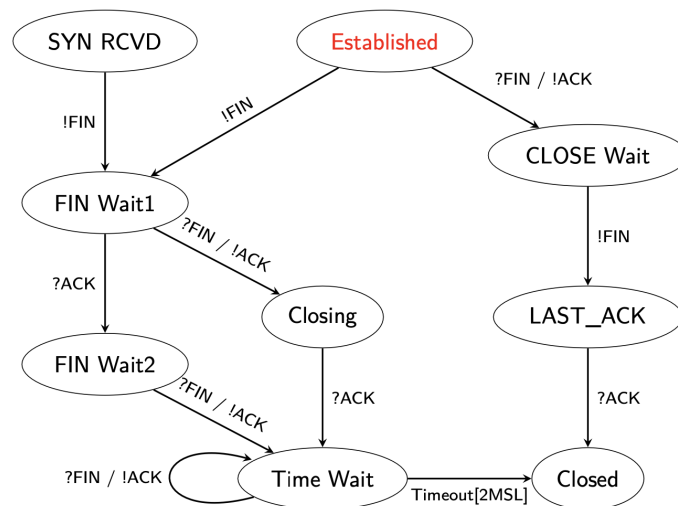


Figura 9.4: TCP connection stop - FSM [1]

Chi ha inviato tutti i dati e vuole terminare invia **!FIN**, ma può ancora ricevere dati. Deve anche assicurarsi di ricevere l'ultimo ACK **?FIN/!ACK** nello stato **TIME_WAIT** (2MSL: 4', 3.5*RTO in Linux).

Abrupt Release: un endpoint invia un segmento **RST**, l'altro termina la connessione immediatamente. (non rispondere **RST** a un **RST** e confermare l'ultimo seqNum ricevuto per evitare attacchi).

Esercizio

Memoria: $32GB = 2^{35}B$, TCB size: $131072B = 2^{17}B$.

Max connessioni simultanee: $2^{35}/2^{17} = 2^{18} = 262144$

Le connessioni durano 1', quante connessioni si possono accettare al secondo?

Una connessione rimane in memoria per 1+4', quindi si possono accettare $262144/(60*5) \simeq 874$ connessioni al secondo.

9.1.3 Reliability

Trasmissione go-back-n, destinatario con selective repeat e ACK cumulativi.

TCB

- **Identificazione:** (srcIP:srcPort, dstIP:dstPort)
- **Buffer:** sndBuff (unacked), rcvBuff (non letti)

Sending buffer

- **Stato:** (*TIME WAIT*, *ESTABLISHED*, ...)
- **MSS:** Maximum Segment Size

- **Controllo flusso:** `snd.next`, `snd.una`, `snd.wnd`

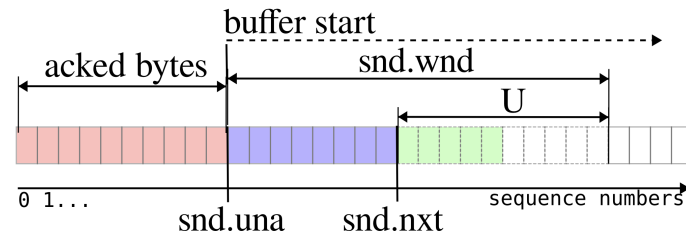


Figura 9.5: Sending buffer TCP

Scrittura dati e ricezione ack

```

1 write_sndBuff:
2     # 1 Check spazio sndBuff e scrivo al massimo min(MSS, rcv.wnd)
3     # 2 SeqNum del nuovo segmento = snd.next
4     # 3 Aggiorno snd.next += len(payload segmento TCP)
5     # 4 Mantengo i dati nel sndBuff in attesa di ack
6
7 ack_received:
8     # 1 Update snd.wnd = rcv.wnd nuova
9     # 2 Se ACK > snd.una -> rimuovo i dati dal sndBuff e aggiornno snd.una

```

Receiving buffer

- **Controllo flusso:** `rcv.next`, `rcv.wnd`

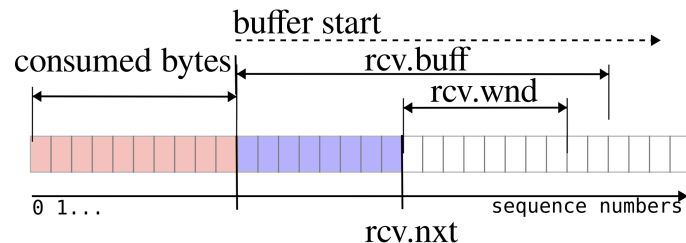


Figura 9.6: Receiving buffer TCP

Ricezione dati e invio ack

```

1 data_received:
2     # 1 Ricezione segmento e update rcv.next se SeqNum == rcv.next
3     # 2 Check se altri dati possono essere passati all'applicazione
4     # 3 Invio ACK e rcv.wnd

```

9.1.4 Sender Side

Quando inviare i dati? Appena disponibili, ma spreco capacità per inviare piccoli segmenti oppure aspetto segmenti pieni (MSS), ma il buffer potrebbe essere lento a riempirsi.

Algoritmo di Nagle

Usable Window (U) = `snd.una + snd.wnd - snd.nxt`: chunk di dati inviabili senza sfiorare la `sndWindow` (vedi figura 9.5).

```
1 # Send full TCP segment
2 if len(data) >= MSS and U >= MSS:
3     send one MSS-sized segment
4 # Send at least one segment per RTT
5 else:
6     if there are unacked data:
7         push data into sndBuff while waiting for acks
8     else:
9         send one TCP segment containing at most snd.wnd data
```

Nella prima parte dell'algoritmo 9.1.4, se il destinatario non invia gli ack, la finestra non viene aggiornata e U diminuisce fino a $U < MSS$.

L'algoritmo cerca di raggruppare i dati per inviare pacchetti grandi, ma aumenta il delay e lo rende inadatto in un contesto real-time.

Dimensione della finestra?

Quando inviamo i dati e riempiamo la finestra dopo rimaniamo in attesa di ack. Possiamo quindi inviare al massimo una finestra per RTT.

Il campo window è 16 bit, quindi la finestra massima è 65535B e il throughput massimo è $65535/RTT$.

TCP Window Scale Option: opzione facoltativa nell'header TCP per ingrandire la finestra, stabilito in fase di connection set-up.

La finestra effettiva è $rcv.wnd * 2^S$, $S \in 0, 14$.

Come gestire il Retransmission Time Out (RTO)?

Se alla fine di un RTT non arriva l'ack, dobbiamo ritrasmettere i dati, quindi RTO sarà basato su RTT stimato.

È difficile stimare RTT perché varia nel tempo e dipende dal carico di rete.

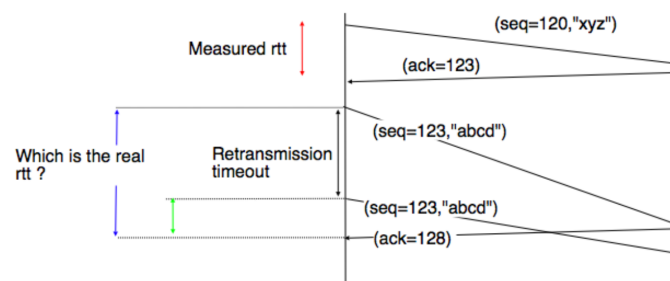


Figura 9.7: Errore di misura del RTT [1]

TCP Timestamps: opzione TCP per misurare RTT con precisione.

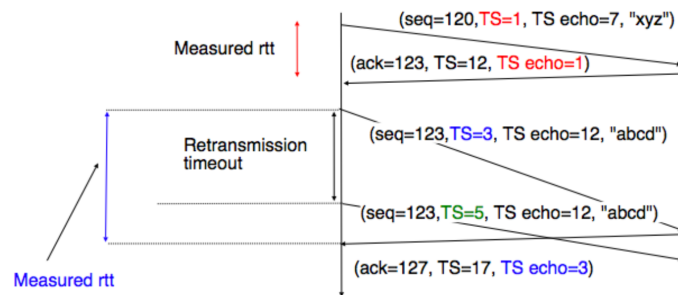


Figura 9.8: Misura del RTT con TCP Timestamps [1]

Inoltre, sequence number a 32 bit wrappano prima dell'MSL, ma con i timestamp riusciamo a capire se un segmento è troppo vecchio.

Smoothing RTT - Van Jacobson Algorithm

Exponential Moving Average (EMA): $\bar{s} = (1 - \alpha)\bar{s} + \alpha s_i$

Inizialmente:

$$\begin{aligned} srtt \text{ (smoothed RTT)} &= rtt \\ rttvar &= rtt/2 \\ rto &< 1s \end{aligned}$$

Dopo ogni misura del RTT:

$$\begin{aligned} srtt &= (1 - \alpha)srtt + \alpha rtt \\ rttvar &= (1 - \beta)rttvar + \beta |srtt - rtt| \\ rto &= srtt + 4 \cdot rttvar \end{aligned}$$

Con $\alpha = 1/8$ e $\beta = 1/4$.

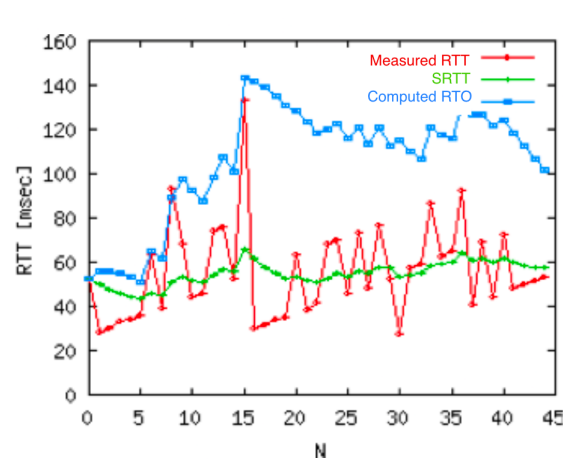


Figura 9.9: Stima RTO (adattato da [1])

Una volta stimato RTO, dobbiamo adattarlo alle condizioni di rete.

RTO exponential scaling: Raddoppiamo RTO a ogni fallimento e lo riportiamo al valore stimato a ogni successo.

Fast Retransmit: Il sender potrebbe ricevere ACK non ordinati, ma se ne riceve 3 duplicati, ritrasmette subito senza aspettare il timeout.

9.1.5 Receiver Side

Delayed ACKs: inviare gli ACK dopo N frame ricevuti o dopo un timeout.

Non funziona bene con l'algoritmo di Nagle, perché il sender aspetta l'ack per inviare e il receiver aspetta per inviare l'ack.

Selective ACKs

In TCP, il receiver può salvare dati out-of-order e può usare l'header opzionale **Selective ACK (SACK)** per confermare segmenti non consecutivi (max 3 intervalli).

Esempio: SACK: 100-200, 250-500, 800-900; ACK: 85 indica che sono stati ricevuti tutti i byte fino a 85 e gli intervalli 100-200, 250-500, 800-900.

9.2 Gestione della congestione

In una condizione di throughput massimo in una connessione TCP (rwnd/RTT), i router intermedi possono scartare dei pacchetti.

Inoltre se il receiver invia subito gli ack, $\text{snd.wnd} \simeq \text{rwnd}$.

Ma se ad esempio metà dei pacchetti vengono persi, i pacchetti effettivamente ricevuti saranno $\text{rwnd}/2 * \text{RTT}$

9.2.1 Congestion Window



Congestion Window: *cwnd*: nuova finestra. Il sender non può inviare più dati di $\min(\text{cwnd}, \text{snd.wnd})$.

Il throughput diventa cwnd/RTT e *cwnd* è inizializzata a un valore piccolo: MSS (1460B = MTU - 40B header).

slow-start threshold (ssthresh): stima dell'ultima *cwnd* che non ha causato congestione. Inizializzata a *snd.wnd*

9.2.2 Slow Start Algorithm

```
1 cwnd = MSS
2
3 At each RTT:
4   # slow start phase: exp growth
5   If cwnd < ssthresh:
6     cwnd *= 2
7   Else:
8     # congestion avoidance phase: linear growth
9     cwnd += MSS
```

9.2.3 Eventi di congestione

Mild congestion: 3 ACK duplicati. Il sender usa **fast retransmit**.

2 ACK potrebbero essere per pacchetti duplicati o out-of-order.

```
1 ssthresh = cwnd / 2
2 cwnd = ssthresh
```

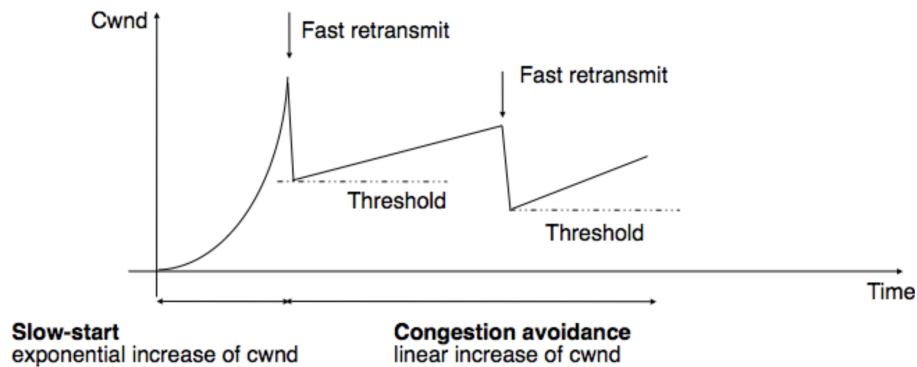


Figura 9.10: Mild Congestion [1]

Severe Congestion: RTO scaduto

```
1 ssthresh = cwnd / 2
2 cwnd = cwnd0
```

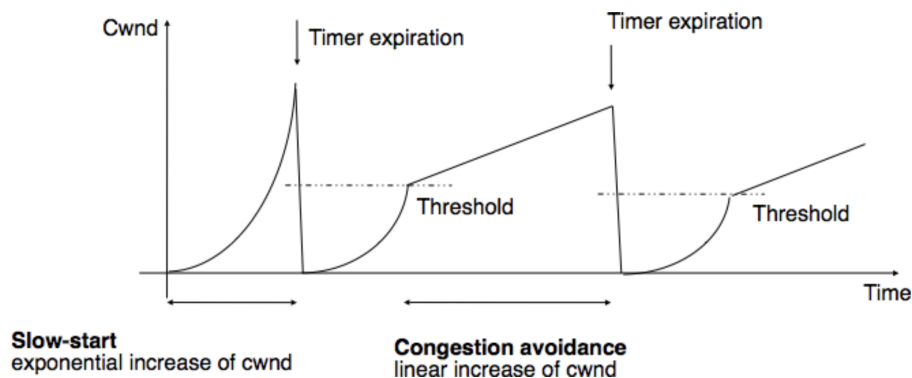


Figura 9.11: Severe Congestion [1]

ECN - Explicit Congestion Control: i router segnalano la congestione e la perdita di pacchetti settando dei bit nell'header.

Esistono varianti di algoritmi per la gestione della congestione: Reno, New Reno, CUBIC [10], BBR (Google).

9.2.4 Socket di rete

Un socket è un'interfaccia standard POSIX tra livello applicazione e livello trasporto. Permette di inviare e ricevere dati su una rete tramite syscall.

Proviamo a startare un server con socket in C e connetterci con un client [11]:

```
1 # SERVER
2 # SOCK_STREAM: TCP, SOCK_DGRAM: UDP
3 int lfd = socket(AF_INET, SOCK_STREAM, 0);
4 # servaddr: AF_INET (IPv4), porta 3000,
```

```

5 # INADDR_ANY (0.0.0.0: accetta connessioni su tutte le interfacce)
6 bind(lfd, &servaddr, sizeof(servaddr)); # associa indirizzo e porta
7 listen(lfd, 8);
8 int cfd = accept(lfd, &cliaddr, &len);
9 recv(cfd, buf, len, 0);
10 send(cfd, buf, n, 0);
11 close(cfd);
12
13 # CLIENT
14 int fd = socket(AF_INET, SOCK_STREAM, 0);
15 # servaddr: AF_INET, porta 3000, IP server
16 connect(fd, &servaddr, sizeof(servaddr));
17 send(fd, buf, len, 0);
18 recv(fd, buf, len, 0);
19 close(fd);

```

```

1 $ ./server
2 Server listening on port 3000...
3 Client connected.
4
5 $ ./client 127.0.0.1
6 Connected to server.

```

Proviamo a killare il server e startarlo immediatamente

```

1 ^C
2 $ ./server
3 Bind error: Address already in use
4
5 $ netstat -an | grep 3000
6 tcp4    0    0  127.0.0.1.3000    127.0.0.1.50132    TIME_WAIT

```

Il socket rimane in stato `TIME_WAIT` per assicurarsi che l'ultimo ACK sia arrivato (vedi Figura 9.4).

Per consentire al socket di riutilizzare la stessa porta, inseriamo `SO_REUSEADDR` dopo `socket()` e prima di `bind()`:

```

1 int opt = 1;
2 setsockopt(lfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

```

9.3 TLS - Transport Layer Security

A quale livello inseriamo la sicurezza? A basso livello cifriamo più informazioni, ma è più complesso, ad alto livello cifriamo meno informazioni.

Livello applicativo	Più semplice, ma le porte TCP restano esposte.
Livello trasporto	NAT e firewall non funzionano correttamente
Livello di rete	I router non sanno più instradare i pacchetti.
Livello datalink	Cifatura reclusa alla LAN.

Possiamo usare una VPN che cifra il livello di trasporto e di rete incapsulando gli header cifrati in header in chiaro, ma aggiunge overhead.

Il miglior compromesso tra usabilità e leak di informazioni è stato introdurlo sopra TCP.

9.3.1 SSL - TLS

SSL - Secure Sockets Layer: protocollo di sicurezza per il livello di trasporto, attualmente deprecato. La versione 3.0 è stata la base per **TLS**.

TLS fornisce servizi di sicurezza ai livelli superiori:

- **Origin authentication** con certificati
- **Segretezza:** cifatura a chiave simmetrica con chiave condivisa
- **Integrità:** HMAC con un'altra chiave condivisa

9.3.2 TLS handshake protocol

Negoziare chiavi e parametri di sicurezza unici per ogni connessione. Consente l'autenticazione delle parti coinvolte.

Proprietà: Connessione **privata** (cifatura) e **affidabile** (integrità).

Negoziazione critica: Si stabiliscono gli algoritmi di cifatura, hash e come scambiare le chiavi simmetriche.

Il protocollo fornisce una connessione sicura con:

- Autenticazione con PKE richiesta per almeno una delle due parti
- Negoziazione di un segreto condiviso sicuro e integro (anche da MiTM)

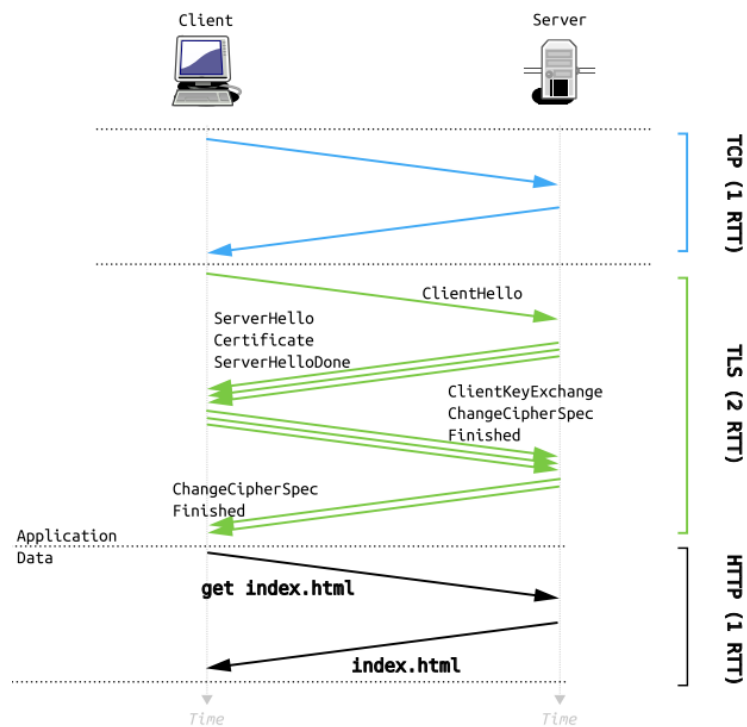


Figura 9.12: TLS handshake

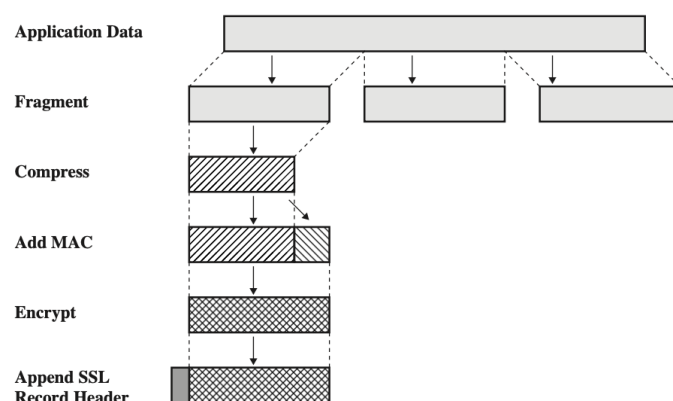
Primi 3 messaggi: 3-way handshake TCP, seguito dall'invio dell'intera catena di certificati e dei parametri di sicurezza. I primi dati arrivano dopo 4 RTT.

- **ChangeCipherSpec**: indica che i messaggi successivi saranno cifrati con i parametri negoziati
- **Finished**: primo messaggio cifrato, contiene un hash di tutti i messaggi precedenti per garantire l'integrità della negoziazione

mTLS - mutual TLS: autenticazione bidirezionale necessaria in sistemi *zero trust*. Servizi cloud nella stessa rete interagiscono tra di loro verificando i certificati a vicenda. Senza mTLS, il client si autentica con una password.

9.3.3 TLS record protocol

Security transformation applicate ai dati



Come mostrato sotto, TLS dipende da molte componenti, per le quali sono state trovate numerose vulnerabilità nel corso degli anni.

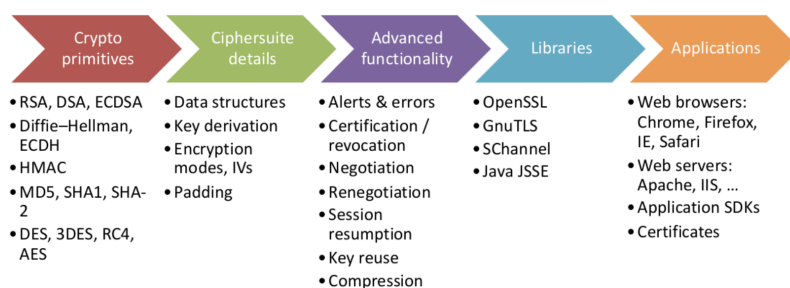


Figura 9.13: TLS

TLS 1.3 semplifica il protocollo: rimuove feature inutilizzate, semplifica l'handshake a 1 RTT e introduce **0-RTT Resumption** per ripristinare sessioni sicure.



Forward Secrecy: proprietà dei protocolli di comunicazione sicura che garantisce che la compromissione di una **privKey** a lungo termine non comprometta le **sessKey** passate.

Obbligatoria in TLS 1.3, richiede l'uso di **pubKey** effimere con DH, eliminate alla fine della sessione.

Protocollo 1 (RSA-based)

1. $B \rightarrow A : \text{Cert}_B, N_1$
2. $A : \text{verify } \text{Cert}_B, N_2$
3. $A \rightarrow B : \{R\}_{K_B}$
4. $A, B : K = \text{PRF}(R, N_1, N_2)$
5. $A, B : \text{AES/HMAC using } K$
6. $A \rightarrow B : \text{Password-based client authentication}$

Protocollo 2 (Ephemeral DH)

1. $B \rightarrow A : \text{Cert}_B$
2. $A : \text{verify } \text{Cert}_B$
3. $A \rightarrow B : m, g, n$ (ephemeral DH)
4. $B \rightarrow A : \{m, g, n, r\}_{K_B^{-1}}$
5. $A, B : K = m^b = r^a$
6. $A, B : \text{AES/HMAC using } K$
7. $A, B : \text{delete } a, b$
8. $A \rightarrow B : \text{Password-based client authentication}$

Il protocollo 2 fornisce forward secrecy, mentre il protocollo 1 no.

Inoltre, i meccanismi challenge/response non sono soggetti ad attacchi brute-force offline, perché con TLS è tutto cifrato.

9.3.4 Socket TLS

OpenSSL: Simple TLS Server[12]

```
1  TLS_SERVER_METHOD # all TLS versions supported
2
3  SSL_CTX_new # create new context object
4
5  SSL_CTX_use_certificate_file # load server certificate
6  SSL_CTX_use_PrivateKey_file # load server private key
7  configure_context(ctx) # set options
8
9  ssl = SSL_new(ctx) # create new SSL structure
10 SSL_set_fd(ssl, client_socket) # bind socket to SSL structure
11 if (SSL_accept(ssl) <= 0) # perform TLS handshake
12 SSL_write(ssl, reply, strlen(reply)) # send data
```

10 Protocolli di Rete

10.1 IPv4

Il livello di rete usa i servizi offerti dal livello datalink, che può essere point-to-point o LAN.

PPPoE, PPPoA: protocolli per lo scambio di frame tra due nodi. Può fornire autenticazione e segretezza.

(Rete DSL con router di casa collegato al provider)

LAN: rete locale in cui ogni nodo ha un indirizzo MAC unico. I frame sono indirizzati tramite indirizzo MAC o broadcast.

Un host connesso a una rete ha una **network interface card (NIC)** alla quale si può associare un indirizzo IP unico per la rete.

Esempio: 185.107.80.231 - 10111001.01101011.01010000.11100111

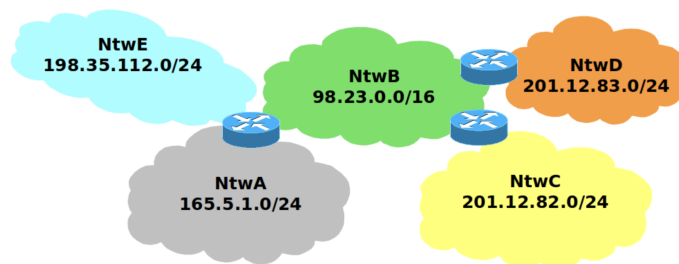


Figura 10.1: Subnet connesse da router

Subnet mask

Un indirizzo IP è composto dal prefisso **netid** e il suffisso **hostid**. La **subnet mask** indica quanti bit sono usati per il **netid**.

CIDR - Classless Inter-Domain Routing: metodo per allocare indirizzi IP con subnet mask di lunghezza variabile.

In passato, gli indirizzi IP erano divisi in classi

Classe	CIDR	Subnet Mask	Indirizzi
A	/8	255.0.0.0	2^{24}
B	/16	255.255.0.0	2^{16}
C	/24	255.255.255.0	2^8

Il **netid** è assegnato dalla IANA e ogni subnet ha degli indirizzi speciali:

Indirizzo di rete: [netid].0

Indirizzo di broadcast: [netid].1..1

Indirizzi speciali

255.255.255.255	Limited broadcast address: invia pacchetti a tutti i nodi della rete locale.
224.0.0.0/4	Indirizzi multicast: usati da gruppi di host. I primi 4 bit sono sempre 1110.
0.0.0.0/8	Indirizzo temporaneo usato come sorgente o per ascolto su tutte le interfacce.
127.0.0.0/8	Loopback interface: gestisce le comunicazioni interne sulla stessa macchina.
10.0.0.0/8 172.16.0.0/12 192.168.0.0/16	Classi private: indirizzi non instradabili su Internet, usati per reti locali interne.
169.254.0.0/16	Link-local (APIPA): indirizzo auto-assegnato d'emergenza.

Gateway

Assumiamo che gli host nella stessa subnet possano comunicare fisicamente tra loro, con ethernet o Wi-Fi, specificando l'indirizzo IP e MAC del destinatario.

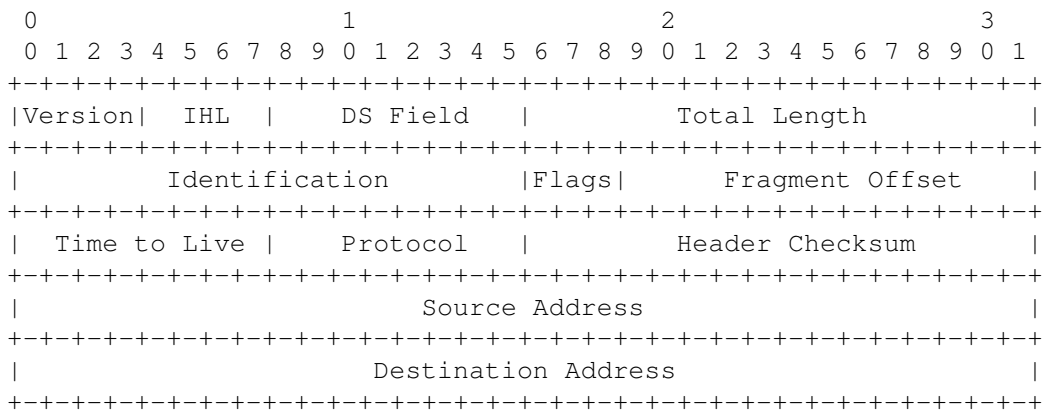
I router hanno più NIC, ognuna con un indirizzo IP per ogni subnet a cui sono connessi.

Ogni host ha un **default gateway** (router predefinito) che usa per comunicare all'esterno, specificando l'IP pubblico del destinatario e l'indirizzo MAC del gateway.

```
$ ip a # STATO INTERFACCE - "CHI SONO"
1: lo                -> 127.0.0.1/8 (Loopback)
2: enp0s31f6         -> Stato DOWN (Cavo Ethernet scollegato, nessun IP)
4: wlp0s20f3         -> Stato UP (Wi-Fi attiva)
   - inet            -> 192.168.26.212/24 (Il mio IP e la mia maschera)
   - link/ether-> b0:47:e9:8e:9e:c8 (Il mio indirizzo fisico MAC)

$ ip r # TABELLA ROUTING - "DOVE VADO"
default via 192.168.26.174 -> Gateway/Route
169.254.0.0/16 scope link -> Link-local APIPA
192.168.26.0/24 scope link -> Rotta Locale
   - src 192.168.26.212   -> Usa questo come mio IP mittente
```

10.1.1 Header IPv4

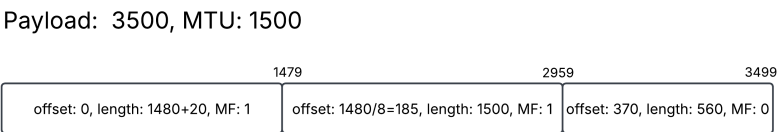
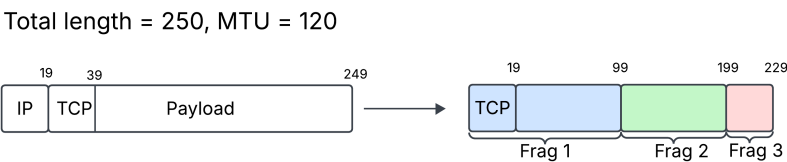


- Version: 4/6
- IHL - Internet Header Length in parole da 32 bit (5-15)
- Identification: id frammenti stesso pacchetto
- Flags: Evil bit, DF (Don't Fragment), MF (More Fragments)
- Fragment Offset in multipli di 8B (solo dati, header escluso)
- TTL: decrementato da ogni router
- Protocol: 1/ICMP, 6/TCP, 17/UDP

Frammentazione

I pacchetti con dimensione maggiore del MTU della rete devono essere frammentati. Ogni frammento ha lo stesso **Identification**, **length** del frammento stesso, **offset** diverso e il flag **MF** a 1, tranne l'ultimo frammento.

Esempi



Ogni frammento è un pacchetto indipendente, con un nuovo header IP da 20B.

10.1.2 Indirizzi pubblici e privati - NAT

A causa della scarsa disponibilità di indirizzi IPv4, è stato introdotto il NAT, che consente a più host di condividere un singolo indirizzo IP pubblico, riutilizzando indirizzi privati all'interno di una rete locale.

Ogni host ha un unico indirizzo IP privato all'interno della rete e comunica con l'esterno tramite un *border router* che traduce gli indirizzi privati in un unico indirizzo pubblico.

La rete privata 100.64.0.0/10 è usata dagli ISP quando non hanno indirizzi pubblici a sufficienza.

NAT - Network Address Translation

Tecnica che opera tra il livello di rete e di trasporto per mappare indirizzi IP privati e pubblici.

NAPT (Port): NAT che distingue le connessioni in base al numero di porta.

Il router mantiene una tabella di traduzione per ogni connessione attiva [Protocol, sIP, dIP, sPort, dPort] e aggiorna sIP quando un host vuole comunicare con l'esterno. Se due host comunicano con lo stesso dIP, dPort, sPort, il router assegna una sPort diversa per ogni connessione.

Incoming private NIC	Outcoming public NIC
[Protocol, sIP, dIP, sPort, dPort]	[Protocol, sIP, dIP, sPort, dPort]
UDP, 192.168.1.2, 8.8.8.8, 2000, 53	UDP, 1.2.3.4, 8.8.8.8, 2000, 53
TCP, 192.168.1.3, 4.3.2.1, 3000, 80	TCP, 1.2.3.4, 4.3.2.1, 3000, 80
TCP, 192.168.1.4, 4.3.2.1, 3000, 80	TCP, 1.2.3.4, 4.3.2.1, 3001, 80

Tabella 10.2: Tabella NAPT

Gli host esterni vedono solo l'indirizzo pubblico del router di una rete privata e non possono contattare direttamente gli host interni.

Due reti NAT non possono comunicare direttamente tra loro senza ulteriori meccanismi (Server relay, STUN, Port forwarding).

Svantaggi

- Necessità di uno stato (di solito si usa un firewall e non un router)
- Riassambla i frammenti per leggere le porte TCP
- Gestisce solo protocolli comuni (TCP, UDP, ICMP): livello di trasporto difficile da cambiare e senza possibilità di crittografia (NAT è un middlebox)

Match Forwarding table

I router instradano i pacchetti matchando l'indirizzo di destinazione nella **forwarding table** con il prefisso più lungo.



Longest Prefix Match: Un indirizzo IP matcha l'indirizzo nella routing table con lo stesso **network prefix** (**netid**) e la netmask più lunga (**bit-per-bit**). In caso di parità si guarda il costo del link.

IP		NIC
0.0.0.0/0	00000000.00000000.00000000.00000000	0
124.156.12.0/24	01111100.10011100.00001100.00000000	1
124.156.13.0/24	01111100.10011100.00001101.00000000	2
124.156.13.128/26	01111100.10011100.00001101.10000000	3
200.212.12.127/26	11001000.11010100.00001100.01111111	4
200.212.12.64/28	11001000.11010100.00001100.01000000	5

IP Destination		Match NIC
124.156.12.12	01111100.10011100.00001100.00001100	1
124.156.13.2	01111100.10011100.00001101.00000010	2
124.156.13.129	01111100.10011100.00001101.10000001	3
200.212.12.79	11001000.11010100.00001100.01001111	5
200.212.12.35	11001000.11010100.00001100.00100011	0
200.156.12.1	11001000.10011100.00001100.00000001	0

Esercizio

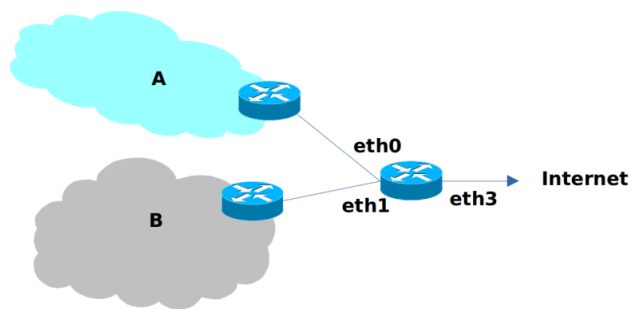
IP: 1.2.0.0/16, rete A: 14 host, rete B: 16 host

Necessitiamo di 30 indirizzi IP per gli host e 2 indirizzi di rete, 2 di broadcast e 2 per i router per le due reti A, B per un totale di 36 indirizzi IP. La netmask più piccola è /26.

Se affittiamo 1.2.255.192/26 (**netId** con tutti 1), assegnamo 1.2.255.192/27 alla rete A (17 indirizzi necessari) e 1.2.255.224/27 alla rete B (19 indirizzi necessari).

Tabella 10.3: Routing table

IP	NIC
0.0.0.0/0	eth3
1.2.255.192/27	eth0
1.2.255.224/27	eth1



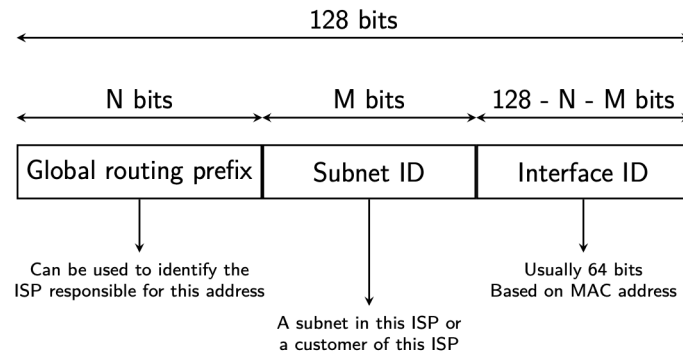


Figura 10.2: Indirizzo IPv6

L'Interface ID IPv6 è derivato dal MAC address, spesso tramite funzioni hash per garantirne la privacy.

Visto il formato degli indirizzi IPv6, possiamo avere 2^{64} subnet con 2^{64} indirizzi ciascuna, eliminando il bisogno di ottimizzare l'allocazione degli indirizzi.

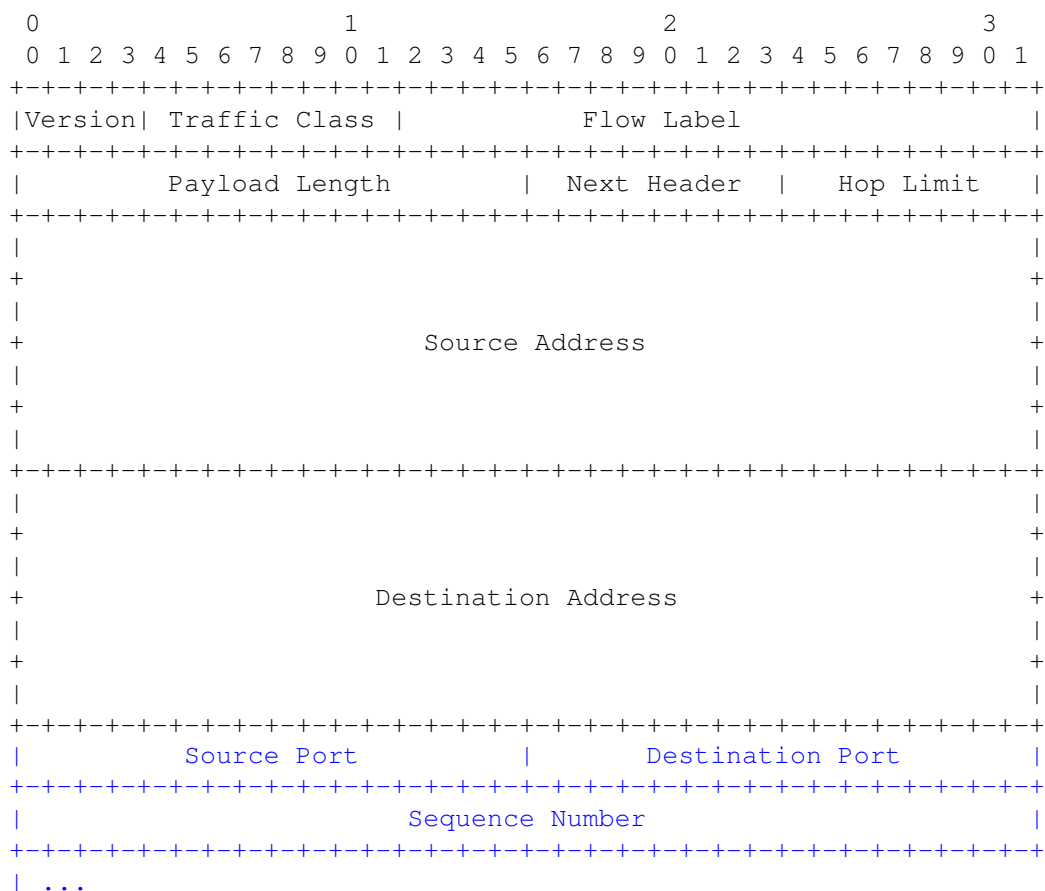
Blocchi di indirizzi IPv6 per il routing

- /32: ISP
- /48: Aziende
- /56: Piccole reti
- /64: Reti locali
- /128: Singolo dispositivo

Indirizzi speciali

- fc00::/7: Unique Local Unicast, come gli indirizzi privati IPv4
- ::1/128: Loopback address
- ff::/8: Multicast address
- fe80::/10: Link Local Unicast

10.2.3 Header IPv6



- **Next Header:** Tipo del prossimo header, protocol type in IPv4. Può creare una catena di header. In figura è riportato l'inizio dell'header TCP (6) in blu.
- **Hop Limit:** TTL in IPv4
- No checksum o meccanismi di frammentazione

LLU - Link Local Unicast

Indirizzo calcolato dall'host: **fe80::/64 (64b) + Interface ID (64b)** per comunicare nella LAN, address discovery e auto-configurazione.

Un router non può inoltrare pacchetti con indirizzi LLU. Quindi, invia periodicamente nella rete dei pacchetti **SLAAC (Stateless Address Auto-Configuration)** in broadcast contenenti il prefisso di rete in modo che gli host possano costruire il loro indirizzo (**netid + MAC**).

IPSec

Set di standard che implementa cifratura e autenticazione sui pacchetti IP, introdotta in IPv4 come opzionale, DEVE ESSERE SUPPORTATA in IPv6.

Frammentazione

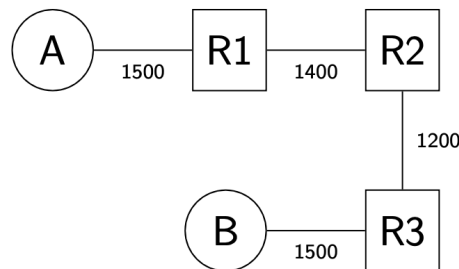
La frammentazione viene gestita solo dal mittente, non dai router intermedi, ma è comunque più efficiente dividere i dati in segmenti più piccoli a livello di trasporto.

In alcuni casi la frammentazione può essere utile (grandi risposte DNS con UDP, visto che UDP non ritrasmette i pacchetti persi).

10.2.4 ICMPv6

Simile a ICMPv4, ha `packet too big code` invece di `Fragmentation needed`.

Funzionamento - IPv4



A,B scelgono inizialmente $MSS = 1460$ e cercano di scoprire l'MTU minimo nel percorso (1160).

1. A invia un segmento di 1460B con $DF = 1$
2. R1 genera un pacchetto ICMPv4 con `code = 4` con l'MTU consentito
3. L'IP layer di A riceve il pacchetto e lo passa al livello TCP che abbassa l'MSS
4. Ripeti i passi 1-3 per il router R2 con un pacchetto di 1400B

IPv6 non usa il bit DF, in quanto i pacchetti troppo grandi producono sempre errori ICMPv6, ma il funzionamento è simile.

10.2.5 Intradomain Routing - OSPF

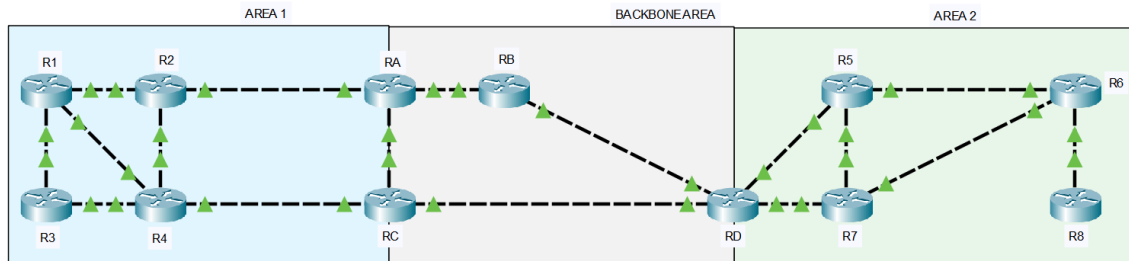
Le reti di grandi organizzazioni sono divise in varie subnet connesse da router.

Se la rete non è troppo grande (decine/centinaia di subnet/router), possiamo implementare il **control plane** con un **intradomain routing protocol** e un unico dominio amministrativo, usando un protocollo di routing come **link-state**, che reagisce velocemente ai cambiamenti della topologia, ma produce più overhead.

OSPF e IS-IS (Intermediate System to Intermediate System).

OSPF - Open Shortest Path First

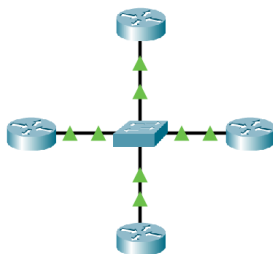
Rete divisa in aree con **router interni** (R1-R4, RB, R5-R8) e **router di confine** (RA, RC, RD).



Area 0 (Backbone): area centrale che aggrega i router di confine a cui tutte le altre aree devono connettersi.

I router interni si scambiano **LSP** per costruire una mappa della propria area e sapere come raggiungere l'area 0, mentre i router di confine scambiano **DV** per gestire il routing tra aree e ridurre l'overhead.

Rete full mesh virtuale



I router che in una rete usano LSP e sono connessi ad uno switch, pensano di essere in una rete full-mesh

Se però lo switch fallisce, i router perdono tempo a scoprire di essere isolati. OSPF consente di scegliere un **Designated Router** verso cui inviare gli LSP, che si occupa di ridistribuirli.

I link failures sono rilevati per perdita di messaggi **HELLO** o usando protocolli dedicati per monitorare lo stato dei link.

10.3 BGP - Border Gateway Protocol

10.3.1 Autonomous Systems (AS)



Autonomous System (AS): collezione di prefissi di instradamento IP connessi e gestiti da un singolo ente o dominio amministrativo

(Collezioni di reti, ciascuna con un proprio protocollo di instradamento interno, gestito dalla stessa entità: ISP, università, azienda, ecc.)



Stub AS: clienti finali che non forniscono servizi di instradamento ad altri AS. Sono detti **single-homed** se connessi a un solo AS, **multi-homed** se connessi a più AS.



Transit AS: connessi a più AS e forniscono servizi di instradamento.

La IANA assegna un numero unico a ciascun AS.

Connessioni tra AS

Gli AS gestiscono prefissi IP e devono comunicare tra loro per instradare i pacchetti.

Point of Presence (POP): Posti fisici in cui un AS si connette ad altri AS.



Private Peering: connessioni dirette tra due AS. I proprietari pagano l'infrastruttura e non sempre rendono pubbliche le informazioni di instradamento.



Internet Exchange Point (IXP): data-center condiviso in cui gli AS mettono i loro POP per connettersi ad altri AS.



Transit Agreement: accordo commerciale tra AS A e B in cui B si offre di connettere A ad internet

Prevede una tassa o può essere un **peering agreement** (gratuito).

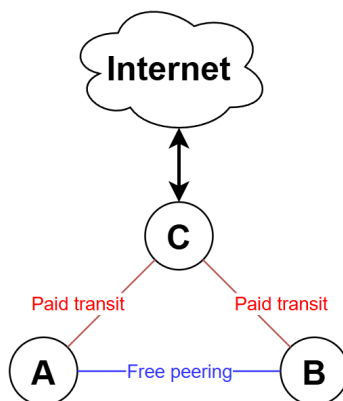


Figura 10.3: Transit Agreement tra AS

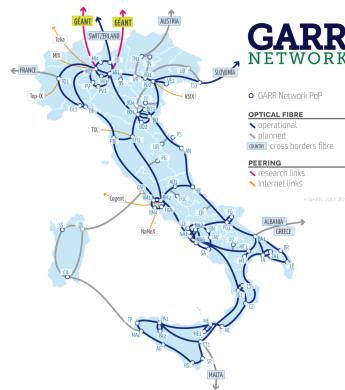


Figura 10.4: GARR - network italiano per università e ricerca

10.3.2 Propagazione dei prefissi BGP

BGP gestisce il routing tra AS (inter-domain routing) e ogni AS deve avere almeno un router che supporti BGP.

OSPF annuncia tutti i prefissi che conosce, mentre BGP annuncia solo i prefissi che un AS è autorizzato a pubblicizzare.

- customer $\rightarrow$ provider: `myPrefixes + childrenPrefixes`
- provider $\rightarrow$ customer: `ANY`
- shared-cost peering: `myPrefixes + childrenPrefixes`

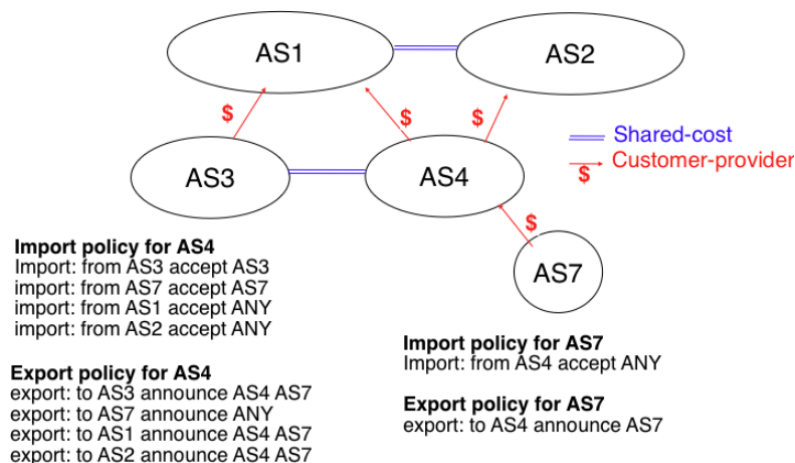


Figura 10.5: Annunci BGP tra AS

Esempio di setup BGP

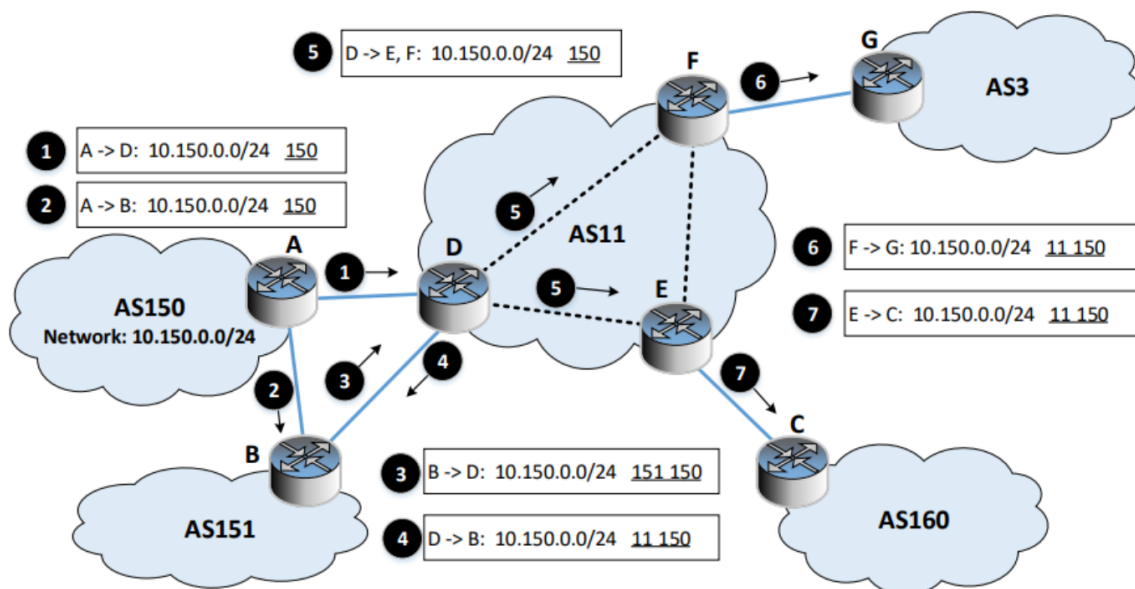


Figura 10.6: Setup rotte BGP

Gli AS annunciano i prefissi ai router vicini, includendo il proprio AS nel percorso.

- Nel punto 5, il router D non aggiunge nulla al path, perché fa parte dello stesso AS di E,F.

Inoltre:

- Un AS non annuncia i prefissi al router dal quale li ha ricevuti (split horizon, evita single link loop)
- I router annunciano l'intero path per raggiungere una destinazione e non accetta path in cui è già presente (**Path Vector Protocol**, evita multi-link loop)

Differenze BGP - DV

- Esporta l'intero path, non solo la distanza
- Invia aggiornamenti solo quando ci sono cambiamenti, con info solo sui prefissi interessati

10.3.3 BGP - Dettagli del protocollo

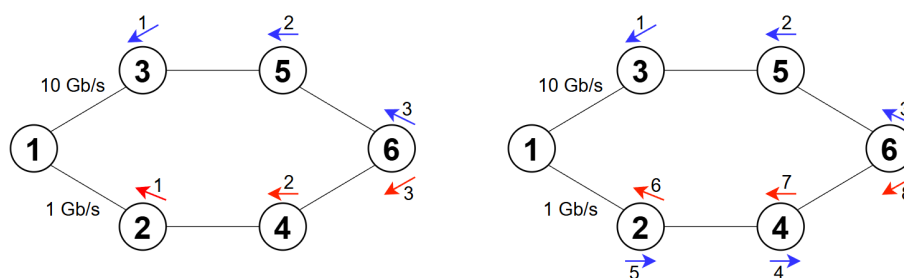
Due router BGP stabiliscono una connessione **TCP:179** per scambiarsi messaggi:

- OPEN (negoiazione sessione)
- UPDATE (annunci prefissi) (no crittografia)
- KEEPALIVE (mantiene viva la connessione)

- NOTIFICATION (errori)
- ROUTE-REFRESH (richiesta riannuncio prefissi).

Path prepending

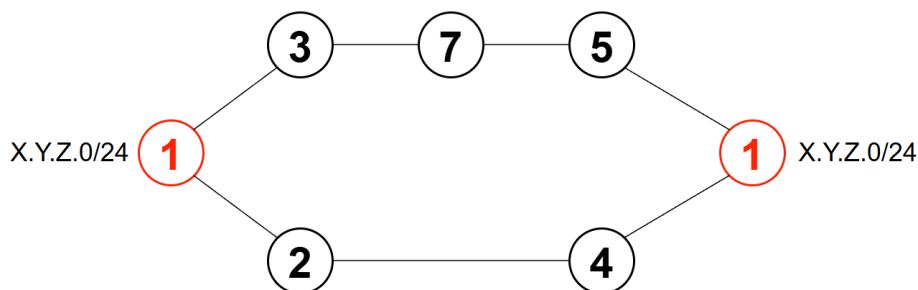
Strategia per influenzare la selezione del percorso in BGP, aggiungendo ripetizioni del proprio AS nel path annunciato. Questo rende il percorso meno preferibile per gli altri AS, poiché BGP tende a scegliere il percorso con il minor numero di AS attraversati.



AS 1 vuole forzare il traffico verso il link a 10 Gbps, quindi annuncia X.Y.Z.0/24 con path 1,1,1,1,1,1 ad AS 2

In figura, le frecce indicano il numero di AS attraversati, prima e dopo l'annuncio.

BGP Anycast



In figura, AS 1 ha due POP che annunciano lo stesso prefisso. Possono quindi esserci più host su internet con lo stesso indirizzo IP. Ciò fornisce ridondanza e bilanciamento del carico, ma un AS può cambiare le sue decisioni di routing nel tempo, rendendo complicato gestire le sessioni (es. connessioni TCP).

Per questo, è più comune per servizi request-response stateless come DNS.

11 Standard del Livello Data Link e Fisico

11.1 Ethernet - IEEE 802.3

11.1.1 Livello fisico

Tecnologia standard per connettere dispositivi in una LAN cablata, che opera a livello fisico e data link.

Il livello fisico definisce le caratteristiche dei cavi, connettori e segnali elettrici.

Doppino intrecciato: coppie di fili di rame intrecciati per ridurre interferenze elettromagnetiche.

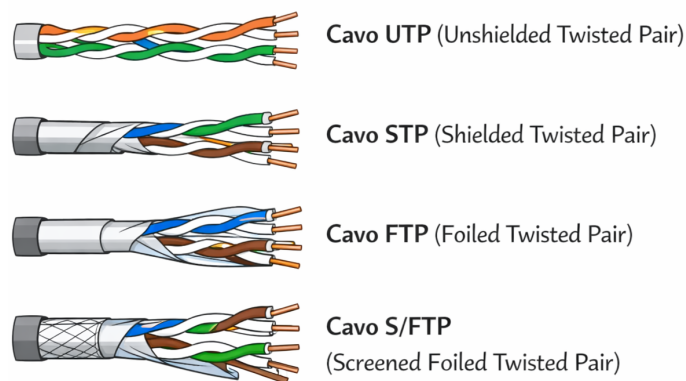


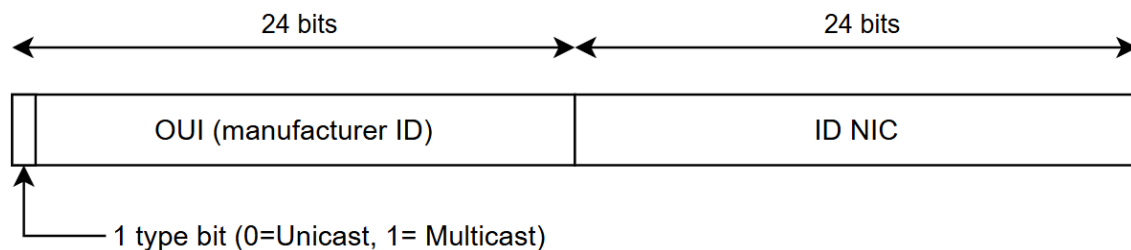
Figura 11.1: Cavi Ethernet

11.1.2 Livello Data Link

A livello data link, Ethernet offre servizi di consegna dei frame unicast, multicast e broadcast con una certa affidabilità.

Indirizzo MAC

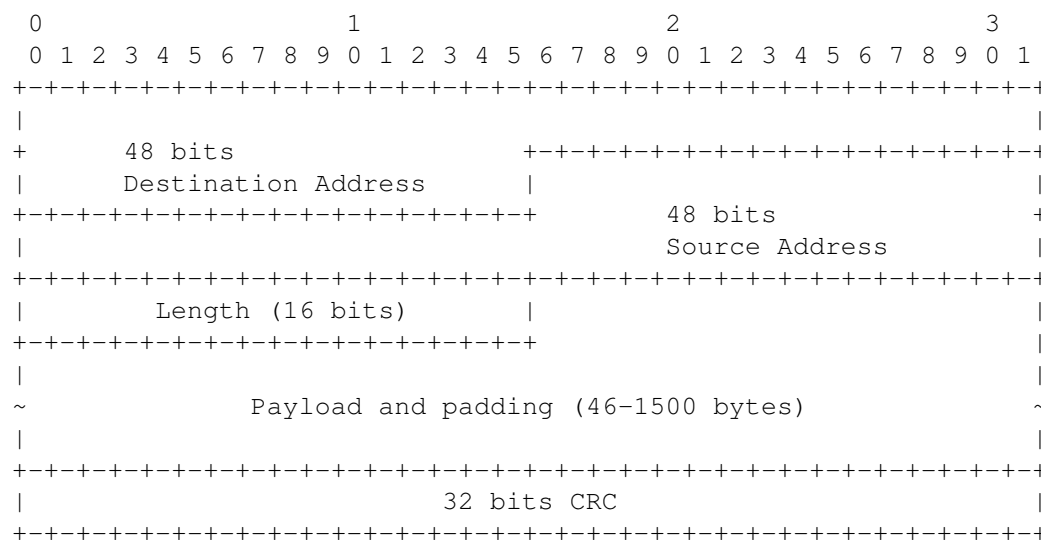
Identifica la NIC a livello data link. È un indirizzo fisico univoco di 48 bit, rappresentato in formato esadecimale.



Es. 01:80:C2:00:00:08 è multicast (1° byte dispari)

Gli OUI (Organisation Unique Identifier) sono tutti unicast.

Header Frame Ethernet



L'header è processato in hardware dalla scheda di rete. La NIC legge subito il **Destination Address** e decide se accettare il frame. Il campo **CRC** è posto alla fine per verificare il checksum senza salvarsi dei dati.

Il campo **EtherType** specifica il protocollo del livello superiore. E' stato sostituito da **length** con l'aggiunta dell'header LLC (Logical Link Control), ora caduto in disuso.

Switch ethernet

Un singolo link può essere esteso per creare una rete più grande usando:

- **Hub:** repeater che inoltra i frame ricevuti a tutte le porte.
- **Switch:** dispositivo che impara gli indirizzi MAC dei dispositivi collegati per inoltrare i frame solo alla porta corretta.

Algoritmo backward learning

Algoritmo usato da uno switch per costruire una tabella di inoltra degli indirizzi MAC.

```
1 # Arrivo di un frame F sulla porta P al tempo t
2
3 # Update table
4 src, dst = F.srcMAC, F.dstMAC
5 T[src] = (P, t)
6
7 # Unicast forwarding
8 if isUnicast(dst) and dst in T:
9     forwardFrame(F, T[dst][0])
10    return
11
12 # multicast or unknown unicast
13 for p in Ports - {P}:
14     forwardFrame(F, p)
```

Periodicamente, lo switch rimuove gli indirizzi MAC non usati per un certo tempo.

STP - Spanning Tree Protocol

Protocollo usato in reti con più switch per evitare loop a livello data link.

I frame ethernet non hanno un TTL per impedire loop infiniti.

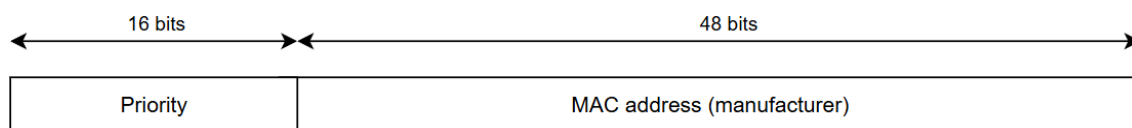


Figura 11.2: Bridge ID

Con STP, ogni switch ha un Id unico da 64-bit, e si elegge un **root bridge** con l'ID più basso, solitamente vicino al router di confine.

E' un protocollo distribuito che richiede una fase di convergenza affinché tutte le porte raggiungano un determinato stato.

- **root**: porta verso lo switch root
- **designated**: porta che inoltra frame
- **blocked**: porta che non inoltra frame per evitare loop

La convergenza è raggiunta quando lo switch root ha tutte le porte **designated** e tutti gli altri switch hanno esattamente una porta **root** e le altre **designated/blocked**.

Periodicamente, ogni switch invia un frame **BPDU (Bridge Protocol Data Unit)** su tutte le porte in multicast, da non inoltrare.

$$< Root_{id}, Cost, Sender_{id}, p >$$

I BPDU sono confrontati usando l'ordinamento lessicografico: prima il *Root_id*, poi *Cost*,...

Quando uno switch A riceve un BPDU da B, calcola $c = Cost + C_{AB}$ e il **root priority vector** per la porta q , sulla quale è stato ricevuto il BPDU:

$$V_q = \langle Root_{id}, c, Sender_{id}, p, q \rangle$$

```

if V_q < self.BPDU:
    self.Root_id = B
    q.state = root
    self.cost += C_AB
else:
    # blocked evita i loop
    q_state = (self.BPDU < BPDU_q) ? designated : blocked

```

Port State	Receive BPDU	Send BPDU	Handle Data
Blocked	✓	✗	✗
Root	✓	✗	✓
Designated	✓	✓	✓

Tabella 11.1: Stati delle porte STP

Gli switch mantengono un *age* per ogni BPDU ricevuto, e se non ne riceve uno nuovo entro un timeout, ricalcolano lo stato delle porte.

Esercizi [13]

11.1.3 ARP - Address Resolution Protocol

Trova l'indirizzo MAC associato a un indirizzo IP all'interno di una LAN.

Ogni host mantiene una **tabella ARP** che mappa indirizzi IP a indirizzi MAC.

Risoluzione indirizzo IP → MAC:

1. Host mittente invia una **ARP request** in broadcast con i suoi indirizzi IP e MAC e l'indirizzo IP del destinatario.
2. Host destinatario risponde con una **ARP Reply** in unicast, fornendo il proprio indirizzo MAC.

11.1.4 DHCP - Dynamic Host Configuration Protocol

Assegna dinamicamente indirizzi IP e altre configurazioni di rete agli host in una LAN.

Architettura client-server, iniziata dal client su **UDP (server:67, client:68)**.

1. Il nuovo host invia un **DHCP Discover** in broadcast con:
srcIP: 0.0.0.0, srcMAC: NIC_MAC, clientID: NIC_MAC

2. Un server DHCP risponde con un **DHCP Offer** con:
`clientID`, IP offerto e subnet mask, `leaseTime`, `serverIP`
3. Il client risponde con un **DHCP Request** in broadcast, con `ServerIP` nel body, in modo che un qualsiasi server DHCP possa processare la richiesta.
4. Un server DHCP risponde con un **DHCP ACK** in unicast, confermando l'assegnazione dell'IP e fornendo altre configurazioni di rete (gateway, DNS).

11.2 Wi-Fi - IEEE 802.11

11.2.1 Livello fisico

Standard per reti locali wireless (WLAN) che opera a livelli fisico e MAC: modulazione, encoding, accesso al mezzo.

La banda utilizzabile (ISM) è limitata dalle normative ed è suddivisa in canali che possono essere aggregati. Aggregando i canali si ha più banda, ma aumenta il rischio di interferenze con reti vicine.

11.2.2 Livello MAC

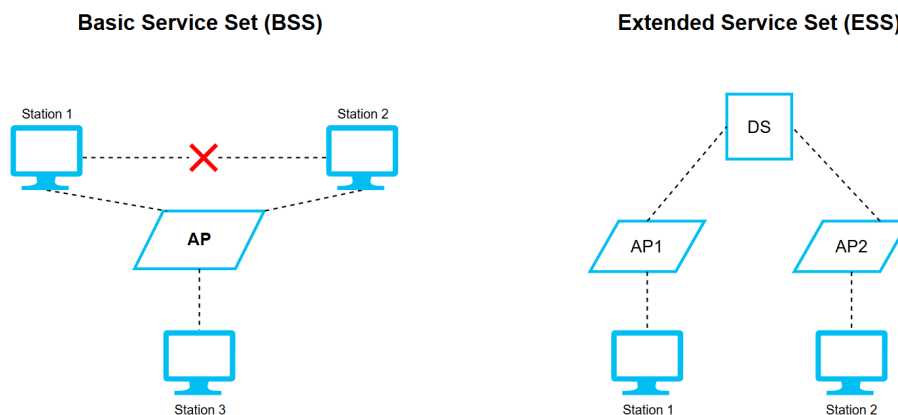
Esistono vari tipi di frame:

- **Management:** gestione comunicazione (auth, assoc, beaconing)
- **Control:** controllo flusso di dati (ACK, request to send, clear to send)
- **Data:** dati cifrati e autenticati

BSS - Basic Service Set: singola rete con Access Point e stazioni connesse, che comunicano solo tramite l'AP.

- Identificata da un **BSSID** (MAC address AP) e può scegliere un **SSID**.

BSS connesse insieme formano un **ESS (Extended Service Set)** tramite un DS (Distribution System), condividendo lo stesso SSID e segmento IP (stessa rete).



L'AP distribuisce **frame beacon** periodicamente per pubblicizzare la rete, contenenti informazioni sul BSS.

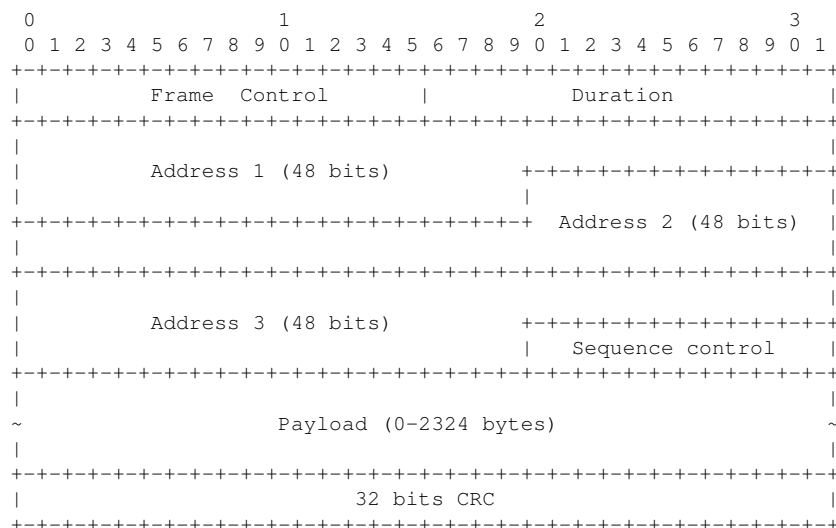
Una nuova stazione, per connettersi, deve effettuare:

1. Scanning

- **Passive scan:** la stazione ascolta per un breve intervallo ogni canale, in attesa di un beacon

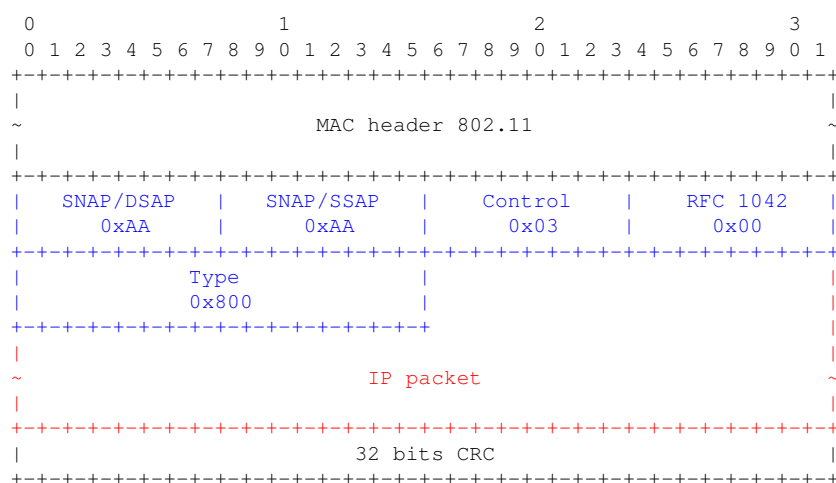
- **Active scan:** la stazione invia un frame **probe-request** su ogni canale e l'AP risponde con un **probe-response**
2. **Autenticazione e Associazione** effettuate una sola volta per creare un canale sicuro tra client e un AP dell'ESS

MAC Header 802.11



- **Frame Control:** tipo frame e opzioni (Type, Subtype, From/ToDS, Retry)
- 3 indirizzi: AP, mittente, destinatario
- **seq control:** id per rilevare ritrasmissioni

I **pacchetti IP** sono incapsulati nei frame MAC grazie a un **header LLC**



11.2.3 Access Control

Il livello MAC gestisce l'accesso al mezzo condiviso (canale radio), affinché le stazioni non trasmettano contemporaneamente.



Collisione: Situazione in cui una radio riceve due o più frame allo stesso momento, rendendoli illeggibili.

Le collisioni si verificano al ricevitore. Il mittente sa se un frame è stato ricevuto correttamente solo se riceve un ACK.



Failed Transmission: Si verifica quando il mittente non riceve un ACK per un frame inviato entro un timeout.

Carrier sensing: un terminale ascolta il canale prima di trasmettere, per sentire se è occupato.



CSMA/CA: Carrier Sensing Multiple Access / Collision Avoidance: protocollo di accesso al mezzo usato in 802.11 per ridurre le collisioni.



Channel state: stato del canale wifi: **idle** (libero) o **busy** (occupato)

L'obiettivo è massimizzare l'efficienza, mantenendo il canale occupato, ma i terminali hanno bisogno di sincronizzarsi e inviare messaggi di controllo.

PCF - Point Coordination Function

Meccanismo di accesso centralizzato. L'AP coordina l'accesso al canale (TDMA-like centralizzato).

DCF - Distributed Coordination Function

Meccanismo di accesso casuale. Tutti i nodi competono per l'accesso al canale (ALOHA-like, CSMA/CA).

```
1 # DIFS/SIFS: DCF/Short Interframe Space
2
3 transmit():                                receive(frame):
4     if channel.state is idle:                compute CRC
5         wait DIFS                             if CRC is correct:
6         if channel.state is idle:             wait SIFS
7         send(frame)                           send(ACK)
```

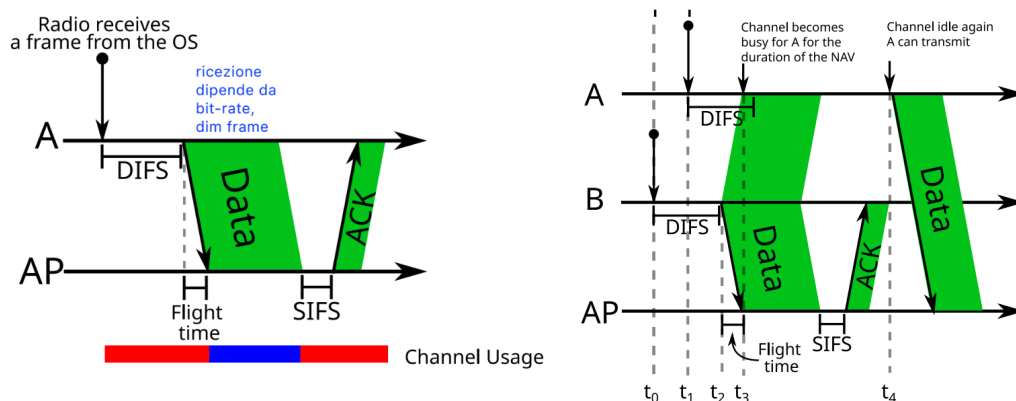


Figura 11.3: DCF - trasmissioni senza collisioni con uno e due trasmettitori

NAV - Network Allocation Vector: frame header che indica per quanto tempo il canale sarà occupato. I terminali nel raggio di trasmissione possono controllare il NAV e spegnere la radio per risparmiare energia.

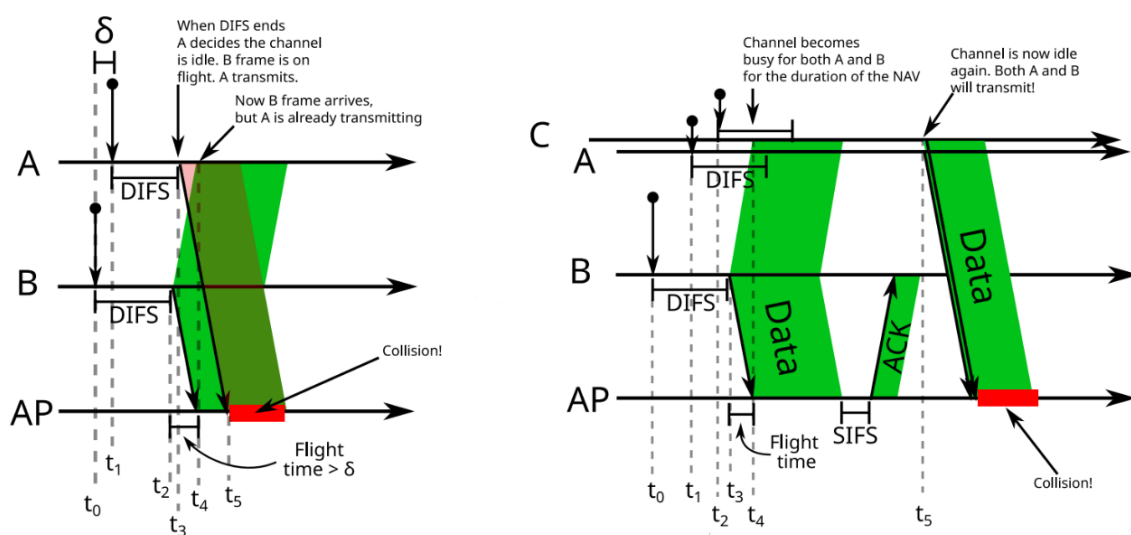


Figura 11.4: DCF - trasmissioni con collisione

Definiamo $\delta = t_1 - t_0$ (Figura 11.4 a sx), ma se δ è piccolo ($< \text{flight time}$) si ha una collisione al ricevitore.

Nella figura a dx, la collisione non è dovuta al δ piccolo, ma al NAV che sincronizza i terminali A e B.

Per evitare ciò, quando il canale ritorna idle, il terminale aspetta un tempo casuale $[0, CW]$ e se il canale rimane libero per tutto il tempo, trasmette. Altrimenti, sospende il conto alla rovescia e lo riprende quando il canale torna libero.

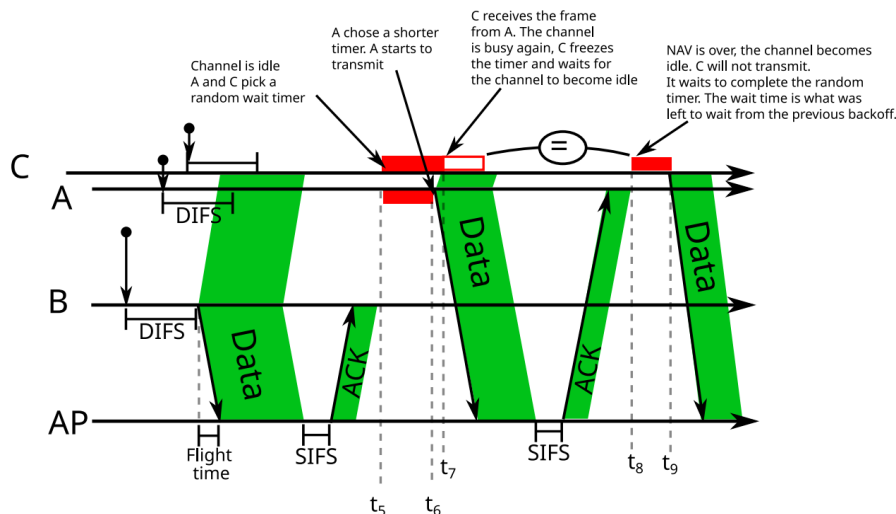


Figura 11.5: DCF - Backoff

Dopo ogni trasmissione fallita, la **CW** viene raddoppiata per ridurre la probabilità di collisione. Se ha successo, **CW** torna al valore iniziale, altrimenti, dopo un certo numero di tentativi, dropa il frame.

Questa strategia, chiamata **Binary Exponential Backoff**, riduce le collisioni, ma può causare ritardi eccessivi perché i terminali attendono troppo prima di ritentare.

11.2.4 Meccanismo RTS/CTS

Hidden Node Problem: due terminali sono fuori portata tra loro, ma possono comunicare con l'AP. Non possono fare carrier sensing.

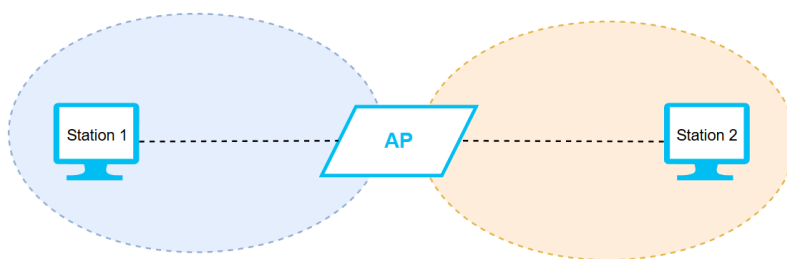


Figura 11.6: Hidden Node Problem

Soluzione

1. Se un terminale vuole comunicare, invia un frame **RTS - Request To Send** all'AP, contenente il **NAV**
2. L'AP risponde con un frame **CTS - Clear To Send** (dopo **SIFS**), contenente il **NAV**

Tutti i terminali ricevono almeno il **CTS**, essendo in portata con l'AP, e sanno che il canale è occupato per la durata indicata nel **NAV**.

Utile per frame grandi, in quanto RTS/CTS aumenta il tempo di attesa prima di trasmettere e diminuisce il bit-rate. I frame RTS possono collidere, ma essendo piccoli, ciò è meno probabile.

11.2.5 Stima bit-rate

Il bit-rate effettivo dipende dall'ampiezza del canale, modulazione, potenza del segnale, ... Quindi, non possiamo sapere in anticipo il bit-rate, ma ogni dispositivo cercherà di usare il miglior **MCS** disponibile.



MCS - Modulation and Coding Scheme: Schema di modulazione e codifica per adattare il bit-rate alle condizioni del canale.



Adaptive Coding and Modulation: Tecnica che adatta dinamicamente il MCS in base alle condizioni del canale.

Parte III

Appendici

Glossario

AP: Access Point

DOS: Denial of Service

DV - DVR: Distance Vector - Distance Vector Routing

LSR - LSP: Link State Routing - Link State Packet

Ordine lessicografico

Criterio di ordinamento tra stringhe basato sul confronto tra caratteri in posizione corrispondente, simile all'ordinamento alfabetico.

Siano due sequenze $A = (a_1, a_2, \dots, a_n)$ e $B = (b_1, b_2, \dots, b_n)$.

Si ha $A < B \iff$ nella prima posizione i in cui $a_i \neq b_i$, risulta $a_i < b_i$.

PPoE: Point-to-Point over Ethernet

PPPoA: Point-to-Point over ATM

RT: Routing Table

Spanning Tree

Sia $G = (V, E)$ un grafo non orientato e connesso con $|V| = n$. Uno **spanning tree** è un sottografo $T = (V, E')$:

1. T è connesso e aciclico (albero)
2. T include tutti i vertici di G
3. $|E'| = n - 1$

Bibliografia

- [1] Olivier Bonaventure. *Computer Networking: Principles, Protocols and Practice, 3rd Edition*. <https://github.com/cnp3/ebook>.
- [2] Neso Academy. *Computer Networks: Checksum in Computer Networks*. URL: <https://www.youtube.com/watch?v=AtVWnyDDaDI>.
- [3] R. Braden, D. Borman e C. Partridge. *Computing the Internet Checksum*. RFC 1071. Set. 1988. URL: <https://www.rfc-editor.org/rfc/rfc1071>.
- [4] Wikimedia Commons. *Common datacenter interconnect topology*. URL: https://commons.wikimedia.org/wiki/File:Common_datacenter_interconnect_topology.png.
- [5] Dan Boneh. «Twenty Years of Attacks on the RSA Cryptosystem». In: *Notices of the American Mathematical Society* 46.2 (1999), pp. 203–213. URL: <https://crypto.stanford.edu/~dabo/papers/RSA-survey.pdf>.
- [6] Stack Exchange. *Decrypt an RSA message when it's encrypted by same modulus*. Mathematics Stack Exchange. 2018. URL: <https://math.stackexchange.com/questions/2730675/>.
- [7] Internet Security Research Group (ISRG). *Let's Encrypt CA Hierarchy*. 2024. URL: <https://letsencrypt.org/certificates/>.
- [8] Amazon Web Services. *Certificate Authority Hierarchy*. AWS Private CA User Guide. Accessed: 2026-01-30. Amazon Web Services, Inc. 2024. URL: <https://docs.aws.amazon.com/privateca/latest/userguide/ca-hierarchy.html>.
- [9] *DMARC Alignment*. URL: <https://dmarcadvisor.com/alignment/>.
- [10] GeeksforGeeks. *TCP Congestion Control Algorithms: Reno, New Reno, BIC, CUBIC*. URL: <https://www.geeksforgeeks.org/computer-networks/tcp-congestion-control-algorithms-reno-new-reno-bic-cubic/>.
- [11] Andrew T. Campbell. *Socket Programming*. Dartmouth College. URL: <https://www.cs.dartmouth.edu/~campbell/cs60/socketprogramming.html>.
- [12] OpenSSL Project. *Simple TLS Server*. https://github.com/openssl/openssl/wiki/Simple_TLS_Server.
- [13] Leonardo Maccari and Filippo Orsi. *Routing Exercise Generator - Spanning Tree Protocol Exercises*. URL: <https://github.com/leonardomaccari/routing-exercise-generator/tree/testing-STP>.